

Masterarbeit

Auf dem Weg zum autonom fahrenden Fahrrad: Eine videogestützte Regelung des Weges mit Webbots

Amine Karabila

Sommersemester 2025

1 Abstract

Diese Masterarbeit beschäftigt sich mit der Entwicklung einer kombinierten Regelungsstrategie für ein autonom fahrendes, selbstbalancierendes Fahrrad, basierend auf den Ergebnissen dreier vorangegangener Forschungsarbeiten. Ziel dieser Arbeit ist die erfolgreiche Integration einer PID-Balanceregulung mit einer MPC-basierten Fahrtrichtungsregelung, die mithilfe eines YOLO-Modells zur visuellen Spurdetektion ergänzt wird. Zur Validierung und Optimierung der kombinierten Regelung wurde die Simulationsumgebung Webots verwendet, wodurch die Robustheit und Funktionalität der entwickelten Regelungsalgorithmen realitätsnah überprüft werden konnten. Im Rahmen der Arbeit wurde eine zweistufige Regelungsarchitektur umgesetzt, welche die präzise Balancierung des Fahrrads mittels eines PID-Reglers und die zuverlässige Spurverfolgung sowie Navigation mithilfe eines MPC-Reglers sicherstellt. Die gewonnenen Simulationsergebnisse bestätigen die Machbarkeit und Effizienz des entwickelten Ansatzes und legen damit eine solide Grundlage für die zukünftige Implementierung der Regelung in einem realen autonomen Fahrrad.

Contents

1	Abstract	2
2	Einleitung	5
2.1	Problemstellung	5
2.2	Stand der Technik	5
2.3	Bisherige Arbeiten	6
2.3.1	Anna Ranz (2021) – Hardware-Grundlagen	7
2.3.2	Amine Karabila (2022) – Erste funktionierende Regelung	7
2.3.3	Jonah Zander (2023) – Optimierte PID-Balanceregung	8
2.3.4	Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung	8
3	Grundlagen	10
3.1	PID-Regelung – Prinzip und Eignung für das Balance-Problem	10
3.2	MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem	11
3.3	YOLO-Grundlagen und Segmentierung	12
4	Webots-Simulationsumgebung	14
4.1	Überblick	14
4.2	Struktur der Simulationsumgebung	15
4.3	Physikmodell: Open Dynamics Engine (ODE) in Webots	17
4.4	Sensoren	17
4.4.1	Inertialmesseinheit (IMU)	17
4.4.2	Kamera (Videosensor)	19
4.4.3	Motorsensoren (Inkrementalgeber)	21
4.5	Aktuatoren	22
4.6	Controller	26
4.7	Integriertes Konfigurationssystem	27
5	Kombinierte Regelung	30
5.1	Portierung des Balance-PID (Zander)	30
5.2	Portierung des Fahrtrichtungs-MPC (Giray)	32
5.3	Strategien der Spurführung	36
5.3.1	YOLO-Segmentierung (YOLOv8)	36
5.3.2	Fallback-Strategie (ROI-basiert)	37

6	Ergebnisse	39
6.1	Fahrrad Datenblatt	39
6.2	Teststrecken	40
6.2.1	Kurvenfahrt	40
6.2.2	S-Kurve	45
6.2.3	Gerade Strecke mit Seitenwind	49
6.2.4	Gerade Strecke mit Höhenunterschied	53
6.3	Rechenzeit & Echtzeitfähigkeit in Webots (Vergleich zum Realfahrrad)	59
7	Grenzen der Arbeit	62
8	Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)	63
9	Fazit	64
10	Literaturverzeichnis	65

2 Einleitung

2.1 Problemstellung

In diesem Kapitel wird die zentrale Herausforderung dieser Arbeit präsentiert. Ohne regelnden Eingriff kippt ein Fahrrad im Stand oder bei niedriger Geschwindigkeit. Erst durch die Kombination aus gyroskopischen Effekten der rotierenden Räder, dem Nachlauf der Lenkgeometrie und einem aktiven Regler lässt sich eine stabile Fahrt realisieren. Die vorliegenden Arbeiten verfolgen daher das Ziel, ein autonomes fahrendes Fahrrad zu entwickeln, das sich sowohl selbst balancieren als auch eine vorgegebene Strecke verfolgen kann. Hierfür wurde eine hochrealistische Simulationsumgebung in Webots aufgebaut, die eine duale Reglerarchitektur abbildet. Ein schneller Balance-Controller, implementiert in C, stabilisiert den Rollwinkel mit einer Abtastzeit von 2 ms (500 Hz) und gibt einen Lenkwinkel an den Antrieb weiter. Parallel dazu extrahiert ein Vision-Controller in Python mit niedrigerer Frequenz (20 Hz) Informationen aus Kamerabildern, plant eine Trajektorie und überträgt entsprechende Lenkwinkel- und Geschwindigkeitsbefehle. Die Herausforderung besteht darin, diese beiden Regelschleifen so zu koppeln, dass das Fahrzeug stabil bleibt und zugleich auf visuelle Reize reagiert. Darüber hinaus soll die simulierte Physik so realitätsnah sein, dass die Ergebnisse auf reale Fahrversuche übertragbar sind.

Abgrenzung zu realitätsnahen und simulationsbasierten Ansätzen Im Gegensatz zu experimentellen Untersuchungen an realen Fahrradplattformen (z. B. Arbeiten von Zander [31] und Karabila [19]) liegt der Fokus dieser Arbeit auf einer vollständig simulationsbasierten Methodik unter Nutzung von Webots. Frühere Simulationsstudien (z. B. Zander [31], Karabila [19]) berücksichtigen dynamisch variierende Geschwindigkeiten, etwa aufgrund von Gefälle oder Bremsmanövern. In der hier vorliegenden Simulation hingegen wird die Geschwindigkeit als konstant angenommen. Auch die Sensorik und Regelungshardware unterscheiden sich wesentlich: Reale Versuchsaufbauten (etwa bei Karabila [19]) nutzten einen BNO-005 IMU-Sensor zur Erfassung von Eulerwinkeln sowie PWM-basierte Motoransteuerung. Die Webots-Simulation ermöglicht hingegen direkten Zugriff auf Zustandsvariablen, ohne den Einsatz realer Hardwarekomponenten oder deren Signalverarbeitung. *Zusammenfassend zeichnet sich diese Arbeit durch ihre rein simulationsbasierte Herangehensweise, die Annahme konstanter Geschwindigkeit, den Verzicht auf adaptive Regler sowie den Ausschluss hardwarebedingter Schnittstellen (Sensorik, PWM, ...) aus.*

2.2 Stand der Technik

Die Regelung eines selbstbalancierenden Fahrrads basiert auf zwei zentralen Grundlagen:

- (i) einem hinreichend genauen dynamischen Modell, das die wesentlichen Fahr- und Stabilitätseigenschaften beschreibt, und
- (ii) einer abgestimmten Reglerarchitektur, die sowohl die Aufrechterhaltung des Gleichgewichts als auch die zuverlässige Spurführung gewährleistet.

Auf der Modellseite etablierten Meijaard et al. den Benchmark der linearisierten Whipple-Gleichungen, der bis heute als Referenz für Analyse, Identifikation und Reglersynthese dient [22]. Ergänzend zeigte Kooijman et al., dass Selbststabilität *ohne* gyroskopische oder Caster-Effekte ¹ möglich ist,

¹„Caster-Effekte“ entstehen, wenn der Aufstandspunkt des Vorderrads hinter der Lenksäule liegt, wodurch das Rad sich durch die Bewegung selbständig in Fahrtrichtung ausrichtet :contentReference.

womit die Interaktion von Massenverteilung, Nachlauf und Lenkkinematik als zentrale Gestaltungsgrößen belegt wurde [20]. Diese Ergebnisse motivieren regelungstechnische Ansätze, die das Prinzip „*Steer into the Fall*“ explizit nutzen und die Lenkung als primären Stellgrad einsetzen.

Für die aktive Balanceregung über die Lenkung liegt mit Andreo et al. eine LPV-Formulierung² vor, die die geschwindigkeitsabhängige Dynamik berücksichtigt und Stabilität in einem definierten Geschwindigkeitsbereich nachweist [1]. Damit wird ein wichtiger Praxisaspekt adressiert, da Fahrräder ihre Vorwärtsgeschwindigkeit ständig ändern und eine rein LTI-Betrachtung³ nur lokal gültig ist. Als Alternative wurden Balanceregler mit Reaktionsrädern untersucht. Kanjanawanishkul vergleicht LQR- und nichtlineare MPC-Ansätze unter Berücksichtigung von Aktuatorsättigungen und zeigt, dass bereits einfache Quadratikregler bei Stand- oder Niedriggeschwindigkeit Stabilisierung ermöglichen, jedoch mit zusätzlicher Masse und höherem Energiebedarf verbunden sind [18]. Ein dritter Ansatz, der in der Robotik verbreitet ist, nutzt Control-Moment-Gyroskope (CMG). Park und Yi belegen experimentell, dass ein scissored-pair-CMG störende Quermomente kompensiert und so eine robuste Rollstabilisierung erlaubt, die geeignet für stationäre oder sehr langsame Fahrzustände ist, allerdings mit erheblichem mechanischem Aufwand [25].

Für die gekoppelte Aufgabe aus Balance *und* Spurführung hat sich eine Kaskadenarchitektur bewährt. Eine schnelle innere Balanceschleife (PID/LQR) und eine langsamere äußere Spurführungsschleife (häufig MPC) wurden von Persson et al. entworfen. Der äußere MPC optimiert dabei unter harten Nebenbedingungen (Roll-, Lenk- und Heading-Grenzen sowie Positionsbeschränkungen) die Soll-Rollwinkel und damit die Kursführung, während ein innerer PID die Soll-Rollwinkel über die Lenkrate stabilisiert [27]. Die Autoren zeigen auf realitätsnahen Strecken (OpenStreetMap-Go-Kart-Track), dass die Kaskade enge Radien und variable Geschwindigkeiten meistert und zugleich eine saubere Trennung der Zeitskalen ermöglicht. Dieser Aufbau ist unmittelbar anschlussfähig an *do-mpc*-basierte Implementierungen, da Optimierungs-, Nebenbedingungs- und Prädiktionshorizonte explizit parametrierbar sind.

Parallel zur Reglerseite hat sich die Umfeldwahrnehmung stark weiterentwickelt. Ein prominenter Systemdemonstrator ist das Tsinghua-„Tianjic“-Fahrrad, das Kameraperzeption (Objekterkennung/-tracking), Sprachkommandos, Hindernisvermeidung und Balance auf einem eingebetteten Chip integriert [26]. Die Studie ist weniger eine methodische Neuentwicklung der Fahrdynamik, zeigt aber überzeugend, dass visuelle Spurführung und Balanceregung *in Echtzeit* auf kompakten Plattformen zusammengeführt werden können, somit ein wichtiger Befund für kamerabasierte Spurhaltung.

2.3 Bisherige Arbeiten

Die im vorherigen Abschnitt dargestellten Forschungsarbeiten zeigen, dass sowohl klassische als auch moderne Regelungsverfahren prinzipiell geeignet sind, ein selbstbalancierendes Fahrrad zu stabilisieren und mit Trajektorienfolgeregelungen zu kombinieren. Während sich diese Studien überwiegend auf externe Forschungsgruppen beziehen, existieren auch an der Goethe-Universität Frankfurt eine Reihe aufeinander aufbauender Abschlussarbeiten, die wesentlich zur Entwicklung des hier untersuchten Systems beigetragen haben.

²LPV (Linear Parameter-Varying) beschreibt Systeme, deren Dynamik von zeit- oder zustandsabhängigen Parametern beeinflusst wird und erlaubt so eine erweiterte Stabilitätsanalyse über verschiedene Betriebsbereiche hinweg.

³LTI (Linear Time-Invariant) beschreibt Systeme mit zeitlich unveränderlichen Parametern; diese Annahme ist nur lokal gültig, wenn die Geschwindigkeit des Fahrrads konstant bleibt.

2.3.1 Anna Ranz (2021) – Hardware-Grundlagen

Die Bachelorarbeit von Anna Ranz markierte den praktischen Ausgangspunkt des Projekts und schuf eine erstmals vollständig lauffähige Experimentalplattform, auf der regelungstechnische Konzepte an realer Hardware untersucht werden konnten [28]. Zentrales Element ist ein *BeagleBone Black* als Steuereinheit, der den Lenkantrieb über einen *MD49*-Controller (Motor: *EMG49*) via *UART* ansteuert und die Inertialmessdaten einer *BNO055*-IMU über *I²C* erfasst. Ergänzt wird die Architektur durch einen Nabenmotor am Hinterrad, der eine konstante Vorwärtsfahrt sicherstellt. In dieser Kombination standen erstmals mechanische Plattform, Sensoren und Aktoren mit klar definierten Schnittstellen zur Verfügung, um Stabilisierung und Lenkregelung systematisch zu evaluieren.

Im Rahmen der Inbetriebnahme implementierte Ranz erste PID-Regelkreise, validierte die IMU-Sensorik in separaten Testprogrammen und analysierte die Dynamik des Gesamtsystems mit Blick auf Massenträgheit, Raddynamik und Lenkkinematik. Besondere Aufmerksamkeit galt dem Zusammenspiel von Trägheitsmoment, Lenkwinkel und Rückstellkräften, da diese Größen die Querstabilität wesentlich bestimmen. Ergänzende Messreihen dienten der Bewertung der BNO055 hinsichtlich Genauigkeit und Latenz der Rollwinkelbestimmung. Eine sorgfältige Kalibrierung stellte sicher, dass die Datenqualität den Anforderungen eines geschlossenen Regelkreises genügt.

Die Arbeit dokumentiert die Hardware umfassend anhand von Schaltplänen, 3D-Modellen und Messaufnahmen und bildet damit die hardwareseitige Referenzbasis für alle nachfolgenden Entwicklungen [28]. Zugleich wurden systemische Grenzen der ersten Ausbaustufe transparent herausgearbeitet, darunter die begrenzte Reaktionsgeschwindigkeit des *MD49*-Motorcontrollers sowie Latenzen in der *UART*-Kommunikation und somit konkrete Ansatzpunkte für spätere Verbesserungen identifiziert.

Insgesamt kennzeichnet die Arbeit von Ranz den Übergang von theoretischen Vorüberlegungen hin zu einer belastbaren Forschungsplattform.

2.3.2 Amine Karabila (2022) – Erste funktionierende Regelung

Amine Karabila schloss 2022 unmittelbar an den Hardware-Prototyp von Ranz an und überführte das System von einzelnen Stellversuchen zu einem geschlossenen, echtzeitfähigen Regelkreis. Kernbestandteil ist eine dreistufige P-Regelung auf dem *BeagleBone Black*, die den Rollwinkel der BNO055-IMU [2] und die Lenkerstellung des MD49-Encoders [8] auswertet, während ein hinterer Nabenmotor die Vortriebsdrehzahl konstant hält [19]. Damit gelang erstmals eine softwareseitig realisierte Balancekontrolle, die den Forschungsprototyp in eine echte Versuchsplattform überführte.

Ein erstes Hindernis war die Strombegrenzung des MD49, die den EMG49-Lenkmotor zu langsam ansprechen ließ. Karabila ersetzte deshalb den Leistungsteil durch eine BTS7960-H-Brücke [16] und verwendete den MD49 nur noch zur hochauflösenden Wegmessung. Nicht-blockierende *UART*-Aufrufe verkürzten die Encoderabfrage von rund 4 ms auf knapp 1 ms, sodass die Gesamttransportzeit des Reglers auf etwa 2 ms sank. Damit wird deutlich, dass die Stabilität des Regelkreises aus dem Zusammenspiel leistungsfähiger Algorithmen und einer auf minimale Latenzen ausgelegten Hard-/Software resultiert.

Die erste Regelversion koppelte die Motorgeschwindigkeit über einen gestreckten tanh-Verlauf an die Verweildauer des Rollwinkels in einer „habitablen Zone“. Außerhalb des Labors erwies sich dieses Verfahren jedoch als zu träge. Der Lenker schlug regelmäßig an die Endanschläge, und

Stützräder mussten das Fahrrad vor dem Kippen bewahren. Die finale Lösung verknüpfte dagegen den gemessenen Rollwinkel direkt mit einem geschwindigkeitsabhängigen Soll-Encoderwert. Dadurch pendelte der Lenker nur noch um wenige Grad um die Nullposition.

Validiert wurde der Regler im Hof des Informatikgebäudes ($\approx 9.5 \times 26.5$ m, fünf Prozent Steigung). Bei Sollgeschwindigkeiten von 6–10 km/h blieb das Fahrrad während aller aussagekräftigen Fahrten aufrecht.

Damit bewies Karabilas Arbeit erstmals die Realisierbarkeit einer selbstbalancierenden Regelung und schuf die Grundlage für die PID-Optimierungen von Zander sowie die MPC-Navigation von Giray. Insbesondere machte sie deutlich, dass die Regelgüte stark von niedrigen Latenzen und einer robusten Kopplung zwischen Sensoren und Aktoren abhängt, ein zentrales Motiv, das auch in den nachfolgenden Arbeiten beibehalten und weiterentwickelt wurde.

2.3.3 Jonah Zander (2023) – Optimierte PID-Balanceregung

Zander knüpfte 2023 an die Vorarbeiten an und formulierte als Kernziel, das Fahrrad mindestens 60 s auf ebener Strecke ohne Bodenkontakt der Stützräder stabil fahren zu lassen [31]. Zur Umsetzung entwickelte Zander eine modular aufgebaute C-Software auf dem BeagleBone Black (Debian), in der Multi-Threading Sensor-/Aktor-I/O, Reglerkern und Logging entkoppelt. Zeitkritische Abtast- und Vorverarbeitungsaufgaben wurden in die Programmable Realtime Units (PRUs) verlagert, wodurch deterministische Lesezyklen für Fahr- und Lenkwinkel im zweistelligen kHz-Bereich möglich wurden. Die effektive Transportverzögerung zwischen Messung und Stellgröße sank dadurch deutlich, was die Stabilitätsreserven des Reglers erhöhte [31].

Die Balanceregung realisierte Zander als klassisch ausgelegten PID-Regler, dessen Parameter iterativ auf Basis umfangreicher Mess- und Logdaten optimiert wurden. Im Vergleich zur zuvor eingesetzten dreistufigen P-Logik [19] reduziert der I-Anteil den stationären Regelfehler (Ausregelgüte), während der D-Anteil die Einschwingdämpfung erhöht. In der Software wurden robuste Elemente berücksichtigt, unter anderem Begrenzung und Glättung des Lenkmotor-Solls (Rate- und Amplitudenlimits), sorgfältige Filterung der Eingangssignale (Encoder/IMU) zur Rauschunterdrückung bei erhaltener Phasenreserve sowie latenzarme Pfade zwischen PRU-Puffern und Reglerkern zur Minimierung von Jitter. Das Loggingsystem zeichnete synchronisierte Sensor- und Aktordaten auf und erlaubte eine systematische Auswertung der Reglerwirkung [31].

Neben dem Reglerkern stärkte Zander die Betriebssicherheit. Dabei wurden zwei Notausschalter (separate Lenkungs- bzw. Gesamtabschaltung), eine Reißleine zur sofortigen Trennung der Lenkungsversorgung in kritischen Situationen sowie ein Fail-Safe für den RC-Empfänger (Stop bei Signalverlust) begrenzen Risiken unkontrollierter Lenkereinschläge und mechanischer Schäden eingesetzt [31].

In experimentellen Fahrttests blieb das Fahrrad über mehr als 60 s stabil in Balance, ohne dass die Stützräder den Boden berührten. Bei konstanter Geschwindigkeit auf ebenem Untergrund waren zudem gesteuerte Richtungsänderungen per Fernbedienung möglich, ohne die Balance zu verlieren [31]. Damit erzielte Zander den bis dahin längsten autonom balancierten Fahrbetrieb des Prototyps.

2.3.4 Yasin Giray (2024) – YOLO + MPC-Fahrtrichtung

Aufbauend auf der in [31] erreichten stabilen Balanceregung verlagerte Giray (2024) [10] den Schwerpunkt auf die autonome Fahrtrichtungswahl mittels Computer Vision und modellprädiktiver

Regelung. Ziel war eine monokamera-basierte Pipeline, die in Echtzeit auf eingebetteter Hardware lauffähig ist und aus Bilddaten eine Referenztrajektorie ableitet, der das Fahrrad folgen kann, ohne die Balance zu verlieren.

Die Wahrnehmungskomponente setzt auf ein effizientes Segmentierungsmodell aus der YOLOv8-Familie [17]. Nach einer Evaluation alternativer Verfahren (u. a. FastSAM [32]) fiel die Wahl aufgrund des guten Verhältnisses von Genauigkeit und Latenz auf ein Ultralytics-Modell in der Segmentation-Variante ⁴. Das Netz wurde mit einem angepassten Datensatz trainiert und für die Inferenz mittels TensorRT auf dem NVIDIA Jetson Nano beschleunigt, sodass die Bildauswertung in Zyklen von etwa 100 ms erfolgen konnte. Aus den segmentierten Szenen werden Stützpunkte extrahiert und über Spline-Interpolation zu einer glatten Referenzspur verdichtet, die die gewünschte Fahrlinie repräsentiert.

Für die Fahrregelung implementierte Giray einen Model Predictive Controller (MPC) auf Basis eines Einspurmodells [15]. Der MPC minimiert über einen endlichen Horizont Abweichungen von der Referenzspur bei gleichzeitig moderatem Stellaufwand und berücksichtigt Nebenbedingungen des Systems. In jedem Rechenzyklus wird das Optimierungsproblem mit der aktuellen Zustandsschätzung gelöst. Modellierung, Trajektorien-Interpolation und MPC-Optimierung wurden zunächst in Python umgesetzt und auf die Jetson-Plattform portiert.

Die Pipeline detektierte in den Simulationen Fahrbahnkanten und einfache Hindernisse zuverlässig. Daraus ergaben sich stabile Referenztrajektorien. Zugleich wurden Grenzen sichtbar, da eine explizite Kollisionsvermeidung noch nicht integriert wurde. Eine Tiefeninformation war nur indirekt über Bildgeometrie vorhanden. Nach Giray ist mit neueren Segmentierungsansätzen und leistungsfähigeren Edge Plattformen eine weitere Reduktion der Latenz zu erwarten [32] [17].

Für die vorliegende Arbeit ist die Einordnung klar: Die von Zander übernommene PID-Balanceregung bildet die innere Schleife, während Girays MPC-basierter Spurhalter als äußere Schleife in *Webots* integriert wird. Diese Schichtung erlaubt es, die Kopplung von Balancehaltung und visionbasierter Trajektorienverfolgung systematisch zu untersuchen und in einer realitätsnahen Simulation zu kalibrieren, bevor reale Fahrversuche erfolgen.

⁴Gemeint ist die YOLOv8-Ausprägung, die neben Klassen/Boxen pro Objekt Pixelmasken (Semantik/Instanz) vorhersagt.

3 Grundlagen

Dieses Kapitel führt die theoretischen und methodischen Grundlagen ein, die für das Verständnis der im Folgenden vorgestellten Reglerarchitekturen und Bildverarbeitungsverfahren notwendig sind. Zunächst wird die PID-Regelung als klassischer Vertreter der Regelungstechnik erläutert und ihre Eignung zur Stabilisierung des Balance-Problems des Fahrrads diskutiert. Anschließend folgt eine Einführung in das Prinzip der modellprädiktiven Regelung (MPC), die eine vorausschauende Berechnung von Stellgrößen erlaubt. Abschließend werden die Grundlagen der Objekterkennung mit YOLO sowie der Bildsegmentierung vorgestellt, um die visuelle Wahrnehmung und Spurführung des autonomen Fahrrads zu ermöglichen.

3.1 PID-Regelung – Prinzip und Eignung für das Balance-Problem

Die Proportional-Integral-Differential (PID)-Regelung ist der am weitesten verbreitete Ansatz der klassischen Regelungstechnik. Ein PID-Regler ermittelt die Stellgröße $u(t)$ aus dem Fehler $e(t)$ zwischen Sollwert SP und dem gemessenen Istwert PV nach

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} \quad (1)$$

wobei K_P , K_I und K_D die Verstärkungsparameter für proportionale, integrale und derivative Anteile sind. Der proportionale Anteil erzeugt eine Stellgröße, proportional zur momentanen Fehlergröße und reduziert so die Anstiegszeit. Der Integrator akkumuliert den Fehler und eliminiert stationäre Abweichungen, kann aber langsame Dynamik verursachen [14]. Die Derivative wirkt der Änderung des Fehlers entgegen und dämpft dadurch Überschwinger. Sie erhöht jedoch die Empfindlichkeit gegenüber Messrauschen, weshalb in vielen Anwendungen filternde Maßnahmen erforderlich sind.

PID-Regler werden eingesetzt, wenn eine robuste und einfach implementierbare Rückführung gefordert ist und das Systemverhalten hinreichend durch lineare Modelle beschrieben werden kann. In der Literatur wird die PID-Regelung häufig als Standardlösung für nicht integrierende Prozesse bezeichnet. Ihre Parametrierung erfordert eine sorgfältige Abwägung zwischen Ansprechverhalten und Stabilitätsreserve. Eine Erhöhung des proportionalen Anteils verringert die Regelabweichung, kann aber zu Instabilitäten führen. Ein zu hoher Integrationsanteil kann Integrator-Windup verursachen, die wiederum zu Instabilitäten führen kann.

Beim selbstbalancierenden Fahrrad lässt sich die Balanceproblematik auf das klassische inverses-Pendel-Problem zurückführen. Ein Rad, dessen Schwerpunkt über der Aufstandsfläche liegt, befindet sich in einem instabilen Gleichgewicht, wodurch kleinste Störungen zum Kippen führen. Um das System aufrecht zu halten, muss der Regler in Echtzeit Stellmomente ansetzen, die den kippspezifischen Drehmomenten entgegenwirken. Die Anforderungen an geringe Latenz und hohe Abtastrate werden etwa beim inversen Pendel mit Updatefrequenzen von mehreren Kilohertz umgesetzt. Dies verdeutlicht, dass auch beim Fahrrad eine schnelle Regelungsschleife erforderlich ist, wie sie beispielsweise in den Bachelorarbeiten von Karabila und Zander implementiert wurde. Beide Arbeiten basieren auf P- bzw. PID-Reglern, die den Neigungswinkel mittels IMU messen und über den Lenkmotor oder den Antrieb korrigieren [3].

Insgesamt ist die PID-Regelung für das Balance-Problem des autonomen Fahrrads wegen ihrer geringen Komplexität und Rechenlast, der einfachen Implementierung auf Mikrocontrollern sehr gut geeignet. Die Nichtlinearität des Fahrverhaltens und die starken Störmomente durch Schräglagen erfordern jedoch eine sorgfältige Abstimmung der Parameter und gegebenenfalls Erweiterungen wie

Anti-Windup-Mechanismen oder Filterung der derivativen Komponente, um die Regelung stabil und performant zu halten.

3.2 MPC-Regelung – Prinzip und Eignung für das Fahrtrichtungsproblem

Model predictive control (MPC) nutzt ein internes Modell des Systems, um dessen zukünftiges Verhalten über einen endlichen Zeithorizont vorherzusagen. Auf Grundlage dieser Vorhersage und des aktuellen gemessenen bzw. geschätzten Zustands werden optimale Stellgrößen berechnet, die einen definierten Kostenfunktional minimieren und gleichzeitig die Systembeschränkungen einhalten. Nach jedem Abtastintervall wird nur die erste Stellgröße angewendet und die Optimierung mit vorwärts verschobenem Horizont neu gelöst, weshalb man auch von *receding horizon control* spricht. Zu den wesentlichen Vorteilen von MPC zählen die proaktive Reaktion auf künftige Störungen, die Möglichkeit nichtlineare Modelle ohne Linearisierung zu berücksichtigen und die explizite Berücksichtigung von physikalischen und sicherheitsrelevanten Beschränkungen. Im Gegensatz zu reaktiven PID-Reglern kann MPC damit frühzeitig gegensteuern und gleichzeitig harte Grenzen, etwa Stellweg- oder Geschwindigkeitsbeschränkungen, einhalten [9].

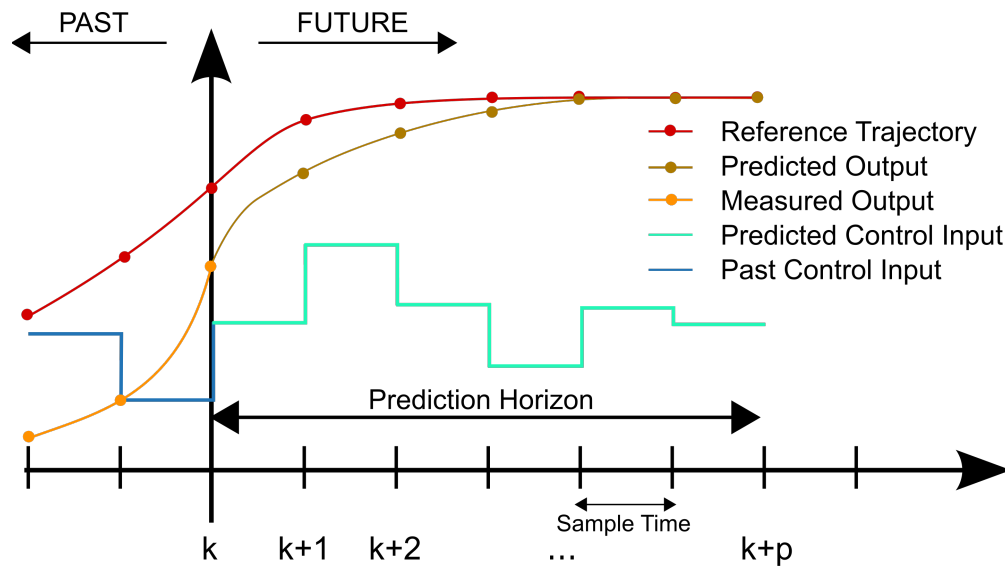


Figure 1: Verständnisschema der MPC-Regelung

Beim selbstbalancierenden Fahrrad besteht das Fahrtrichtungsproblem darin, aus Kamerainformationen einen Sollkurs abzuleiten und das Fahrzeug mit begrenztem Lenkradius und Lenkrate entlang dieser Trajektorie zu führen. Der in der Vision-Regelung verwendete `VisionMPCController` modelliert das Fahrzeug als einfaches Fahrraddynamikmodell mit Zustandsvektor $x = [x, y, v, \psi]$ für Position, Geschwindigkeit und Gierwinkel sowie Eingangsvektor $u = [a, \delta]$ für Beschleunigung und Lenkwinkel. Aus dem aktuellen Zustand und dem lateral gemessenen Fehler wird ein Referenzpfad erzeugt (Ziel-Yaw proportional zum Fehler), der über einen Horizont T vorwärts projiziert wird. Anschließend löst der MPC ein konvexes Optimierungsproblem, das die Abweichung von diesem Pfad, die Eingangsgrößen sowie deren zeitliche Änderung in einer quadratischen Kostenfunktion gewichtet und durch Beschränkungen auf maximalen Lenkwinkel, Lenkrate, Geschwindigkeitsbereich und Beschleunigung ergänzt. Stehen geeignete Optimierungslöser (z. B. `cvxpy`) zur Verfügung, werden aus diesem Problem kontinuierliche Stellgrößen $\delta \in [-\delta_{\max}, \delta_{\max}]$ berechnet, andernfalls

fällt der Controller auf eine einfache P-Regelung zurück. Vor der Weitergabe an den schnellen Balance-Controller werden die resultierenden Lenkwinkel und Beschleunigungen normiert, geglättet und auf Änderungsraten begrenzt, um ruckfreie Übergänge zu gewährleisten [12]. Eine spezifische Erklärung der implementierten Regelung findet sich in Kapitel 5.2.

Die Eignung der MPC Regelung für das Fahrtrichtungsproblem ergibt sich aus der Vorhersagefähigkeit und der Einbeziehung von Beschränkungen. Das Fahrradsystem ist nichtlinear und um einer gekrümmten Spur zu folgen, muss der Regler zukünftige Kurvenverläufe berücksichtigen und gleichzeitig mechanische Limits (Lenkeinschlag, Lenkrate) respektieren. MPC kann diese Randbedingungen explizit in das Optimierungsproblem integrieren und die Stellgrößen so wählen, dass sowohl die Trajektorienabweichung als auch der Stellaufwand minimiert werden. In Yasin Girays Masterarbeit wurde der visionbasierte MPC-Regler mit einem Horizont von $T = 3$ und den genannten Zuständen und Eingängen implementiert und mit Gewichten auf Zustände, Eingänge und deren Differenzen parametrisiert. Simulationen zeigten, dass das Fahrrad mithilfe des MPC gleichmäßig entlang der aus der Bildsegmentierung abgeleiteten Spur navigierte und dabei die mechanischen Limits einhielt. Somit stellt MPC einen geeigneten Ansatz dar, um das autonome Fahrrad stabil und vorausschauend entlang einer vorgegebenen Fahrtrichtung zu steuern.

3.3 YOLO-Grundlagen und Segmentierung

You Only Look Once (YOLO) ist eine Reihe von Echtzeit-Objekterkennungsverfahren, die auf Convolutional Neural Networks basieren. Das ursprüngliche YOLO wurde 2015 von Redmon et al. vorgestellt und erfordert im Gegensatz zu region-proposal-basierten Ansätzen nur einen Vorwärtsthroughlauf durch das Netz, um Klassen und Bounding Boxes für alle Objekte in einem Bild bestimmen [30]. Neuere Modelle wie YOLOv8 erweitern die ursprüngliche YOLO-Architektur. Sie können nicht nur Objekte erkennen, sondern auch Bilder klassifizieren und Instanzen segmentieren. Ein moderner, leichter Backbone ⁵, ein ankerfreier Detektionskopf ⁶ und verbesserte Verlustfunktionen erhöhen Effizienz und Genauigkeit. Dadurch laufen die Modelle auf vielen Hardwareplattformen zuverlässig und schnell [24].

Bei der Bildsegmentierung wird jedem Pixel eines Bildes ein Label zugeordnet. Semantic Segmentation ordnet jedem Pixel eine Objektklasse zu, ohne zwischen einzelnen Instanzen zu unterscheiden, während Instance Segmentation jedes Pixel einer individuellen Objektinstanz zuordnet [29]. Panoptic Segmentation kombiniert beide Ansätze, indem sie sowohl die Klasse als auch die Instanz für jedes Pixel identifiziert [29]. In der Instanzsegmentierung liefert das Netzwerk neben Bounding Boxes auch Masken, die die Form jedes erkannten Objekts abbilden.

YOLOv8 erweitert die Objekterkennung um Instanzsegmentierung, wobei ein auf YOLACT ⁷ basierender Kopf für jedes erkannte Objekt eine zusätzliche Maske berechnet [24]. Durch diese Erweiterung kann ein Modell in einem einzigen Forward-Pass sowohl Objekte lokalisieren als auch deren pixelgenauen Umrisse bestimmen. Wie bei früheren YOLO-Versionen werden Varianten unterschiedlicher Größe bereitgestellt (Nano, Small, Medium, Large usw.), um den Kompromiss zwischen Geschwindigkeit und Genauigkeit zu steuern [24].

⁵Der Backbone ist das Merkmalsextraktionsnetz der Architektur. Er transformiert das Eingabebild in hierarchische, mehrskalige Feature-Maps, die die Grundlage für Klassifikation, Lokalisierung und Segmentierung bilden.

⁶Ankerfrei bedeutet ohne vordefinierte Ankerboxen. Der Kopf schätzt pro Bildposition direkt Objektzentren und Boxparameter sowie Klassenwahrscheinlichkeiten, was Hyperparameter reduziert und das Matching im Training vereinfacht.

⁷YOLACT steht für You Only Look At Coefficients und ist ein einstufiges Verfahren zur Instanzsegmentierung. Es erzeugt Prototypmasken und kombiniert sie mit Koeffizienten zu Objektmasken

In dieser Arbeit wird eine YOLOv8-basierte Segmentierung für die Umfeldwahrnehmung genutzt. Das Modell ist auf vier Klassen trainiert: Wiese („meadow“), Hindernis („obstacle“), Hauptfahrbahn („street_main“) und Seitenfahrbahn („street_side“), wie in Abbildung 2 zu sehen ist [11]. Dabei liefert es pro Klasse ein Pixelmasken Overlay sowie die zugehörigen Bounding Boxes. Anschließend wird aus dem Segment „street_main“ der laterale Fehler bestimmt, der als Eingang für die Pfadverfolgung dient. Die genauen Details zur Modellauswahl, Datensatzgenerierung und Implementierung werden in einem späteren Kapitel beschrieben. Hier soll lediglich hervorgehoben werden, dass die Kombination aus schneller YOLO-Detektion und Instanzsegmentierung eine effiziente und genaue Grundlage für die kamerabasierte Fahrspurerkennung bildet.



Figure 2: YOLO-Segmentierung mit YOLOv8

4 Webots-Simulationsumgebung

In diesem Kapitel wird die in Webots realisierte Simulationsumgebung vorgestellt, die als zentraler Versuchsstand für Entwurf, Parametrierung und Evaluation der kombinierten Regelung des selbst-balancierenden Fahrrads dient. Ziel ist es, eine reproduzierbare und sichere Testumgebung zu schaffen, in dem das physikalische Fahrverhalten, die sensorgeführte Pfadverfolgung sowie die Kopplung von schneller Balance-Regelung und langsamer Vision-/Trajektorienplanung unter kontrollierten Bedingungen untersucht werden können. Webots bietet dafür eine deterministische Dynamiksimulation auf Basis von ODE, eine hierarchische Szenenbeschreibung und eine klare Trennung von Geometrie, Physik, Sensorik, Aktuatorik und Controllern [4].



Figure 3: Minimalstruktur einer Webots `.wbt`-Welt

4.1 Überblick

Webots ist ein frei verfügbares, quelloffenes und plattformunabhängiges Robotik Simulationsprogramm, das seit 1998 kontinuierlich gepflegt wird und in Forschung, Lehre und Industrie breite Anwendung findet [4, 23]. Es kombiniert eine moderne QtBenutzeroberfläche mit einer Mehrkörper-Dynamiksimulation auf Basis der *Open Dynamics Engine* (ODE) sowie einer OpenGLgestützten RenderingPipeline. Dadurch lassen sich fotorealistische 3DSzenen mit deterministischem Integrationschema und feingranularer Kontrolle der Simulationszeit schrittgenau ausführen [4].

Kern des Webotsmodells ist die hierarchische Szenenrepräsentation in *World*-Dateien (`*.wbt`; VRML-basiert). Eine Welt kapselt globale Parameter (z. B. Zeitauflösung, Kontakt und Reibungsmodelle), Umgebungsobjekte (Boden, Licht, Hintergrund) sowie Roboterknoten mit Sensorik, Aktuatorik und Masseneigenschaften. Geräte (z. B. Kamera, IMU, Lidar, RotationalMotor) sind als Nodes mit klar definierten Schnittstellen realisiert, ihre Parameter (Auflösung, Bildfrequenz, Dämpfung, Grenzwerte) sind reproduzierbar konfigurierbar. Controller laufen als getrennte Prozesse in C/C++, Python, Java oder MATLAB und kommunizieren zur Laufzeit über APIs mit der Simulationswelt [4]. Eine spezielle *SupervisorAPI* erlaubt höherwertige Eingriffe (Positionsreset, automatisiertes Logging, MetrikAuswertung) und bildet die Grundlage für reproduzierbare Benchmarking-Pipelines [4].

Für regelungsorientierte Studien bietet Webots drei Eigenschaften, die in dieser Arbeit zentral sind:

- (i) *Determinismus* durch festen Zeitschritt und explizite Integrationsreihenfolge, was faire Parameterstudien ermöglicht.
- (ii) *Modularität* durch die Trennung von Geometrie, Physik, Sensorik/Aktuatorik und Controllern, wodurch Varianten (z. B. Reibungskoeffizienten, Trägheiten, Abstraten) isoliert untersucht werden können.
- (iii) *Automatisierbarkeit* über den Supervisor, u. a. für SzenarioResets, Störgrößeninjizierung, Batch-Runs und standardisierte Ergebnisberichte [4].

Für diese Arbeit ist Webots folglich eine geeignete Testumgebung, um die Kopplung einer schnellen Balance-PID-Regelung mit einer langsameren vision bzw. modellprädiktiven Pfadführung systematisch zu entwerfen, zu parametrieren und zu evaluieren. Die klare Trennung der Subsysteme (Physik, Sensorik, Controller) erlaubt Variantenstudien (z. B. Variation von Zeitschritt/Reibung, Sensorausfall, Verzögerungen), während die Supervisor Schnittstelle reproduzierbare Testreihen mit konsistentem Logging unterstützt [4].

4.2 Struktur der Simulationsumgebung

Die Versuche werden in eine `.wbt`-Welt beschrieben, die die Szene hierarchisch (VRML) aufbaut. Dabei sind globale Einstellungen, Geometrie, Materialien und Roboter mit Sensorik, Aktuatorik und Controllern zu finden. Zentral sind ein fester Simulationszeitschritt sowie die globalen Physikparameter, die deterministische Abläufe und reproduzierbare Kontakte sicherstellen. Der Fahrradroboter (*BICYCLE*) ist mit Massen- und Trägheitseigenschaften, vereinfachten Kollisionskörpern und Gelenken für Vortrieb und Lenkung modelliert. Der schnelle Balance-PID in C läuft direkt auf dem Roboter, während ein externer *Supervisor* mit Kamera die Spur wahrnimmt und mittels MPC Lenksollwerte berechnet. Der Datenaustausch erfolgt über *Emitter/Receiver*, wobei das Logging und die Auswertung beim Supervisor liegen. Wie schon erwähnt sind daher, durch die Trennung von Physikmodell, Sensorik und Regellogik gezielte Parameterstudien bei gleichbleibender Szenengeometrie möglich. Der in Listing:1 gezeigte Pseudocode beschreibt die Struktur der in unseren Testfahrten verwendeten Webots-`.wbt`-Weltdatei und zeigt die minimalen Knoten (`WorldInfo`, `RectangleArena`, `Robot`, `Supervisor`) in ihrer Kernkonfiguration.

```
1  #VRML_SIM R2025a utf8
2
3  # -- Externe PROTOs (Arena, Hintergrund, Supervisor) --
4  EXTERNPROTO "projects/objects/floors/protos/RectangleArena.proto"
5  .... (andere EXTERNPROTOs)
6
7  # -- Globale Welt- und Kontaktparameter --
8  WorldInfo {
9    basicTimeStep 2
10   contactProperties [
11     ContactProperties {
12       material1 "wheel"
13       material2 "ground"
14       coulombFriction [ 0.8 ]
15       bounce 0.3
16     }
17   ]
18 }
```

```

17     ]
18 }
19
20 # -- Kamera/Blickpunkt --
21 Viewpoint {
22     orientation 0 1 0 0
23     position 0 2 8
24     follow "Little_Bicycle_V2"
25 }
26
27 # -- Hintergrund + Arena (Textur als Fahrspur) --
28 TexturedBackground { skybox FALSE }
29 TexturedBackgroundLight { }
30 RectangleArena {
31     contactMaterial "ground"
32     floorSize 64 100
33     floorTileSize 64 100
34     floorAppearance PBRAppearance {
35         baseColorMap ImageTexture { url [ "textures/S-Kurve.png" ] }
36         roughness 1
37         metalness 0
38     }
39     wallThickness 0.001
40     wallHeight 0.001
41 }
42
43 # -- Fahrradroboter (Sensorik, Aktorik, Kommunikation) --
44 DEF BICYCLE Robot {
45     name "Little_Bicycle_V2"
46     controller "balance_control_c"
47     supervisor TRUE
48     children [
49         InertialUnit { name "imu" }
50         Camera { name "front_cam" width 480 height 320 fieldOfView 2 }
51         Receiver { name "command_rx" channel 1 bufferSize 64 }
52         Emitter { name "status_tx" channel 2 bufferSize 64 }
53
54         # Hinterradantrieb (vereinfachtes Gelenk)
55         HingeJoint {
56             jointParameters HingeJointParameters { axis 0 0 1 }
57             device [ RotationalMotor { name "motor::wheel" maxTorque 120 } ]
58             endPoint Solid { contactMaterial "wheel" }
59         }
60
61         # Lenkung (vereinfachtes Gelenk)
62         HingeJoint {
63             jointParameters HingeJointParameters { axis 0 0 1 }
64             device [ RotationalMotor { name "handlebars_motor" maxTorque 25 } ]
65             endPoint Solid { }
66         }
67     ]
68     boundingObject Box { size 0.4 1.1 1.8 }
69     physics Physics { mass 4 }
70 }
71
72 # -- Externer Supervisor (Overhead-Kamera, Funk) --
73 Supervisor {
74     name "Vision_Controller"
75     controller "vision_control_py"

```

```

76     children [
77         Camera { name "overhead" width 640 height 360 fieldOfView 2 }
78         Emitter { name "command_tx" channel 1 bufferSize 64 }
79         Receiver { name "status_rx" channel 2 bufferSize 64 }
80     ]
81 }

```

Listing 1: Minimalstruktur einer Webots .wbt-Welt

4.3 Physikmodell: Open Dynamics Engine (ODE) in Webots

Die *Open Dynamics Engine* (ODE) ist eine freie, plattformunabhängige Physikbibliothek zur Simulation starrer Mehrkörpersysteme mit Gelenken und Kollisionen. Sie kombiniert integrierte Kollisionserkennung mit einem constraintsbasierten Dynamikmodell und wird in zahlreichen Simulationsumgebungen eingesetzt. Das Welt- und Objektverhalten wird über `WorldInfo` und `Physics` konfiguriert. Dort werden unter anderem der Zeitschritt sowie die ODE-Kernparameter ERP (Error Reduction Parameter) und CFM (Constraint Force Mixing) gesetzt. ERP/CFM steuern die „Feder-/Dämpfer“-Eigenschaften restringierter Bewegungen und Kontaktauflösungen und sind für stabile, nicht-schwingende Lenkeingriffe wesentlich. Kontakte werden als *contact joints* mit Coulomb-Reibung gelöst, zusätzliche Felder Rückprall und weiche Grenzen erlauben.

In Webots wird die Reibung über den Knoten `ContactProperties` festgelegt (Listing 1 Zeile 31). Neben der üblichen Gleitreibung können auch Roll- und Drehwiderstände definiert werden. Für bestimmte einfache Grundkörper ist zudem richtungsabhängige, Reibung möglich. Auf *Körper*-Ebene definiert der `Physics`-Knoten Masse, Trägheit, Dämpfung und optionale ODE-Erweiterungen (Listing 1 Zeile 69). Gelenke (z.B. *Hinge* für Lenkung/Vortrieb) werden durch Motoren (`Motor`) angetrieben. In Summe ergibt sich ein kompaktes, gut abstimmbares Modell, das das Fahrrad als Mehrkörpersystem mit Kontakten, Reibung und gelenkgetriebenen Freiheitsgraden realistisch genug für Reglerentwurf und Vergleichsstudien abbildet.

4.4 Sensoren

Für die Stabilisierung des selbstbalancierenden Webots-Fahrrads kommen mehrere Sensorsysteme zum Einsatz. Zentral sind eine **Inertialmesseinheit** (IMU) am Fahrzeug zur Lageerfassung und eine **Kamera** zur optischen Fahrwegdetektion. Ergänzend liefern **Motorsensoren** (Encoder) an den Achsen Messgrößen wie Lenk- und Raddrehwinkel. Die IMU versorgt den inneren schnellen Regelkreis (Balance-PID) laufend mit dem aktuellen Rollwinkel, während die Kamera den äußeren, langsamer abtastenden Regelkreis (vision-basiertes MPC) mit Spurabweichungsinformationen speist. Beide Sensorströme werden anschließend in einfacher Weise fusioniert, um die Regelgrößen an die Aktuatoren (Lenkmotor und Antrieb) auszugeben. Im Folgenden werden die einzelnen Sensoren hinsichtlich physikalischer Funktion, Modellierung in Webots, Rolle im Regelkreis und Integration im Quellcode beschrieben.

4.4.1 Inertialmesseinheit (IMU)

Physikalisches Prinzip: Eine Inertial Measurement Unit (IMU) erfasst Trägheitsgrößen des Fahrzeugs, typischerweise mit kombinierter **Beschleunigungsmessung** (Accelerometer) und **Drehratensensoren** (Gyroskope) auf drei Raumachsen [21]. Durch Fusion dieser Signale (oft

ergänzt um einen Magnetometer-Kompass), lässt sich die Orientierung im Raum bestimmen. Im realen Prototyp kommt z.B. der IMU-Sensor **BNO055** zum Einsatz, der intern die Rohdaten filtert und direkt **Euler-Winkel** (Roll, Pitch, Yaw) bereitstellt [2]. Die Rollachse (Seitenneigung) ist beim Fahrrad die entscheidende Stabilitätsgröße. Reale IMUs liefern diese Winkel mit begrenzter Auflösung und unterliegen Rauschen sowie Drift, was meist durch Kalibrierung und Sensorfusion kompensiert werden muss. Die Ausgaberate moderner IMUs liegt im Bereich mehrerer hundert Hertz (der BNO055 etwa ~ 200 Hz [2]), was für eine schnelle Lagekontrolle ausreicht.

IMU-Modell in Webots: Webots stellt mit dem `InertialUnit`-Knoten ein abstraktes IMU-Modell bereit, das idealisierte Orientierungsdaten liefert. Konkret berechnet und aktualisiert das `InertialUnit`-Gerät in jeder Simulationsschleife den **Kipp- und Gierwinkel** des Roboters relativ zu einem globalen Bezugssystem [21]. Im Gegensatz zu einer realen IMU entfallen hier Kalibrierfehler und Rauschen, wodurch die Orientierungswinkel praktisch **fehlerfrei und ohne Drift** sind. Das im Webots-Weltmodell definierte Gerät `imu` (Listing 1 Zeile 49) ist starr im Fahrradrahmen verbaut (an geeigneter Position, etwa nahe dem Schwerpunkt) und gibt standardmäßig Roll-, Pitch- und Yaw-Winkel in *Radian* aus. Die Abtastung erfolgt synchron mit dem physikalischen Simulationszeitschritt.

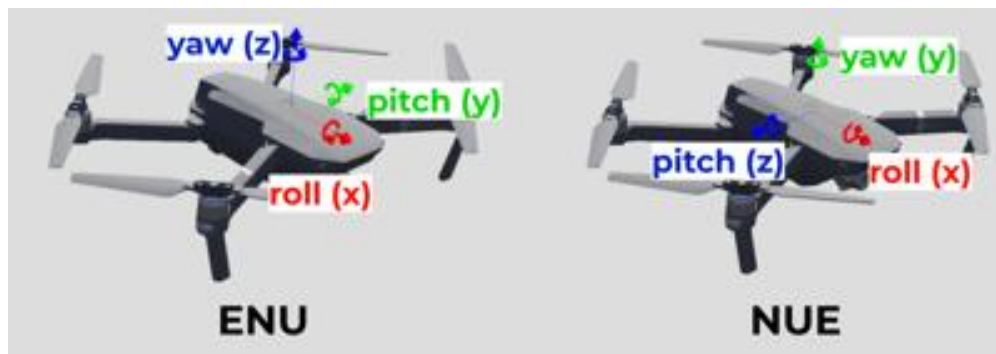


Figure 4: IMU-Knoten in Webots.

Rolle im Regelkreis: Der IMU-Sensor bildet die zentrale **Rückführung** für die **Balance-Regelung**. In jedem Simulationsschritt liest der PID-Regler den aktuellen Rollwinkel φ_{roll} aus der IMU und berechnet daraus die Stellgröße für den Lenkantrieb. Konkret fungiert der Rollwinkel als *Regelgröße* im inneren Regelkreis, wobei das Fahrrad nach links/rechts abweicht, soll durch einen entsprechenden Lenkeinschlag gegengesteuert werden, bis $\varphi_{\text{roll}} = 0$ (senkrechte Lage) erreicht ist. Im vorliegenden Modell ist der `basicTimeStep` der Welt auf 2 ms gesetzt [13], was einer Messfrequenz von 500 Hz entspricht (Listing 1 Zeile 9). Dadurch können selbst schnelle Kippbewegungen des Fahrrads in Echtzeit erfasst werden [13]. Dadurch reagiert der PID-Regler sehr schnell auf Lageänderungen, was eine Grundvoraussetzung ist, um die Instabilität beherrschen zu können. Im äußeren (vision-basierten) Regelkreis spielt die IMU direkt keine Rolle, allerdings fließen die Balancierinformationen indirekt ein, wenn die Vision-Steuerung, z.B. die Fahrgeschwindigkeit reduziert, sobald der Stabilitätsfaktor sinkt (starke Schräglage detektiert). Insgesamt sorgt die IMU dafür, dass die **Regelstrecke** mit hoher Güte beobachtet wird, und bildet so das Fundament der Stabilisierung.

Im C-Controller `balance_control.c` wird die IMU über die Webots-API initialisiert und ausgelesen. Listing 2 zeigt auszugsweise, wie das Gerät referenziert, aktiviert und genutzt wird. Zunächst wird der Device-Tag der IMU mit ihrem Namen "`imu`" angefordert und das Sensor-Sampling mit dem Simulationstimestep (2 ms) eingeschaltet. In der Hauptschleife ruft der Regler dann kontinuier-

lich die Funktion `wb_inertial_unit_get_roll_pitch_yaw()` auf, welche einen Vektor der Winkel zurückliefert. Der Rollwinkel (Index 0 des Arrays) wird extrahiert und in die Regelung eingespeist. Da die Webots-IMU idealisiert ist, entspricht dieser Winkel exakt der aktuellen Fahrzeugneigung ohne Messfehler. Im Code ist dennoch ein Fallback vorgesehen: Falls der Supervisor-Knoten verfügbar ist, wird alternativ dessen Orientierungsmatrix ausgewertet, dies diene experimentell der Verifikation der IMU-Daten und kann die gleiche Information liefern. Der exemplarische Code zeigt, wie einfach die Orientierung in Webots abgefragt werden kann, verglichen mit einer realen IMU (wo man per I²C erst Rohdaten lesen und konvertieren müsste). Die geringe Latenz der `wb_inertial_unit_get_roll_pitch_yaw`-Funktion (nur ein Zeiger auf interne Simulationsdaten) trägt dazu bei, dass der Balance-Regelkreis praktisch ohne zusätzliche Verzögerung arbeitet.

```

1 // IMU-Geraet initialisieren (in init_devices)
2 imu_sensor = wb_robot_get_device("imu");
3 wb_inertial_unit_enable(imu_sensor, timestep); // Abtastrate = Welt-
    Zeitschritt (2 ms)
4
5 // ... (im Hauptprogramm der Balance-Regelung)
6
7 // Roll-, Pitch- und Gierwinkel vom IMU-Sensor abrufen
8 const double *rpy = wb_inertial_unit_get_roll_pitch_yaw(imu_sensor);
9 double roll_angle = rpy[0]; // aktueller Rollwinkel [Bogenmass]

```

Listing 2: IMU-Initialisierung und -Abfrage im C-Controller (Balance-Regelung)

4.4.2 Kamera (Videosensor)

Physikalisches Prinzip: Die Kamera dient als **optischer Sensor**, um die Strecke vor dem Fahrrad zu erkennen. Physikalisch basiert sie auf einem *digitalen Bildaufnehmer* (z.B. CMOS-Sensor), der durch eine Linse abgebildetes Licht in Pixelwerte umwandelt. In einem realen System würde eine am Fahrzeug montierte Weitwinkel-Kamera laufend **Videobilder** der Fahrbahn liefern, wobei essentielle Parameter sind die **Auflösung** und die **Bildrate** sind. Für eine stabile Spurhaltung in Echtzeitanwendungen genügen in der Regel zweistellige Bildfrequenzen ($\sim 10\text{--}30\text{ Hz}$), sofern das Fahrzeug nicht extrem schnell fährt. Die Kamera erfasst visuelle Merkmale der Umgebung, die im vorliegenden Kontext insbesondere die **Straßenmarkierung** bzw. den Verlauf der Fahrspur beinhalten, die dann von Bildverarbeitungsalgorithmen ausgewertet werden. Physikalisch können **Umgebungsfaktoren** wie Beleuchtung, Bewegungsunschärfe oder Linsenverzerrungen die Bildqualität beeinflussen, was eine robuste **Merkmalsextraktion** herausfordernd macht. Im Gegensatz dazu operiert eine Kamera jedoch **kontaktlos** und kann reichhaltige Umgebungsinformationen liefern, was sie ideal für die **Pfadführung** macht (im Vergleich zu z.B. lokalen Sensorsignalen eines Gyroskops).

Simulation in Webots: Webots simuliert Kameras über den **Camera-Node**, der virtuelle Renderings der 3D-Szene erzeugt. Im *Weltfile* ist für den Supervisor (die übergeordnete Instanz) eine Kamera mit definierter Position, Ausrichtung und Eigenschaften eingebettet (Listing 1 Zeile 79). Diese sogenannte *Vision-Kamera* blickt aus erhöhter Perspektive nach vorne unten auf die Fahrbahn (vergleichbar mit einer Helm- oder Drohnenkamera, die dem Fahrrad folgt). Die in der Simulation gewählte Auflösung beträgt z.B. 640×360 Pixel bei einem Gesichtsfeld von ca. 114° (Field of View $\approx 2.0\text{ rad}$). Diese Einstellungen sind ein Kompromiss zwischen Sichtfeld und Bilddetail. Die Kamera liefert pro aktiviertem Zeitschritt ein Bild im **RGBA-Format** (Rot–Grün–Blau plus Alphakanal) als Array von Byte-Werten. In der Controller-Software muss die Kamera explizit aktiviert werden, wobei die Abtastrate frei wählbar ist. Der Vision-Controller nutzt eine Bildrate

von rund 20 Hz, entsprechend einem Abtastintervall von 50 ms. Dies ist deutlich langsamer als der Balance-Regelkreis, spiegelt aber die realistische Anforderung wider, dass Bildverarbeitung zeitaufwändiger ist und nicht in jedem Simulationsschritt erfolgen muss. Im Webots-Code wird die Kamera z. B. mit `camera.enable(50)` konfiguriert, sodass alle 50 ms ein neues Frame bereitgestellt wird. Jedes Frame kann vom Controller über `camera.getImage()` abgeholt und weiterverarbeitet werden. Zu beachten ist, dass Webots standardmäßig keine Kamerarausfälle oder Sensorschwächen simuliert, sodass die gelieferten Bilder idealisiert sind (scharf, bei gleichbleibender Beleuchtung).

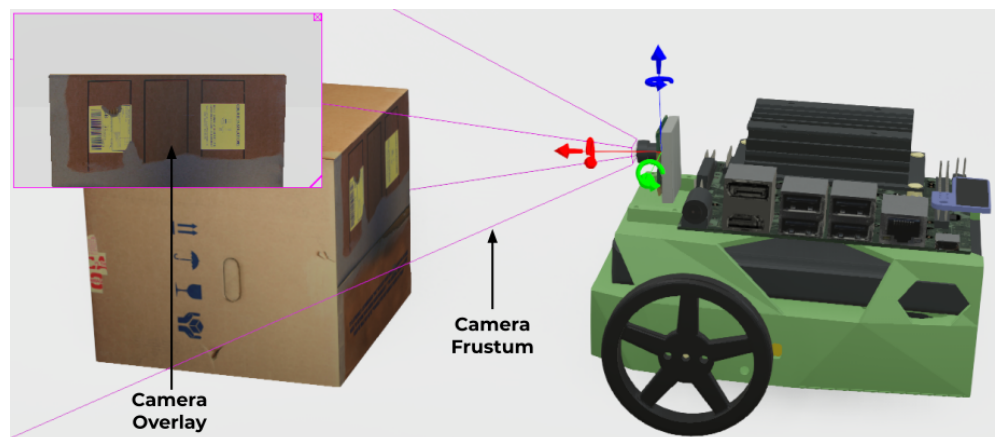


Figure 5: Kamera-Knoten in Webots.

Rolle im Regelkreis: Die Kamera ist der **Schlüsseingang** für den vision-basierten äußeren Regelkreis. Ihre Aufgabe besteht darin, dem **MPC-Fahrtrichtungsregler** Informationen über den Verlauf der Fahrspur relativ zum Fahrzeug zu liefern. Im konkreten Szenario der S-Kurve extrahiert der Vision-Controller aus jedem Kamerabild den lateralen Positionsfehler e der Straße.

Diese Bildverarbeitung geschieht in mehreren Schritten:

1. **Merkmalsextraktion:** Zunächst werden relevante Bildmerkmale berechnet, etwa mittels **Segmentierung** der Straße. Im Projekt wurde hierzu ein vortrainiertes YOLO-Modell verwendet, das die Fahrbahn in der Vogelperspektive segmentiert. Falls kein KI-Modell verfügbar ist, greift ein Fallback auf klassische Bildverarbeitung, mittels Kantenerkennung und Auswertung eines definierten *Region-of-Interest* liefert ebenfalls einen Schätzwert für die Spurmittellage. Ergebnis dieses Schritts ist der Querfehler e (in normierter Form, z. B. $-0.5 \cdots +0.5$ relativ zur Bildmitte) sowie ggf. zusätzliche Qualitätskennzahlen (etwa die erkannte Kontur oder den Bedeckungsgrad der Fahrbahnmaske).
2. **Spurführungsregelung:** Auf Basis des visuellen Fehlers bestimmt der Controller einen passenden **Lenksollwert**. Im entwickelten System kommt hierfür vorzugsweise ein **modell-prädiktiver Regler (MPC)** zum Einsatz, der das Fahrrad als dynamisches Modell (Zustand $\mathbf{x} = [x, y, v, \psi]$) vorausberechnet. Der MPC optimiert die Steuergröße Lenkwinkel δ (und ggf. Beschleunigung) über einen kurzen Zeithorizont so, dass der Querfehler minimiert und Fahrdynamikgrenzen eingehalten werden.

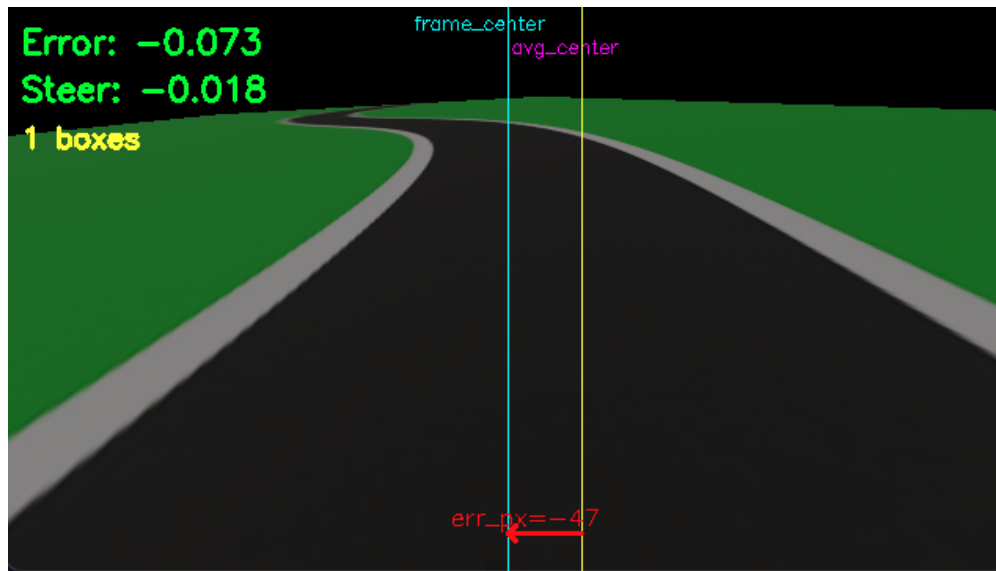


Figure 6: Querfehler e der Straße.

Die Kamera beeinflusst den Balance-Regler auch indirekt, indem deren Messdaten den **Kurs der Fahrt** vorgeben. Wichtig ist, dass die Vision-Daten mit deutlich geringerer Frequenz ankommen als die IMU-Daten. Der Balance-Controller hält während der 50 ms zwischen zwei Kamerabildern die zuletzt empfangene Soll-Lenkvorgabe bei und stabilisiert weiterhin autonom. Sobald ein neues Bild ausgewertet ist, wird der Lenksollwert ggf. korrigiert. Dieses **zweistufige Regelungskonzept** sorgt dafür, dass das Fahrrad einerseits aufrecht bleibt, andererseits aber auch aktiv Richtungskorrekturen vornimmt, um einer kurvigen Referenzstrecke zu folgen.

4.4.3 Motorsensoren (Inkrementalgeber)

Physikalisches Prinzip: Neben IMU und Kamera, die den *Zustand* des Fahrzeugs in Bezug auf Lage und Umwelt erfassen, spielen sogenannte **Motorsensoren** eine wichtige Rolle für die interne Zustandsmessung. Gemeint sind hier **Winkelgeber (Encoder)** an den drehbaren Komponenten, hierbei konkret am Lenkmotor und am Antriebsrad. Physikalisch handelt es sich meist um *inkrementelle Drehgeber*, die auf der Motorachse montiert sind und pro Umdrehung eine bestimmte Anzahl Impulse (Zählschritte) liefern. Durch Mitzählen der Impulse kann der Steuerrechner den zurückgelegten Winkel bestimmen, sodass über die Impulsfrequenz sich auch die Drehgeschwindigkeit ermitteln lässt. In echten Systemen werden beispielsweise **optische Encoder** (eine Scheibe mit Strichmuster und Fotodetektor) oder **Hall-Sensoren** (Magnet und Hallsensor für Umdrehungszählung) eingesetzt. Im Kontext unseres selbstbalancierenden Fahrrads wird der Lenkwinkel-Encoder benötigt, um den aktuellen **Lenkeinschlag** zu kennen, was wichtig ist, um Lenkwinkelgrenzen nicht zu überschreiten. Der Hinterrad-Encoder erfasst die **Raddrehstellung** bzw. via Differenz auch die momentane Raddrehzahl, was zur Abschätzung der Fahrgeschwindigkeit dient. Physikalisch sind solche Sensoren sehr präzise (hochauflösende Encoder bieten > 1000 Impulse/Umdrehung) und liefern in schneller Abfolge Signale, typischerweise synchron zum Motor. Allerdings muss die Ansteuerungselektronik die Impulse auswerten (Interrupts zählen etc.), was in der realen Implementierung zusätzlichen Programmieraufwand bedeutet.

Rolle im Regelkreis: Die Positionssensoren dienen vor allem der Überwachung und der Ableitung von Regelgrößen. Der **Lenkwinkelsensor** liefert dem Balance Controller das Feedback, welcher

Lenkeinschlag vom Motor aktuell realisiert wird. Für alle in dieser Arbeit verwendeten Webots Simulationen gilt dieselbe Lenkkonfiguration. Am Lenker des selbstbalancierenden Fahrrads ist ein `RotationalMotor` mit dem Namen `"motor::handlebars"` verbaut. In der Welt-Datei (.wbt) ist `"maxTorque"` auf 25 gesetzt, was einen Moment von 25 Nm bedeutet. Was dazu führt, dass der Istwinkel nicht 1:1 dem Sollwert folgen kann. Die Parameter `dampingConstant 0.3` und `staticFriction 0.1` erzeugen zusätzliche Gegenmomente, die besonders um den Nullbereich kleine Sprünge abbremsen. Die Gabel sowie das Vorderrad besitzen eine Masse von 2.5 kg und eine Trägheitsmatrix von 0.06 0.07 0.12, die schnelle Winkeländerungen begrenzen. Die beobachtete Verzögerung liegt damit nicht an einer Rechen oder Übertragungsverzögerung, sondern an den Momentgrenzen und Verlusten im Gelenk. Das ist in Abbildung 7 erkennbar, da steile Sollsprünge geglättet werden, was dämpfend wirkt und zu weniger harten Lenkreaktionen führt.

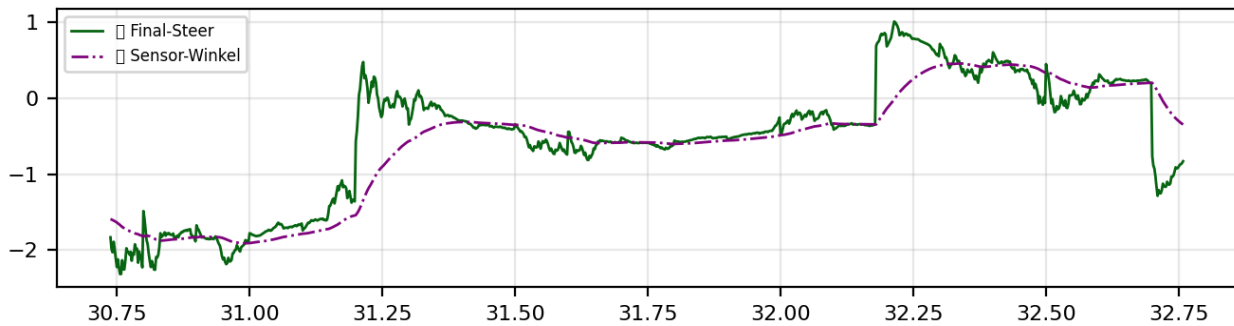


Figure 7: Lenk-Sollwert und Lenk-Istwert.

4.5 Aktuatoren

Das selbstbalancierende Fahrradmodell verfügt über zwei zentrale **Aktuatoren**, die als Stellglieder in die Dynamik des Systems eingreifen. Dabei handelt es sich um einen **Lenkantrieb** an der Vorderadgabel und einen **Antriebsmotor** für das Hinterrad. Beide sind in Webots als `RotationalMotor`-Geräte realisiert und jeweils an einem drehbaren `HingeJoint` angebracht. Physikalisch handelt es sich um **elektrische Rotationsmotoren** (Gleichstrommotoren mit Getriebe), die in der Lage sind, Drehmomente aufzubringen und so entweder einen bestimmten Winkel (Servomodus) oder eine bestimmte Drehgeschwindigkeit einzustellen. Im Folgenden werden zunächst die realen Grundlagen dieser Aktuatoren betrachtet, dann ihre Umsetzung in der Simulation und schließlich ihr konkreter Einsatz im Regelungssystem mitsamt Code-Beispielen erläutert.

Physikalisches Wirkprinzip:

Bei **bürstenbehafteten DC-Motoren**, wie sie in vielen Robotikanwendungen Verwendung finden, erzeugt ein stromdurchflossener Anker in einem Magnetfeld ein Drehmoment (Lorentz-Kraftwirkung) auf die Leiter. Das resultierende **Motordrehmoment** ist annähernd proportional zum Ankerstrom, während die Leerlauf-Drehzahl von der angelegten Spannung bestimmt wird. Im offenen Regelkreis würde ein DC-Motor also schneller drehen bei höherer Spannung und stärker drehen (mehr Kraft) bei höherem Strom.

Um präzise **Stellbewegungen** auszuführen, werden Motoren oft mit **Getrieben** (zur Erhöhung von Drehmoment und Auflösung) und mit **Sensoren** rückgekoppelt (Encodern, s. o.). Ein sogenannter **Servoantrieb** ist ein Motor-Getriebe-System mit interner Positions- oder Drehzahlregelung. Im realen selbstbalancierenden Fahrrad gibt es zwei solcher Antriebe:

- **Der Lenkmotor** wird von einem BTS7960 Treibermodul mit PWM Signalen angesteuert. Die Winkelrückmeldung kommt vom Encoder am Lenkgetriebe. Der MD49 verarbeitet die Encodersignale und stellt Zählwerte zur Positionsbestimmung bereit. Der Antrieb arbeitet als Positionsantrieb auf einen Sollwinkel mit begrenztem Bereich in beide Richtungen. Abweichungen zwischen Soll und Ist Lenkeinschlag, werden durch den Regler ausgegeregelt bis der gewünschte Winkel erreicht ist. Der Antrieb muss ausreichend schnell und drehmomentstark sein um die Querstabilität sicherzustellen.
- **Der Antriebsmotor** am Hinterrad sorgt für den Vortrieb. Dieser wird im Unterschied zum Lenker meist als **Drehzahl- bzw. Drehmomentantrieb** betrieben. Im realen System regelt ein separater PID die Raddrehzahl konstant, indem er den Motorstrom anhand des Hall-Encoder-Feedbacks justiert. Physikalisch verleiht das rotierende Hinterrad dem System einen **gyroskopischen Effekt**, der stabilisierend wirkt, weshalb eine gewisse Grundgeschwindigkeit hilfreich ist. Gleichzeitig muss der Motor aber auch schnell drosseln oder bremsen können, um Kurven sicher zu durchfahren, ohne umzukippen.

Webots-Umsetzung: Webots stellt mit dem `RotationalMotor`--Knoten in Ihrer Simulationsumgebung einen Aktuator zur Verfügung, der an einen `HingeJoint` gekoppelt, Drehmomente ausüben kann. Im Hintergrund berechnet der Physik-Kernel (ODE) die Wirkung des Motors auf das Gelenk, unter Berücksichtigung von Maximalmoment und Zielvorgabe. Jeder Rotationsmotor kann in drei Modi betrieben werden:

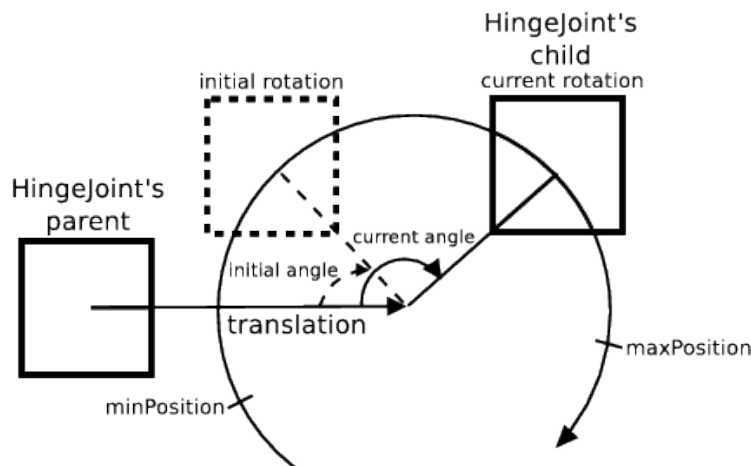


Figure 8: Aktuatoren im Fahrradmodell.

- **Positionsmodus:**

Der Motor strebt einen vorgegebenen Zielwinkel an (`setPosition(angle)`), wobei er innerhalb der konfigurierten Kraft- und Geschwindigkeitsgrenzen das nötige Drehmoment erzeugt, um diesen Winkel zu erreichen. Im Webots-Modell ist der Lenkungsaktor in den Positionsmodus geschaltet. Dazu wird im Code einmalig `wb_motor_set_position(handlebars_motor, 0.0)` aufgerufen, um die Start-Sollposition (geradeaus) zu setzen. Anschließend erhält der Lenkmotor in jeder Regeliteration einen neuen Sollwinkel. Die Regelung der Position übernimmt Webots intern, dabei wirkt der Motor wie eine Feder-Dämpfer-Kopplung ans Gelenk, die Differenz zwischen Ist- und Sollwinkel generiert ein entsprechendes Gegendrehmoment (virtueller P-Regler). Parameter wie `maxTorque` und `maxVelocity` im Weltfile bestimmen,

wie schnell und kräftig an den Sollwinkel nachgeführt wird. Im Modell sind z.B. für den Lenkmotor `maxTorque = 25Nm` festgelegt, ein Wert, der sicherstellt, dass genügend Kraft zum Aufrichten aus Schräglagen vorhanden ist, ohne das System unrealistisch zu gestalten.

- **Geschwindigkeitsmodus:**

Hier wird dem Motor eine Vorgabe-Drehgeschwindigkeit vorgegeben (`setVelocity(omega)`), während die Position auf unendlich gesetzt wird (`setPosition(INFINITY)`). In Webots bedeutet dies, dass der Motor keinen festen Endwinkel hat, sondern kontinuierlich dreht. Der von uns verwendete Antriebsmotor des Hinterrads am Fahrrad wird in unseren Simulationen so betrieben. Im Initialisierungscode wird `wb_motor_set_position(wheel_motor, INFINITY)` aufgerufen, was den Wechsel in den Geschwindigkeitsmodus bewirkt. Fortan interpretiert Webots den Befehl `wb_motor_set_velocity(wheel_motor, v)` als Soll-Drehzahl (in Rad/s) für das Rad. Intern wirkt wieder ein virtueller Regler, der das nötige Drehmoment aufbringt, um diese Geschwindigkeit zu erreichen/halten. Im Weltmodell ist für den Hinterradmotor ein recht hohes maximales Drehmoment von 120 Nm erlaubt, um Beschleunigungsvorgaben schnell umsetzen zu können. Allerdings ist die erreichbare Drehzahl in der Praxis auch durch die Antriebsspannung bzw. Webots-Parameter `maxVelocity` limitiert. In S-Kurven-Simulationen⁸ wurde typischerweise eine Grundgeschwindigkeit von ca. 7–10 km/h gefahren, was durch den entsprechenden `velocity`-Wert in Rad/s vorgegeben wird.

- **Kraft-/Drehmomentmodus:** Alternativ kann man in Webots Motoren auch direkt ein Drehmoment vorgeben (`setTorque(t)`), um z.B. eigene Regelalgorithmen für die Positions- und Geschwindigkeitssteuerung umzusetzen. In unserem Modell wird dieser Modus nicht explizit genutzt, da die beiden in Steuerungsmodellen gängigen Modi bereits ausreichen. Dennoch ist es wichtig zu wissen, dass Webots stets mit Kräften/Momenten rechnet.

Die beiden Motoren sind im Weltfile als Geräte `motor::handlebars` und `motor::wheel` z.B. deklariert und jeweils dem passenden HingeJoint zugeordnet. Webots erlaubt es, mehrere Motoren pro Gelenk zu definieren, doch hier ist jeweils nur einer aktiv.

Die Namensgebung `motor::wheel` für den Hinterradantrieb ist ein Spezialfall: Das Präfix `::` dient dazu, den Motor vom gleichnamigen (per DEF) vorderen Rad zu unterscheiden, welches ebenfalls als `wheel` benannt ist, so können Kollisionen in der Namensauflösung vermieden werden. Im Controller-Code greift man dann genau mit diesem Namen zu.

Rolle im Regelkreis:

Die Aktuatoren setzen die vom Regler berechneten Sollgrößen in physikalische Bewegung um und schließen damit die Wirkkette des Regelkreises. Konkret berechnet der Balance-Controller in jedem Zeitschritt zwei Stellwerte:

- den **Lenksollwinkel** δ_{sol} (aus dem PID-Regler + Vision-Überlagerung), und
- die **Antriebs-Sollgeschwindigkeit** v_{sol} .

Diese Sollwerte werden direkt an die Motoren übergeben. Wie in Abschnitt 4.4 erläutert, fusioniert der Controller den **Balance-Anteil** (zur Selbstausrichtung) mit dem **Vision-Anteil** (zur Kurvenfahrt) zu einer endgültigen Vorgabe `final_steer`. Dieses `final_steer` entspricht einem Winkel

⁸In Kapitel 6.2.2 wurde eine S-Kurve simuliert.

in Radiant, der begrenzt auf den physikalisch möglichen Bereich (± 0.524 rad) an den Lenkmotor übergeben wird.

Der Hinterrad-Sollwert `target_speed` wird ebenfalls bestimmt, im aktuellen Konzept abhängig vom Lenkeinschlag, (starker Lenkeinschlag $\rightarrow$ automatische Reduktion der Grundgeschwindigkeit). Damit fungiert der Balance-Controller gleichzeitig als einfacher **Geschwindigkeitsmanager**, der z. B. in Kurven abbremst, um die Balance nicht zu gefährden. Die Berechnung von `target_speed` erfolgt durch einen separaten Algorithmus (`speed_pid_compute`), der den Vision-Lenkbefehl in eine prozentuale Reduktion der Grundgeschwindigkeit umsetzt.

Sobald die Stellgrößen vorliegen, werden sie an die Motor-API übergeben. Damit findet die **Regler-Ausgabe** statt, die Software schreibt die Sollwerte in die Motoren und die Physiksimulation reagiert, indem sie die entsprechenden Kräfte auf das Modell einleitet.

Bereits während der Initialisierung werden die Motoren konfiguriert, Listing 3 zeigt, wie der Lenkmotor und der Radmotor als Devices geholt und in die gewünschten Modi versetzt werden. Für den Lenkantrieb wird die Position initial auf 0.0 gestellt. Für den Hinterradantrieb wird `wb_motor_set_position(wheel_motor, INFINITY)` gesetzt, was (wie oben beschrieben) den Geschwindigkeitsmodus aktiviert. Beide Motoren würden standardmäßig mit `velocity = 0` starten und der Hinterradmotor erhält später im laufenden Betrieb eine Startgeschwindigkeit (Basiswert).

Im Regelprozess selbst (Hauptschleife) werden dann die aktuellen Stellbefehle übergeben:

Listing 3: Motorsteuerung im Regelkreis

```
wb_motor_set_position(handlebars_motor, final_steer);
wb_motor_set_velocity(wheel_motor, target_speed);
```

Der Wert `target_speed` wird in Rad/Sekunde an das Hinterrad übergeben. Beispielsweise entspricht eine Zielgeschwindigkeit von z. B. 4 m/s (bei ~ 0.7 m Raddurchmesser) ungefähr 11.4 rad/s Winkelgeschwindigkeit.

Nachdem die Sollwerte gesetzt sind, greift die Webots-Physik die neuen Sollwerte im nächsten Zeitschritt auf und beschleunigt/verstellt die Motoren entsprechend. Im Code werden nach dem Setzen der Motoren noch Statuspakete versendet und Logging betrieben, was für die Aktuatorwirkung selbst aber keine Rolle mehr spielt.

Insgesamt ist die Motoransteuerung in Webots sehr direkt, ein Funktionsaufruf genügt um das physikalische Stellglied zu beeinflussen. Dies erlaubt es, die Regelalgorithmen unkompliziert mit der Simulationswelt zu koppeln. Listing 3 fasst die wichtigsten Schritte zusammen. Hier sind sowohl die Initialisierung, als auch die Laufzeitschleife der Motoren zu sehen.

```
1 // Motoren-Devices abrufen
2 handlebars_motor = wb_robot_get_device("motor::handlebars");
3 wheel_motor      = wb_robot_get_device("motor::wheel");
4 if (handlebars_motor == 0 || wheel_motor == 0) {
5     exit(1);
6 }
7 // Initialisierung der Motoren
8 // Lenkmotor: Positionsmodus, Startwinkel = 0 rad
9 wb_motor_set_position(handlebars_motor, 0.0);
10 // Hinterradmotor: Geschwindigkeitsmodus aktivieren
11 wb_motor_set_position(wheel_motor, INFINITY);
12 // (Velocity bleibt zunächst 0, wird später gesetzt)
13
14 //...(im Reglerloop, nach PID/MPC-Berechnung von final_steer/target_speed)
15
```

```

16 // Lenkantrieb: Soll-Lenkwinkel setzen
17 wb_motor_set_position(handlebars_motor, -final_steer);
18 // Radantrieb: Soll-Drehgeschwindigkeit setzen
19 wb_motor_set_velocity(wheel_motor, target_speed);

```

Listing 4: IMU-Initialisierung und -Abfrage im C-Controller (Balance-Regelung)

4.6 Controller

Ein Webots-Controller ist ein separates Prozess, der einen Roboter in der Simulation steuert und über eine definierte API mit der Simulationswelt kommuniziert. Für jeden Roboter kann ein Controller in C/C++, Python, Java oder MATLAB betrieben werden, wobei Webots den Datenaustausch (Sensorwerte, Aktuatorbefehle) bei jedem Simulationsschritt synchronisiert. Typischerweise durchläuft ein Controller eine Endlosschleife, in der er Sensoren liest, Berechnungen durchführt und Aktuatoren ansteuert, wobei am Ende jeder Iteration die Funktion `wb_robot_step()` aufgerufen werden muss. Dieser Aufruf übergibt die gewünschte Control-Step-Dauer (in Millisekunden) an den Simulator und bewirkt, dass Webots die Simulation um genau diesen Zeitschritt fortsetzt und dabei alle bis dahin vom Controller gesetzten Befehle anwendet. Danach erhält der Controller die aktualisierten Sensorwerte des neuen Simulationszustands zurück. Abbildung 9 veranschaulicht diese Synchronisation [5].

Alle Anweisungen vor dem nächsten Zeitschritt (`wb_robot_step`-Aufruf) werden zunächst gesammelt. Mit dem Step-Aufruf rechnet der Simulator den nächsten Physikschrift und aktualisiert die Sensoren, bevor der Controller weiterläuft. Auf diese Weise laufen Simulation und Controller *synchron* in gleichmäßigen Zeitschritten. Standardmäßig wartet Webots in diesem synchronen Modus darauf, dass jeder Controller seinen `wb_robot_step` ausführt, bevor der nächste Simulationsschritt erfolgt. Dies garantiert Konsistenz, erlaubt aber auch differierende Abtastraten, wobei der Kontrollschritt ein ganzzahliges Vielfaches des Simulations-Basisschritts sein muss.

Beispielsweise kann bei einer Weltzeit Schrittauflösung von 2ms ein Controller mit 2ms, 10ms oder 50ms Schrittweite betrieben werden. Webots führt dann pro Controller-Schritt entsprechend 1, 5 bzw. 25 Simulationsschritte aus und wartet am Ende ggf. auf den Controller. Diese Entkopplung nutzt Webots, um auch langsamere Controller (z. B. in Python) in eine schnelle Physiks simulation einzubetten, ohne die Synchronisation zu verlieren.

Um auf Sensoren und Aktuatoren zuzugreifen, stellt Webots gerätespezifische API-Funktionen bereit. Im C-Controller werden z. B. Geräte über `wb_robot_get_device("name")` referenziert und Sensoren mit `wb_sensor_enable()` aktiviert. Entsprechende Methoden existieren dabei ebenfalls in Python (`robot.getDevice()`, `device.enable()`).

Ein Sensor liefert erst ab dem nächsten `wb_robot_step()`

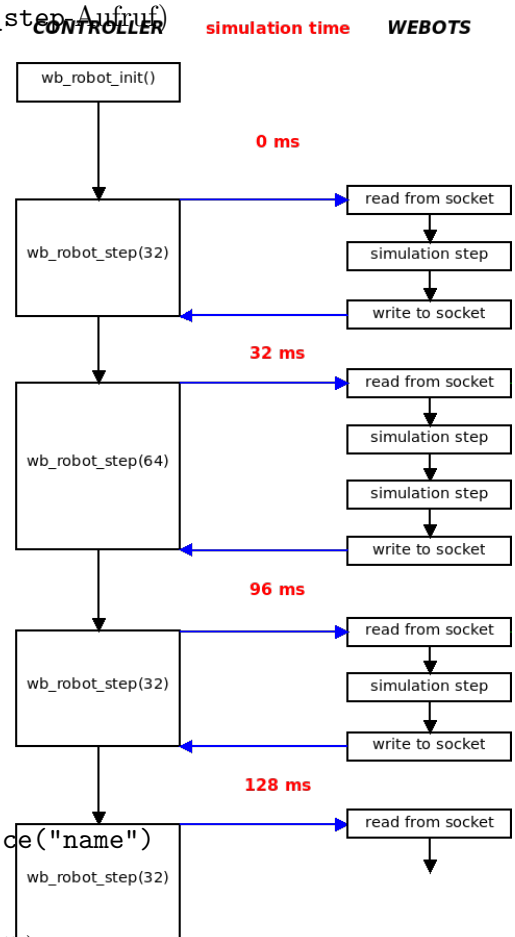


Figure 9: Synchronisation zwischen Controller und Simulation.

gültige Messwerte, da die Simulationszeit dann fortgeschritten ist.

Aktuatorbefehle (z.B. `wb_motor_set_position()` oder `motor.setPosition()` in Python) werden zunächst lokal gepuffert. Die tatsächliche Ausführung in der Physik-Engine erfolgt erst am nächsten Step-Synchronisationspunkt. Ebenso bleiben zwei unmittelbar nacheinander gelesene Sensorsamples identisch, solange kein Simulationsschritt dazwischen liegt. Dieses deterministische Schritt-für-Schritt- Ausführungsmodell erleichtert die Reproduzierbarkeit und verhindert *race conditions*⁹ zwischen Physik und Reglercode.

Da für das selbstbalancierende Fahrrad zwei Controller-Prozesse implementiert wurden, die in Webots zeitgleich laufen und über die oben beschriebene Mechanik gekoppelt sind, wurden einerseits der *Balance-Controller* auf dem Fahrrad (in C, hoher Takt), andererseits ein *Vision-Controller* auf einem Supervisor-Knoten (in Python, langsamer Takt) implementiert. Die Welt-Datei ist dabei so konfiguriert, dass das Fahrrad einen **Receiver** und einen **Emitter** trägt, während der Supervisor das komplementäre Gegenstück besitzt, damit entsteht ein bidirektionaler Nachrichtenkanal innerhalb der Simulation:

```
1  Robot {
2      Controller "balance_control_c"
3      Receiver { name "command_rx"  channel 1  bufferSize 64 }
4      Emitter  { name "status_tx"   channel 2  bufferSize 64 }
5  }
6  Supervisor {
7      Controller "vision_control_py"
8      Emitter  { name "command_tx"  channel 1  bufferSize 64 }
9      Receiver { name "status_rx"   channel 2  bufferSize 64 }
10 }
```

Listing 5: Nachrichtenkanal zwischen Controller und Simulation

Über Channel 1 sendet der Vision-Controller Steuerkommandos an das Fahrrad, und über Channel 2 übermittelt der Balance-Controller seinerseits Statusdaten zurück (z. B. aktueller Rollwinkel, Lenkwinkel, Geschwindigkeit). Beide Controller laufen im *synchronen Modus*, d. h. Webots hält die Physik so lange an, bis sowohl der schnelle C-Controller (alle 2 ms) als auch der langsamere Python-Controller ihren Schritt abgearbeitet haben [6] [7]. Praktisch bedeutet dies, dass der Balance-Regler bei jedem Simulationsschritt neue Motorbefehle berechnet, während der Vision-Algorithmus nur alle 25 Schritte (50 ms) ein neues Lenkkommando liefert. , Webots gewährleistet die Synchronisierung beider Teilprozesse, sodass Kommandos verlustfrei und deterministisch am Step-Synchronisationspunkt ausgetauscht werden.

4.7 Integriertes Konfigurationssystem

Die Regelungsarchitektur erfordert eine koordinierte Parametrierung sowohl des Balance-Controllers in C als auch des Vision-Controllers in Python. Zu diesem Zweck wurde ein JSON-basiertes Konfigurationssystem implementiert, das als zentrale Kommunikationsschnittstelle zwischen einem grafischen Benutzerinterface und den beiden Regelschleifen fungiert. Dieses System ermöglicht die Laufzeitanpassung der Regelungsparameter ohne Unterbrechung der Webots-Simulation und gewährleistet eine konsistente Parametrierung über alle Systemkomponenten hinweg.

⁹Race Conditions bezeichnen unvorhersehbares Verhalten durch konkurrierende Zugriffe.

Alle relevanten Parameter werden in der Datei `balance_config.json` strukturiert abgelegt. Diese enthält sowohl die PID-Parameter für die Balance-Regelung (Kp, Ki, Kd, Ausgangsbegrenzungen) als auch Vision-spezifische Einstellungen (YOLO-Konfidenz, Fallback-ROI-Parameter, Lenkglättung). Die JSON-Struktur ermöglicht eine hierarchische Organisation der Parameter in logische Gruppen (`angle_pid`, `speed_control`, `vision_control`), was sowohl die Wartbarkeit als auch die automatisierte Verarbeitung erleichtert. Listing 6 zeigt die Struktur der zentralen Konfigurationsdatei mit den wichtigsten Parametergruppen.

```

1 {
2   "balance_control": {
3     "angle_pid": {
4       "angle_Kp": 4.5,
5       "angle_Ki": 0.2,
6       "angle_Kd": 0.4,
7       "angle_output_min": -0.3,
8       "angle_output_max": 0.3,
9       "angle_integral_min": -60.0,
10      "angle_integral_max": 60.0
11    },
12    "speed_control": {
13      "base_speed": 6.172839506172839,
14      "min_speed": 3.0864197530864197,
15      "max_speed": 6.172839506172839
16    },
17    "mechanical_limits": {
18      "max_handlebar_angle": 1.5,
19      "max_roll_angle": 45.0
20    },
21    "system": {
22      "enable_logging": 1,
23      "enable_preview": 1,
24      "config_reload_interval": 10,
25      "filter_size": 5
26    },
27    "vision_control": {
28      "method": "yolo",
29      "yolo_conf": 0.5,
30      "yolo_show": 0,
31      "fallback_roi_top_frac": 0.5,
32      "fallback_roi_height_frac": 0.06,
33      "steer_max_delta": 0.001,
34      "steer_max_cmd": 0.06
35    }
36  }
37 }

```

Listing 6: Struktur der zentralen Konfigurationsdatei `balance_config.json`

Der Balance-Controller in C implementiert in `balance_config.c` ein vereinfachtes JSON-Parser, das die Konfigurationsdatei periodisch einliest und die Parameter zur Laufzeit aktualisiert. Die Implementierung verzichtet auf externe JSON-Bibliotheken und nutzt stattdessen String-basierte Suchfunktionen für die Extraktion spezifischer Schlüssel-Wert-Paare. Dies gewährleistet eine schlanke Integration ohne zusätzliche Abhängigkeiten in der zeitkritischen Balance-Regelschleife. Über `balance_config_validate()` werden geladene Parameter auf Plausibilität geprüft, um Systeminstabilitäten durch ungültige Konfigurationen zu vermeiden.

Der Vision-Controller lädt dieselbe JSON-Konfiguration über Pythons native `json`-Bibliothek und

extrahiert vision-spezifische Parameter. Ein konfigurierbares Reload-Intervall ermöglicht die dynamische Anpassung von Vision-Parametern (z. B. YOLO-Konfidenz, ROI-Dimensionen) während der laufenden Simulation. Diese Architektur unterstützt sowohl YOLO-basierte Spurerkennung als auch Fallback-Algorithmen, deren Parameter über die gemeinsame Konfigurationsdatei gesteuert werden.

Die Kommunikation zwischen den Controllern wird durch ein strukturiertes Logging-System in `balance_logging.c` dokumentiert. Dieses System erzeugt zeitgestempelte CSV-Dateien, die sowohl Balance-relevante Größen (Rollwinkel, Lenkausgang, PID-Terme) als auch Vision-Daten (Spurabweichung, Vision-Kommandos, Maskenabdeckung) sowie Positionsinformationen enthalten. Die erweiterte Datenstruktur `balance_log_data_t` kapselt alle relevanten Systemzustände in einem einheitlichen Format, wodurch eine umfassende Systemanalyse ermöglicht wird.

Das grafische Interface fungiert als Vermittler zwischen Benutzer und Konfigurationssystem. Parameteränderungen werden unmittelbar in die JSON-Konfiguration geschrieben, von wo sie durch die periodischen Reload-Mechanismen der Controller aufgenommen werden. Gleichzeitig visualisiert das Interface die von `balance_logging.c` erzeugten CSV-Daten in Echtzeit, wodurch die Auswirkungen von Parameteränderungen sofort sichtbar werden. Diese Architektur ermöglicht ein iteratives Tuning der Regelungsparameter unter direkter Beobachtung des Systemverhaltens.

5 Kombinierte Regelung

5.1 Portierung des Balance-PID (Zander)

Jonah Zanders Bachelorarbeit (2023) implementiert eine dreistufige Kaskadenregelung für ein selbstbalancierendes Fahrrad, die als Grundlage für die in dieser Arbeit entwickelte Simulationsregelung dient [31]. Die Hardware-Architektur umfasst einen BNO055-IMU-Sensor zur Neigungsmessung, einen BeagleBone-Black-Einplatinencomputer als Steuerungseinheit sowie separate Motorcontroller für Lenkung und Antrieb. Die Regelungsarchitektur gliedert sich in drei hierarchische Ebenen: eine primäre Balance-PID-Regelung (Angle-to-Steering), eine sekundäre Lenkpositionsregelung und eine vereinfachte Geschwindigkeitssteuerung.

Primäre Balance-Regelung: Das Kernstück von Zanders Implementierung bildet ein PID-Regler, der die Rollwinkelabweichung in einen Lenkwinkel-Sollwert umsetzt. Listing 7 zeigt die von Zander gewählten PID-Parameter, die sich durch hohe proportionale ($K_p = 10,0$) und differentielle Verstärkungen ($K_d = 2,2$) bei deaktiviertem Integralanteil ($K_i = 0,0$) auszeichnen. Diese Parametrierung gewährleistet eine schnelle Reaktion auf Neigungsänderungen ohne die Gefahr des Aufschaukelns durch Drift-Integration.

```
1 //PID Angle to Steering
2 #define ANGLE_PID_KP          10.0f
3 #define ANGLE_PID_KI          0.0f
4 #define ANGLE_PID_KD          2.2f
5 #define ANGLE_PID_OUTPUT_MIN  -150.0f
6 #define ANGLE_PID_OUTPUT_MAX   150.0f
7 #define ANGLE_PID_INTEGRAL_MIN -60.0f
8 #define ANGLE_PID_INTEGRAL_MAX 60.0f
```

Listing 7: PID-Parameter in Zanders Hardware-Implementierung

Die PID-Berechnung in Zanders System implementiert eine zeitbasierte Integration und Differentiation mit Historien-Pufferung zur Rauschunterdrückung. Der Differentialterm wird über mehrere Zeitschritte gemittelt, um hochfrequente Störungen zu dämpfen, während der Integralterm durch Anti-Windup-Mechanismen begrenzt wird.

Sekundäre Lenkpositionsregelung: Die zweite Regelebene steuert den physischen Lenkmotor über einen MD49-Controller an, um den von der Balance-PID vorgegebenen Sollwinkel zu erreichen. Aufgrund der mechanischen Eigenschaften des Lenkantriebs (Reibung, Trägheit, Totzeit) sind hier deutlich höhere Verstärkungen erforderlich:

```
1 //PID Steering PID
2 #define MD49_PID_KP          850.0f
3 #define MD49_PID_KI          300.0f
4 #define MD49_PID_KD          30.0f
5 #define MD49_PID_OUTPUT_MIN  -100000.0f
6 #define MD49_PID_OUTPUT_MAX  100000.0f
```

Listing 8: Lenkpositions-PID-Parameter in Zanders System

Echtzeit-Threading-Architektur: Zanders Implementierung nutzt eine multi-threaded Architektur mit unterschiedlichen Abtastraten: IMU-Datenerfassung mit 200 Hz, Lenkpositionsregelung mit 1000 Hz und Geschwindigkeitsregelung mit niedrigerer Frequenz. Watchdog-Mechanismen überwachen die Thread-Ausführung und gewährleisten die Systemstabilität bei Hardwareausfällen.

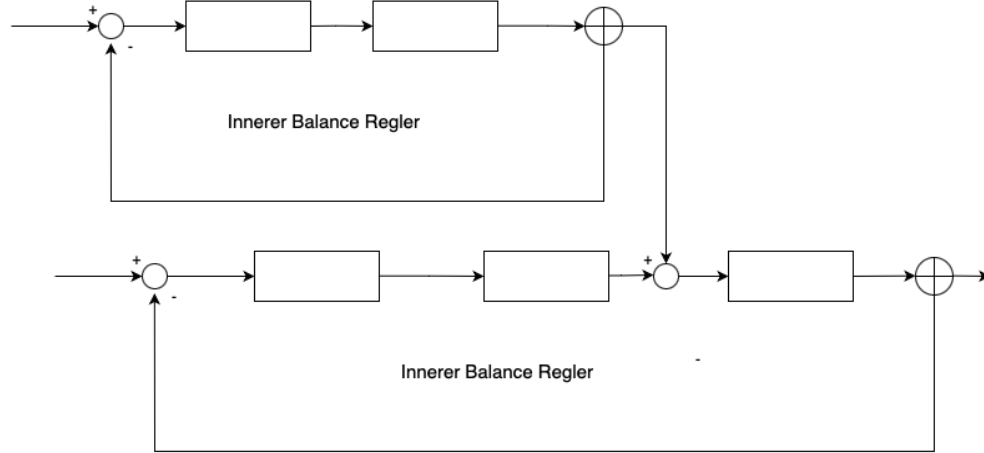


Figure 10: Kombimierter Regler.

Adaptierung für die Webots-Simulation: Die Portierung von Zanders Balance-Regelung in die Webots-Simulationsumgebung erforderte eine systematische Vereinfachung der Hardware-spezifischen Komplexitäten bei Beibehaltung der regelungstechnischen Kernprinzipien. Die wesentlichen Unterschiede zwischen Hardware- und Simulationsimplementierung zeigt Tabelle 1.

Table 1: Vergleich zwischen Zanders Hardware-Implementierung und Webots-Simulation

Aspekt	Zander (Hardware)	Webots-Simulation
Sensorik	BNO055 IMU, Rauschen	Idealer Rollwinkelsensor
Threading	Multi-threaded (200/1000 Hz)	Synchrone Hauptschleife
Lenkpositionsregelung	Separater MD49-PID	Direkter Motoranschluss
PID-Parameter	$K_p = 10, K_i = 0, K_d = 2.2$	Identische Werte
Ausgangsbegrenzung	± 150 (Hardware-Einheiten)	± 0.3 rad

Die Webots-Implementierung vereinfacht die Architektur erheblich durch Elimination der sekundären Lenkpositionsregelung. Listing 9 zeigt die zentrale Balance-Regelschleife, die direkt den Lenkwinkel setzt:

```

1 // 1. Roll-Winkel messen (idealisiert, ohne Rauschen)
2 float roll_angle = get_roll_rad_raw();
3
4 // 2. PID-Regelung: Roll-Winkel -> Lenkwinkel
5 if (control_enabled) {
6     steering_output = -pid_compute(&angle_pid, 0.0, roll_angle, current_time);
7 }
8
9 // 3. Lenkwinkel begrenzen
10 if (steering_output > config.mechanical_limits.max_handlebar_angle)
11     steering_output = config.mechanical_limits.max_handlebar_angle;
12 else if (steering_output < -config.mechanical_limits.max_handlebar_angle)
13     steering_output = -config.mechanical_limits.max_handlebar_angle;
14
15 // 4. Direkter Motoranschluss (keine sekundäre Regelung)
16 wb_motor_set_position(handlebars_motor, -final_steer);

```

Listing 9: Vereinfachte Balance-PID-Implementierung in Webots

Der PID-Algorithmus selbst wurde von Zanders Implementierung adaptiert, jedoch mit verbesserter numerischer Stabilität. Während Zanders Code eine komplexe Historien-Verwaltung mit zirkulären Puffern verwendet, nutzt die Webots-Version eine vereinfachte Zeitberechnung basierend auf dem deterministischen Simulationszeitschritt:

```

1 // I-Term (Integral) - vereinfachte Zeitberechnung
2 int prev_index = (pid->history_counter + HISTORY_LEN - 1) % HISTORY_LEN;
3 float dt = (pid->time_history[pid->history_counter] -
4             pid->time_history[prev_index]) / 1000000.0; // Mikrosekunden ->
                    Sekunden
5
6 if (dt > 0.0 && dt < 1.0) { // Plausibilitaetspruefung
7     pid->integral_term += pid->Ki * error * dt;
8 }
9
10 // D-Term mit gleitendem Durchschnitt (wie bei Zander)
11 float current_derivative = (error - pid->error_history[prev_index]) / dt;
12 pid->derivative_history[pid->history_counter] = current_derivative;
13
14 float derivative_sum = 0.0;
15 for (int i = 0; i < HISTORY_LEN; i++) {
16     derivative_sum += pid->derivative_history[i];
17 }
18 pid->derivative_term = pid->Kd * (derivative_sum / HISTORY_LEN);

```

Listing 10: Vereinfachte PID-Berechnung in der Webots-Implementierung

Die erfolgreiche Übertragung von Zanders Regelungskonzept in die Simulation bestätigt die Robustheit des gewählten PID-Ansatzes. Die identischen Parameter ($K_p = 10$, $K_i = 0$, $K_d = 2,2$) erzielen auch in der idealisierten Webots-Umgebung stabile Balance-Regelung. Der Verzicht auf den Integralanteil erweist sich als vorteilhaft, da er Drift-bedingte Instabilitäten vermeidet, was ein Aspekt, der sowohl in der Hardware- als auch in der Simulationsumgebung relevant ist.

Die Portierung demonstriert, wie Hardware-spezifische Komplexitäten (Multi-Threading, Sensor-Rauschen, Aktor-Totzeiten) in einer Simulationsumgebung abstrahiert werden können, ohne die regelungstechnischen Kernprinzipien zu beeinträchtigen. Diese Vereinfachung schafft die Grundlage für die Integration zusätzlicher Regelungsebenen, insbesondere der vision-basierten Pfadführung, die in den folgenden Abschnitten behandelt wird.

5.2 Portierung des Fahrtrichtungs-MPC (Giray)

Yasin Giray entwickelte in seiner Masterarbeit einen vision-basierten Fahrtrichtungsregler auf Basis von Model Predictive Control (MPC), um das selbstbalancierende Fahrrad autonom der Fahrspur folgen zu lassen [12]. Dieser Regler modelliert die Fahrzeugdynamik mittels Einspurmodell und nutzt zwei Steuergrößen, die aktuelle Geschwindigkeit v und Lenkwinkel ϕ als Eingaben für die Prädiktion. Der MPC berechnet also auf Grundlage von v und Lenkeinschlag die Fahrzeugbewegung voraus. Für den prädiktiven Regler wird ein Zustandsvektor $x = [x, y, v, \text{yaw}]$ (Position, Geschwindigkeit, Gierwinkel) und ein Eingangsvektor $u = [\text{accel}, \text{steer}]$ definiert. Giray wählte einen Zeithorizont von etwa 2 s und formulierte ein Optimierungsproblem, das die Abweichung von einer Soll-Trajektorie minimiert sowie Steuerungsaufwand und -änderungen übernimmt. Die Kostenfunktion des MPC addiert quadrierte Abweichungen der Zustände und **Eingriffe über den Horizont sowie eine** Endzustands-Strafe, z. B. $J = \sum (x^T Q x + u^T R u + \Delta u^T R_d \Delta u) + x_{\text{final}}^T Q_f x_{\text{final}}$. Neben den Kosten wer-

den physikalische Nebenbedingungen wie maximaler Lenkwinkel, Lenkwinkelrate und Geschwindigkeit im Optimierungsproblem berücksichtigt, um realistische Lösungen zu gewährleisten.

Wesentliche Voraussetzung für den Fahrtrichtungs-MPC ist eine geeignete Referenztrajektorie der Straße. Giray implementierte hierfür eine Computer-Vision-Pipeline auf Basis von YOLOv8-Segmentierung, welche im Kamerabild die befahrbare Straße und deren Verlauf erkennt. Aus den segmentierten Bilddaten wird eine Reihe von Stützpunkten der Fahrbahn extrahiert. Anschließend wird mittels Spline-Interpolation eine glatte Kurve durch diese Punkte gelegt, die als Referenzkurve für den MPC dient. Er simuliert auf Basis des Einspurmodells die Fahrzeugbewegung und berechnet die optimalen Steuerbefehle, damit das Fahrrad der Kurve bestmöglich folgt. In Girays Umsetzung wird der Regler zyklisch mit ca. 10 Hz ausgeführt (Tickrate 100 ms). Pro Zyklus initialisiert der Controller den aktuellen Fahrzeugzustand (Position, Geschwindigkeit, Orientierung) und berechnet aus der ersten Sollkurve einen Vorhersagepfad. Die Optimierung liefert dann eine Stellgrößen-Sequenz für Beschleunigung und Lenkwinkel. Tatsächlich wird nur der erste Lenkwinkel-Impuls umgesetzt (Receding Horizon Control), welcher als neuer Lenkbefehl an das Fahrzeug geht. Im nächsten Zyklus wird der MPC erneut mit aktualisiertem Zustand und ggf. angepasster Referenz gestartet. Giray konnte in Simulationen zeigen, dass der MPC-Regler das Fahrrad effektiv der vorgegebenen Spur folgen lässt. Die geplante Spline-Trajektorie und die vom MPC gefahrene Ist-Trajektorie in seiner Arbeit, stimmen dabei eng überein, was die Güte der prädiktiven Regelung belegt.

Adaptierung für Webots-Simulation: Die Portierung von Girays MPC-Konzept in die Webots-Umgebung erforderte eine systematische Anpassung an die spezifischen Anforderungen der Simulation und Integration mit dem vorhandenen Balance-Controller. Tabelle 2 zeigt die wesentlichen Unterschiede zwischen Girays theoretischem Ansatz und der Webots-Implementierung.

Table 2: Vergleich zwischen Girays MPC-Konzept und Webots-Implementierung

Aspekt	Giray (Theorie)	Webots-Implementierung
Prädiktionshorizont	N=20 (2 s - 10 Hz)	T=3 (0.3 s - 20 Hz)
Referenzgenerierung	Spline-Interpolation	Vereinfachte Korrekturtrajektorie
Zustandsvektor	$[x, y, v, \psi]^T$	$[x, y, v, yaw]^T$ (identisch)
Eingangsvektor	$[a, \delta]^T$	$[accel, steer]^T$ (identisch)
Regelfrequenz	10 Hz	20 Hz
Optimierer	CVXPY (theoretisch)	CVXPY (praktisch)
Integration	Standalone	Gekoppelt mit Balance-PID

Die Webots-Implementierung vereinfacht Girays Ansatz erheblich durch Reduzierung des Prädiktionshorizonts auf 3 Zeitschritte ($\approx 0,3$ s). Diese Entscheidung wurde aus Echtzeitgründen getroffen, da die Integration mit dem Balance-Controller eine höhere Regelfrequenz (20 Hz) erfordert. Listing 11 zeigt die Initialisierung des MPC-Controllers mit den angepassten Parametern:

```

1 def __init__(self):
2     # MPC-Parameter (reduziert gegenüber Giray)
3     self.NX = 4 # Zustandsvektor: [x, y, v, yaw]
4     self.NU = 2 # Eingänge: [accel, steer]
5     self.T = 3 # Prädiktionshorizont (reduziert fuer Echtzeitfahigkeit)
6
7     # Kostenmatrizen (aeahnlich Girays Formulierung)
8     self.R = np.diag([0.01, 0.01]) # Eingangskostenmatrix
9     self.Rd = np.diag([0.01, 1.0]) # Eingangsdifferenzkostenmatrix

```

```

10 self.Q = np.diag([1.0, 1.0, 0.5, 0.5]) # Zustandskostenmatrix
11 self.Qf = self.Q                      # Terminale Kostenmatrix
12
13 # Fahrzeugparameter (Bicycle-Modell)
14 self.WB = 1.2 # Radstand [m] (Fahrrad)
15 self.DT = 0.1 # Zeitschritt [s]
16
17 # Physikalische Begrenzungen
18 self.MAX_STEER = math.radians(30.0) # Maximaler Lenkwinkel
19 self.MAX_DSTEER = math.radians(20.0) # Maximale Lenkwinkelrate
20 self.MAX_SPEED = 8.0                 # Maximale Geschwindigkeit
21 self.MIN_SPEED = 0.5                 # Minimale Geschwindigkeit

```

Listing 11: MPC-Controller Initialisierung in der Webots-Implementierung

Anstelle von Girays komplexer Spline-Interpolation aus Bildpunkten verwendet die Webots-Implementierung eine vereinfachte Referenzgenerierung basierend auf dem aktuellen Vision-Fehler. Die Referenztrajektorie wird aus dem lateralen Spurbabweichungsfehler abgeleitet, indem ein Ziel-Gierwinkel berechnet und über den kurzen Horizont interpoliert wird.

MPC-Optimierungsalgorithmus: Die Webots-Implementierung folgt Girays theoretischer Formulierung bei der Lösung des Optimierungsproblems. Listing 12 zeigt den Kern des MPC-Algorithmus:

```

1 def linear_mpc_control(self, xref, xbar, x0):
2     """Loest MPC-Optimierungsproblem"""
3     try:
4         x = cvxpy.Variable((self.NX, self.T + 1))
5         u = cvxpy.Variable((self.NU, self.T))
6
7         cost = 0.0
8         constraints = []
9
10        for t in range(self.T):
11            cost += cvxpy.quad_form(u[:, t], self.R)
12
13            if t != 0:
14                cost += cvxpy.quad_form(xref[:, t] - x[:, t], self.Q)
15
16            A, B, C = self.get_linear_model_matrix(
17                xbar[2, t], xbar[3, t], 0.0) # Linearisierung um Referenz
18            constraints += [x[:, t + 1] == A @ x[:, t] + B @ u[:, t] + C]
19
20            if t < (self.T - 1):
21                cost += cvxpy.quad_form(u[:, t + 1] - u[:, t], self.Rd)
22                constraints += [cvxpy.abs(u[1, t + 1] - u[1, t]) <= self.
23                               MAX_DSTEER * self.DT]
24
25            cost += cvxpy.quad_form(xref[:, self.T] - x[:, self.T], self.Qf)
26
27            # Randbedingungen
28            constraints += [x[:, 0] == x0]
29            constraints += [x[2, :] <= self.MAX_SPEED]
30            constraints += [x[2, :] >= self.MIN_SPEED]
31            constraints += [cvxpy.abs(u[0, :]) <= self.MAX_ACCEL]
32            constraints += [cvxpy.abs(u[1, :]) <= self.MAX_STEER]
33
34            # Problem loesen
35            prob = cvxpy.Problem(cvxpy.Minimize(cost), constraints)

```



```

35     prob.solve(verbose=False)
36
37     if prob.status == cvxpy.OPTIMAL or prob.status == cvxpy.OPTIMAL_INACCURATE
38         :
39         u_opt = u.value
40         if u_opt is not None:
41             return u_opt[0, 0], u_opt[1, 0] # Erste Kontrollaktion
42
43     return self._simple_control_fallback(xref, x0)
44
45 except Exception as e:
46     return self._simple_control_fallback(xref, x0)

```

Listing 12: MPC-Optimierungsalgorithmus in der Webots-Implementierung

Die Implementierung folgt dem Standard-MPC-Schema mit quadratischen Kosten und linearen Constraints. Im Gegensatz zu Girays längerfristigem Horizont ermöglicht die Reduzierung auf 3 Zeitschritte eine Echtzeitausführung mit 20 Hz.

Integration mit Balance-Controller: Ein wesentlicher Unterschied zu Girays Standalone-Ansatz liegt in der Integration mit dem bestehenden Balance-PID-Regler. Die Webots-Implementierung realisiert eine Kaskadenregelung, bei der der MPC-Controller die langfristige Spurführung übernimmt, während der Balance-PID die kurzfristige Stabilisierung gewährleistet. Die MPC-Ausgaben werden normalisiert und über IPC an den C-basierten Balance-Controller übertragen:

```

1 def send_vision_command(self, steer_cmd, speed_cmd):
2     """Sendet erweiterte Vision-Command an Balance-Controller"""
3     try:
4         # Begrenzungen anwenden
5         steer_cmd = max(-1.0, min(1.0, steer_cmd))
6         speed_cmd = max(0.0, min(1.0, speed_cmd))
7
8         # Erweiterte Struct-Format: 8 Felder
9         # (steer_command, speed_command, valid, vision_error,
10        # vision_p_term, vision_i_term, vision_d_term, mask_coverage)
11        command_data = struct.pack('ffffff',
12                                   steer_cmd, speed_cmd, 1,
13                                   self.vision_error, self.mpc_controller.mpc_p_term
14                                   ,
15                                   self.mpc_controller.mpc_i_term, self.
16                                   mpc_controller.mpc_d_term,
17                                   self.vision_mask_coverage)
18
19        self.command_emitter.send(command_data)
20        return True
21    except Exception as e:
22        return False

```

Listing 13: IPC-Übertragung der MPC-Kommandos an den Balance-Controller

Bewertung der Portierung: Die Adaptation von Girays MPC-Konzept für die Webots-Simulation demonstriert sowohl die Übertragbarkeit theoretischer MPC-Ansätze als auch die Notwendigkeit praktischer Kompromisse. Die vereinfachte Referenzgenerierung verzichtet auf Girays komplexe Spline-Interpolation, bietet jedoch ausreichende Genauigkeit für die Spurführung in der Simulation.

Die erfolgreiche Kopplung mit dem Balance-Controller zeigt, dass MPC-basierte Spurführung und PID-basierte Stabilisierung komplementäre Regelungsebenen darstellen können. Girays theoretischer Beitrag zur modellprädiktiven Fahrzeugführung findet somit praktische Anwendung in einem hybriden Regelungskonzept, das die Vorteile beider Ansätze kombiniert.

5.3 Strategien der Spurführung

Die Vision-basierte Spurführung des selbstbalancierenden Fahrrads nutzt zwei unterschiedliche Erkennungsstrategien: eine YOLO-basierte Segmentierung als Hauptverfahren und eine robuste Fallback-Strategie für Ausfallsituationen. Beide Verfahren extrahieren aus dem Kamerabild einen lateralen Spurabweichungsfehler, der vom zuvor beschriebenen MPC in Lenkbefehle umgesetzt wird.

5.3.1 YOLO-Segmentierung (YOLOv8)

Die YOLO-Strategie nutzt ein vortrainiertes YOLOv8-Segmentierungsmodell zur robusten Erkennung der Fahrbahn. Das neuronale Netz identifiziert Straßenbereiche als Klasse “street_main” und liefert sowohl Bounding Boxes als auch pixelgenaue Segmentierungsmasken, wie in Kapitel 3.3 beschrieben ist. Das System sucht beim Start nach einer trainierten Gewichtsdatei (`best.pt`) und lädt das YOLOv8-Modell auf das verfügbare Rechengerät. Bei Nichtverfügbarkeit erfolgt automatischer Wechsel zur Fallback-Strategie:

```

1  if YOLO_AVAILABLE:
2      possible_paths = [
3          "../balance_control_c/yolo_vision/runs/segment/train/weights/best.pt",
4          "yolo_weights/best.pt",
5          "best.pt"
6      ]
7      for path in possible_paths:
8          if os.path.exists(path):
9              try:
10                 device = "mps" if torch.backends.mps.is_available() else \
11                     ("cuda" if torch.cuda.is_available() else "cpu")
12                 self.yolo_model = YOLO(path).to(device)
13                 print(f"[OK] YOLO-Modell geladen: {path} (Device: {device})")
14                 break
15             except Exception as e:
16                 print(f"[WARN] Fehler beim Laden von {path}: {e}")
17                 continue
18         if not self.yolo_model:
19             print("[WARN] Kein YOLO-Modell gefunden - Fallback-Vision aktiviert")
20         YOLO_AVAILABLE = False

```

Listing 14: YOLO-Modell Initialisierung

Im Echtzeitbetrieb (20 Hz) analysiert YOLO jedes Kamerabild und filtert Straßenobjekte (Klasse “street_main”). Aus den Bounding Boxes aller erkannten Straßenbereiche wird der gemittelte Mittelpunkt berechnet. Die laterale Abweichung ergibt sich als normierte Differenz zwischen Straßenmittelpunkt und Bildmitte ($\text{error} \in [-0.5, +0.5]$):

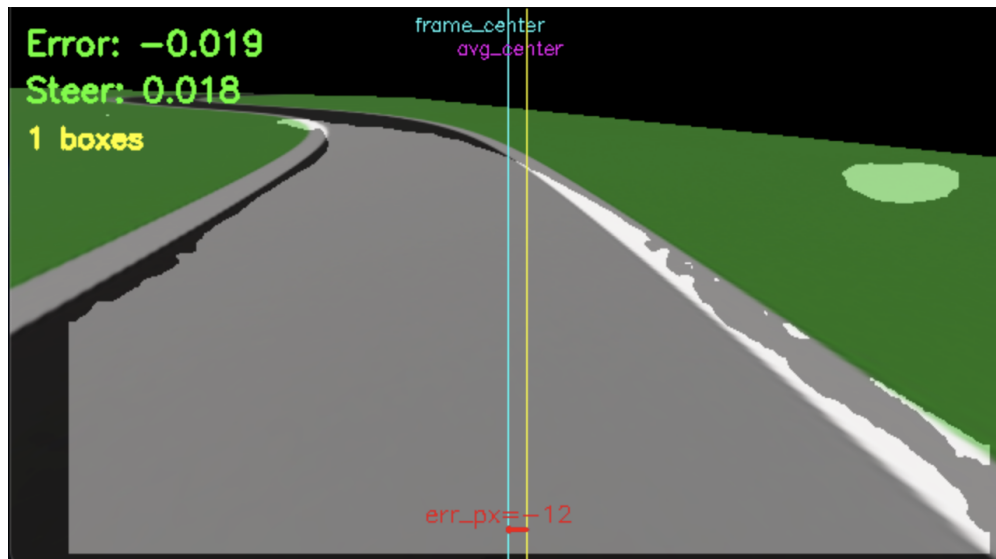


Figure 11: YOLOv8-Segmentierung.

```

1  street_indices = [idx for idx, cls in enumerate(r.bboxes.cls) if cls == 2]
2  if street_indices:
3      x_centers = []
4      for idx in street_indices:
5          xxyy = r.bboxes.xxyy[idx] # Koordinaten der Bounding Box
6          x_center = float((xxyy[0] + xxyy[2]) / 2.0)
7          x_centers.append(x_center)
8      avg_x_center = sum(x_centers) / len(x_centers)
9      frame_center = frame.shape[1] / 2
10     error = (frame_center - avg_x_center) / frame.shape[1] # Normiert
11     self._store_valid_values(error, mask, self.vision_mask_coverage)
12     return error, mask

```

Listing 15: YOLO-Fehlerberechnung

Das System implementiert eine Ausfallüberbrückung durch Zwischenspeicherung der letzten gültigen Fehlerwerte. Bei Detektionsaussetzern wird der gespeicherte Fehler mit Decay-Faktor schrittweise abgeschwächt, bis er nach definierten Ausbleib-Zyklen auf Null fällt. Dies verhindert abrupte Lenkbewegungen bei temporären Erkennungsverlusten.

Limitierungen: Die YOLO-Strategie erfordert ein geeignetes vortrainiertes Modell und ausreichende Rechenleistung. Bei Abweichungen von den Trainingsdaten können Fehlerkennungen auftreten. Diese Fehlerkennungen werden in Kapitel 6.2 beschrieben.

5.3.2 Fallback-Strategie (ROI-basiert)

Bei Nichtverfügbarkeit des YOLO-Modells aktiviert sich automatisch eine Fallback-Strategie, die auf klassischer Bildverarbeitung basiert. Diese nutzt Farb- und Konturfilter innerhalb einer definierten Region of Interest (ROI) zur Straßenerkennung, dabei mit geringerer Präzision, aber ohne Abhängigkeit von trainierten Modellen, bei denen zu fehlern bei der Straßenerkennung kommen kann.

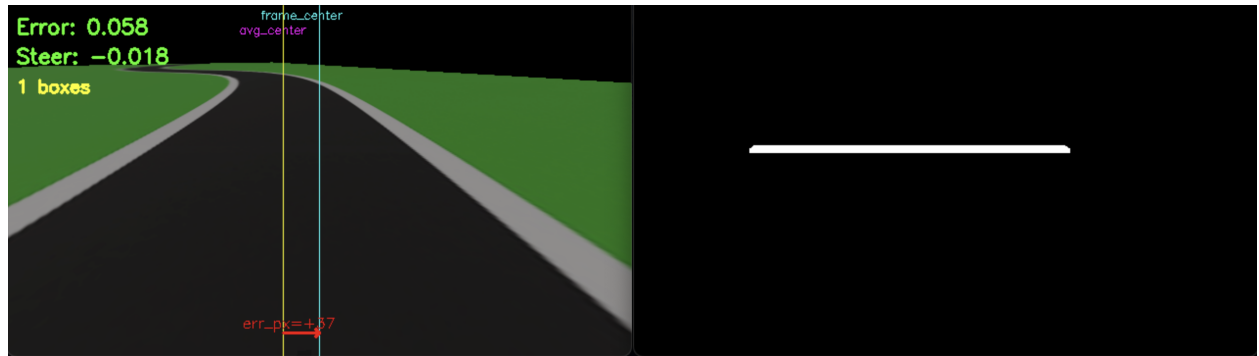


Figure 12: Fallback-Strategie.

Das System definiert eine trapezförmige ROI (ca. 40% Bildhöhe, 2% Breite) zur Fokussierung auf den erwarteten Fahrbahnbereich. Innerhalb dieser ROI erfolgt HSV-basierte Farbsegmentierung für dunkelgraue Straßenbereiche und weiße Markierungen, gefolgt von morphologischen Operationen zur Rauschreduktion. Die größte erkannte Kontur wird als Straßenbereich interpretiert:

```

1  mask_road = cv2.inRange(hsv, road_lower, road_upper)      # Graue Strasse
   filtern
2  mask_white = cv2.inRange(hsv, white_lower, white_upper)   # Weisse Markierungen
   filtern
3  mask0 = cv2.bitwise_or(mask_road, mask_white)
4  mask0 = cv2.morphologyEx(mask0, cv2.MORPH_CLOSE, kernel)
5  mask0 = cv2.morphologyEx(mask0, cv2.MORPH_OPEN, kernel)
6  contours, _ = cv2.findContours(mask0, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
7  if contours:
8      largest_contour = max(contours, key=cv2.contourArea)
9      x, y, w, h = cv2.boundingRect(largest_contour)
10     center_x = int(x + w / 2)
11     error = (x_set - center_x) / float(max(1, width))      # Fehler normiert an
   Bildbreite
12     mask = np.zeros((height, width), dtype=np.uint8)
13     cv2.drawContours(mask, [largest_contour], -1, 255, -1)
14     ... # (Berechnung der Maskenabdeckung)
15     self._store_valid_values(error, mask, self.vision_mask_coverage)
16     return error, mask

```

Listing 16: Fallback-Fehlerberechnung

Der Algorithmus ermittelt die größte Kontur als Straßenbereich und berechnet deren Mittelpunkt. Die normierte Differenz zur Bildmitte ergibt den lateralen Fehler, analog zur YOLO-Methode. Bei Erkennungsverlusten greift die gleiche Decay-Logik wie in der YOLO-Strategie.

Limitierungen: Die Fallback-Strategie ist anfälliger für variierende Lichtverhältnisse und unkonventionelle Straßenbeläge, da sie auf feste Farbschwellenwerte angewiesen ist. Trotz geringerer Präzision gewährleistet sie grundlegende Spurhaltung als Notfallmodus bei YOLO-Ausfall und ist besonders gut für vordefinierte Simulationswege geeignet.

6 Ergebnisse

6.1 Fahrrad Datenblatt

Die folgenden Abschnitte beschreiben eine in Webots aufgebaute Simulation als digitalen Zwilling des realen Testfahrrads. Ziel ist, Masse-, Trägheits- und Reibungsparameter sowie Sensor-/Aktorschnittstellen so realitätsnah zu spiegeln, dass das Fahrverhalten und die Reglerkopplung mit dem Prototyp übereinstimmen. Auf Basis der ODE-Physik in Webots und der Controller-API (C/Python) wird die in der Hardware verwendete Architektur nachgebildet: eine schnelle innere Balance-Schleife und eine langsamere, vision-gestützte MPC-Spurführung. Eine Supervisor-Instanz ermöglicht reproduzierbare Versuchsreihen, Resets und Logging, sodass Parameterstudien und Kalibrierungen unter identischen Bedingungen erfolgen. Damit dient die Simulation als sichere Vorstufe für reale Fahrversuche und als Referenz für die im Anschluss dokumentierten physikalischen Eigenschaften und Regelungsparameter.

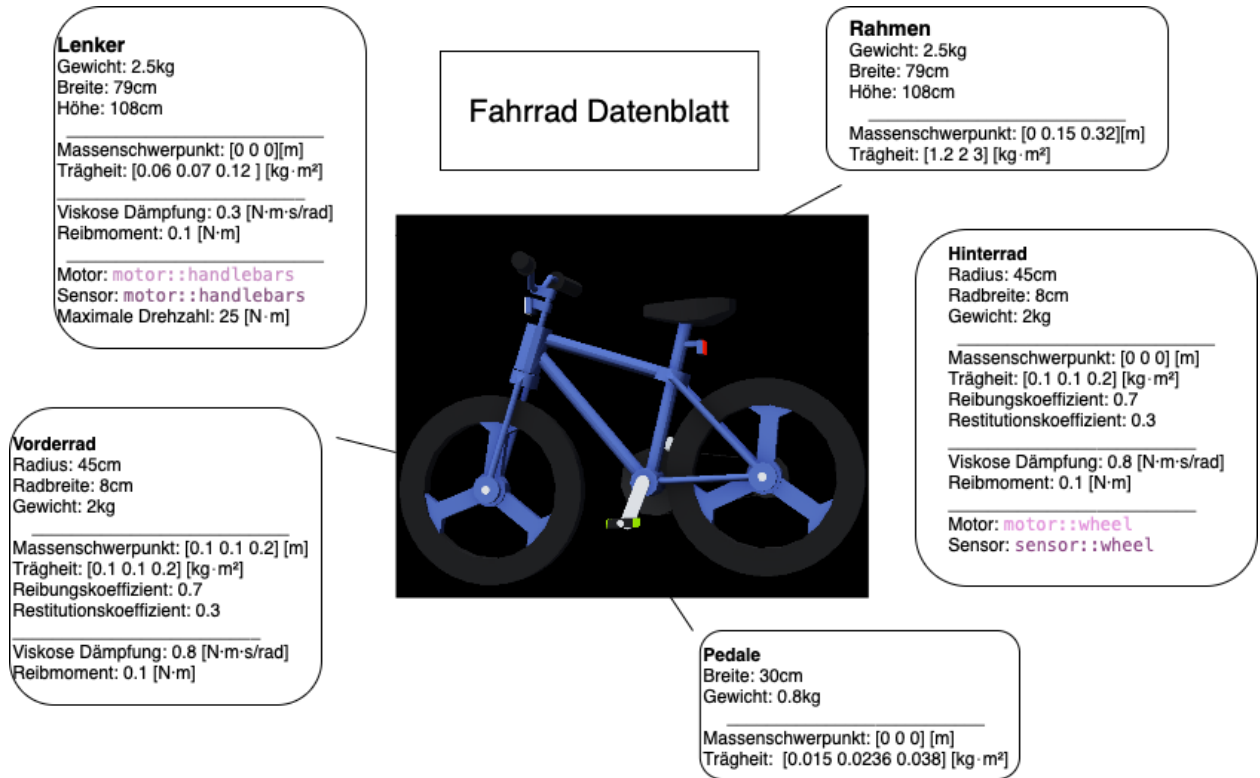


Figure 13: Datenblatt

- **Masse und Schwerpunkt:** Der Rahmen des simulierten Fahrrads wiegt 3,5 kg; zusammen mit den Rädern beträgt die Gesamtmasse ca. 5,1 kg. Der Schwerpunkt liegt etwa 5 cm über dem Boden, was ein stabiles Balanceverhalten begünstigt.
- **Trägheitssensor:** Im Webots-Modell ist der Trägheitssensor als Array in der Form

$$[I_{xx} \ I_{xy} \ I_{xz} \ I_{yx} \ I_{yy} \ I_{yz} \ I_{zx} \ I_{zy} \ I_{zz}]$$

angegeben. Für den Fahrradrahmen ergibt sich ein annähernd diagonal dominanter Trägheitstensor mit $I_{xx} \approx I_{yy} = 0,1$ und $I_{zz} = 0,05 \text{ kg m}^2$. Der kleinere Wert um die Hochachse

(Yaw-Trägheit) erleichtert schnelle Lenkbewegungen; die Off-Diagonaleinträge sind Null, was bedeutet, dass die Hauptträgheitsachsen mit den Achsen des Modells ausgerichtet sind.

- **Räder:** Die beiden Räder haben einen Radius von 5,5 cm und jeweils etwa 0,8 kg Masse. Durch die rotierende Masse entsteht ein kleiner gyroskopischer Stabilisierungs-Effekt (Trägheitsmoment pro Rad ca. $1 \times 10^{-3} \text{ kg m}^2$), der zur Gesamtstabilität beiträgt, aber aufgrund des geringen Werts nur begrenzten Einfluss hat.
- **Reibungsparameter:** Die Radnaben besitzen eine viskose Lagerdämpfung (`dampingConstant` = 0,1) sowie Haftreibung (`staticFriction` = 0,1). Diese Parameter simulieren Lagerverluste und verhindern z. B. ein freies Schwingen oder Durchrutschen der Räder im Stillstand. Zwischen Reifen und Untergrund sorgt ein Coulomb-Reibungskoeffizient von etwa 0,8 für gute Traktion; ein geringer Rückprallkoeffizient von 0,1 begrenzt die Sprungelastizität des Reifenkontakts.

6.2 Teststrecken

6.2.1 Kurvenfahrt

Die Fahrbahn ist als texturierte Ebene von 64m x 64m realisiert; die effektive Spurbreite beträgt an der Referenzlinie etwa 3.6 m. Die Strecke kombiniert eine lange Linkskurve (großer Radius) mit einem engen Rechtsknick und eignet sich damit zur Bewertung von Spurtreue und Balance unter gekrümmter Führung. Sofern nicht anders angegeben, sind für die Balance der *PID*-Satz $K_p=2.0$, $K_i=0.2$, $K_d=0.5$, der zulässige Lenkwinkelbereich $\delta \in [-0.3, 0.3]$ rad sowie $v_{\text{base}}=7 \text{ km/h}$ (Grenzen 5 km/h bis 7 km/h) aktiv. Mechanische Plausibilitätsgrenzen sind mit $\delta_{\text{phys,max}}=1.5 \text{ rad}$ und $\varphi_{\text{roll,max}}=45^\circ$ gesetzt. Für die visuelle Spurführung kann wahlweise YOLO (Konfidenz 0.5) oder eine robuste ROI-Fallback-Erkennung verwendet werden (`roi_top_frac` = 0.5, `roi_height_frac` = 0.06); Lenkbefehle werden durch `max_delta` = 0.001 und `max_steer_cmd` = 0.06 geglättet und begrenzt. Ziel ist die Fahrt von der ersten zur zweiten roten Markierung bei möglichst mittiger Spurhaltung und stabiler Balance.

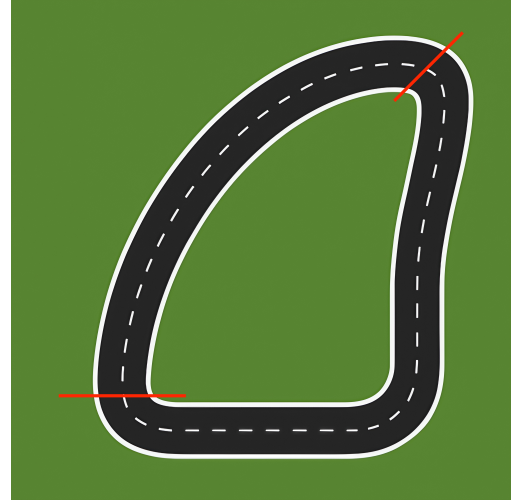


Figure 14: Kurvenfahrt

Einsatz eines vereinfachten Balance-P-Reglers Für eine Baseline wurde der Balance-Regler auf einen reinen P-Anteil gesetzt ($K_p=2.0$, $K_i=0.0$, $K_d=0.0$). Figure 15 zeigt die gemessenen Größen der Fahrt, Figure 16 den zugehörigen ROI-Querfehler und Figure 17 die Trajektorie auf der Strecke.

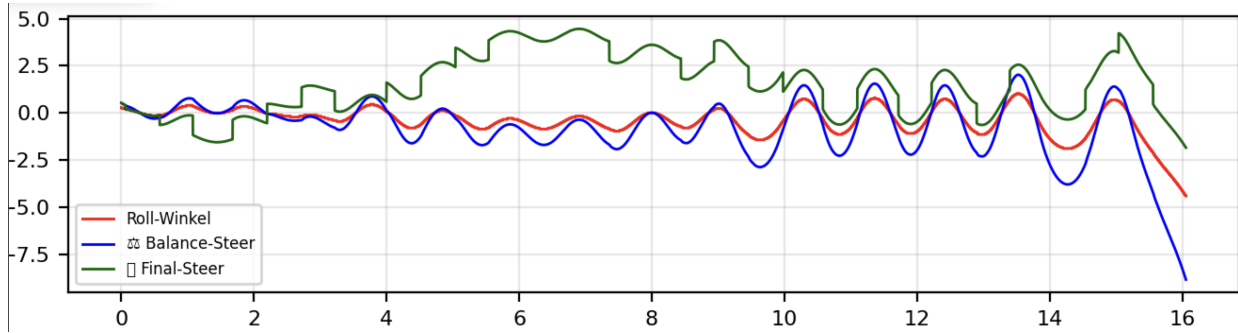


Figure 15: Kurvenfahrt mit reinem P-Regler (Zeitreihen)

Bereits nach wenigen Sekunden setzt ein wachsendes Aufschaukeln ein: Der durch den Rollwinkel induzierte Stellbefehl (`final_steer`) treibt den Lenkeinschlag alternierend nach links/rechts, bis die Phasenreserve erschöpft ist. Infolge der reinen Proportionalrückführung, der Aktuatorbegrenzungen und der unvermeidlichen Latenzen entsteht ein unterdämpftes Verhalten; das Fahrrad kippt nach rund 15 s nach rechts.

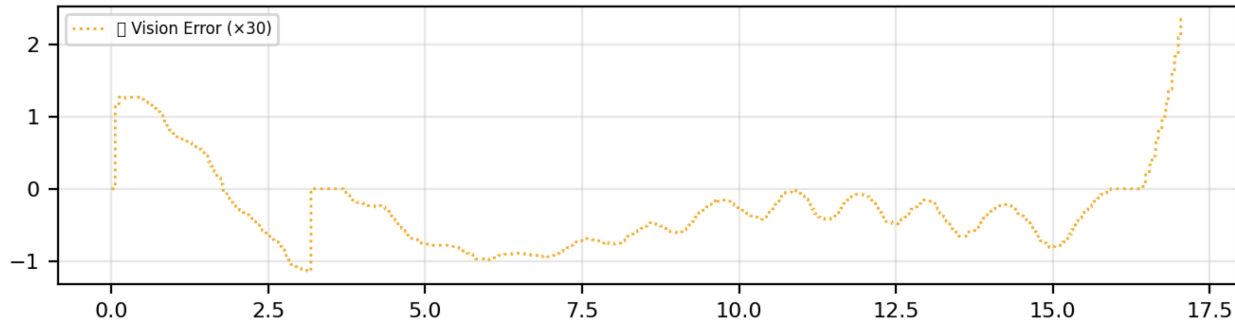


Figure 16: ROI-Querfehler während der P-Regler-Fahrt

Der ROI-Querfehler zeigt eine ausgeprägte Kopplung zum Rollwinkel: Das Pendeln der Balance überträgt sich phasenverschoben auf die seitliche Abweichung. Zur Stabilisierung ist daher ein kleiner Integralanteil (Elimination stationärer Abweichung mit Anti-Windup) und insbesondere ein Derivatanteil (Phasenführung/Dämpfung) erforderlich; in Kombination mit `Rate(max_delta)` und Amplitudenbegrenzung (`max_steer_cmd`) wird das Überschwingen reduziert und ein Kippen verhindert.

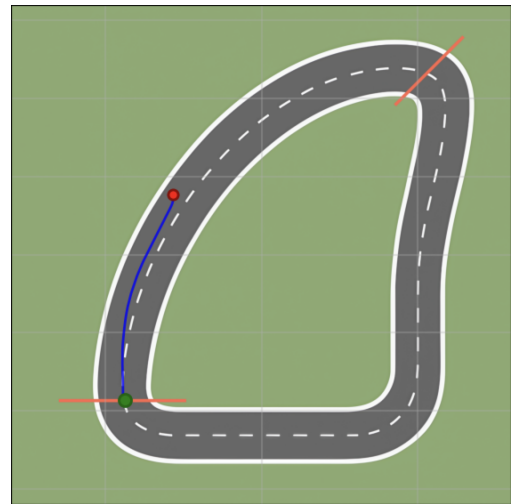


Figure 17: Gefahrene Trajektorie (P-Regler)

Einsatz des erweiterten Balance-PID-Reglers Für diese Versuchsfahrt wurden die erweiterten PID-Parameter $K_p = 2.0$, $K_i = 0.2$ und $K_d = 0.5$ eingesetzt. Die Spurführung erfolgte mittels YOLO-basierter Bildverarbeitung. Während die Parameterwahl eine Verbesserung der Stabilität gegenüber reinen P-Reglern bewirkte, zeigte sich nach etwa 35 s dennoch ein Kippen des Fahrrads, ausgelöst durch abrupte Richtungsänderungen in der vom Vision-Modell vorgegebenen Trajektorie.

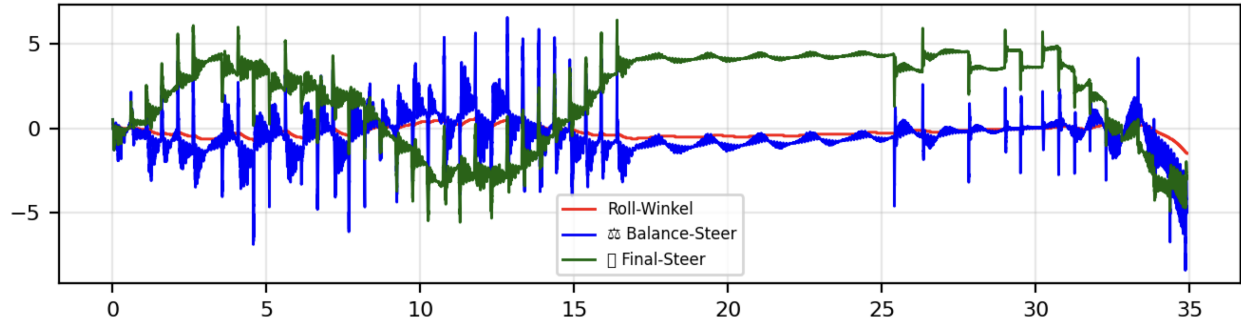


Figure 18: Kurvenfahrt mit PID-Regler und YOLO-Spurführung

Die Fehleranalyse bestätigt, dass die Fahrbahnerkennung zum Zeitpunkt der Versuche noch unzureichend trainiert war. Ab Sekunde 31 steigt der Querfehler abrupt an und fällt kurz darauf wieder ab (Sekunde 34). Dieses unstete Verhalten überträgt sich auf den Rollwinkel, dessen zunehmendes Pendeln schließlich eskaliert und zum Umkippen des Fahrrads nach rechts führt.

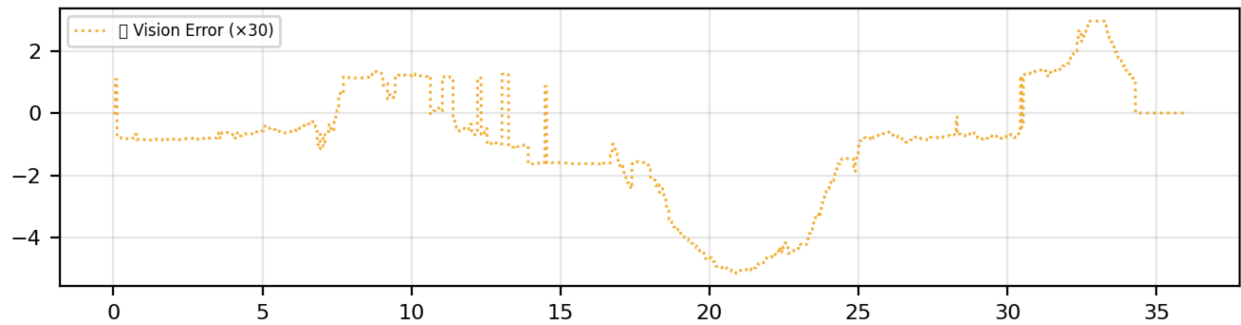


Figure 19: ROI-Querfehler während der PID-Regler-Fahrt (YOLO)

Trotz dieser Limitierungen lässt sich durch die Kombination von Integral- und Differentialanteil eine verbesserte Spurhaltung mit reduzierten Querabweichungen beobachten. Insbesondere werden stationäre Fehler kompensiert und Überschwinger wirksam gedämpft, sodass das Fahrrad über weite Teile der Strecke stabil bleibt – auch bei eingeschränkter Fahrbahnerkennung. Kurz vor dem Ziel versagt die Kompensation jedoch, und das System verliert seine Stabilität. Dies verdeutlicht, dass die Reglererweiterung eine deutliche Verbesserung bewirkt, die Robustheit aber maßgeblich von der Qualität der visuellen Erkennung abhängt.

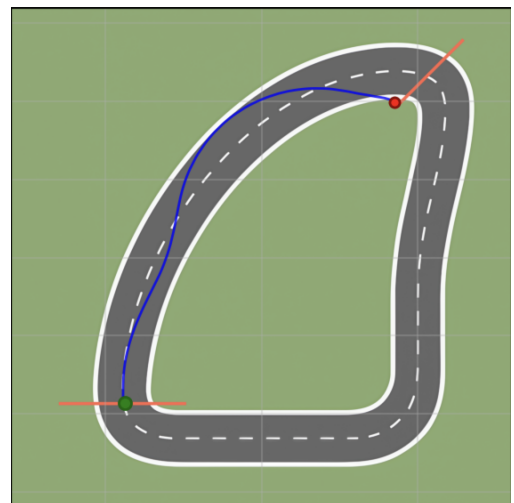


Figure 20: Gefahrene Trajektorie (PID-)

Die Abbildungen 24a und 24b verdeutlichen Limitierungen der *YOLO*-Segmentierung: Abschnitte der Fahrbahn werden nur teilweise bzw. falsch klassifiziert; insbesondere die Trennung zwischen Fahrbahn und Nebenflächen (z.B. Rand-/Grünstreifen) ist inkonsistent. Links (a) ist die Spur weitgehend korrekt maskiert, rechts (b) werden Randbereiche fälschlich als Straße erkannt. Diese Fehlklassifikationen verschieben die geschätzte Spurmitte, erzeugen sprunghafte Querfehler und können instabile Lenkbefehle auslösen.

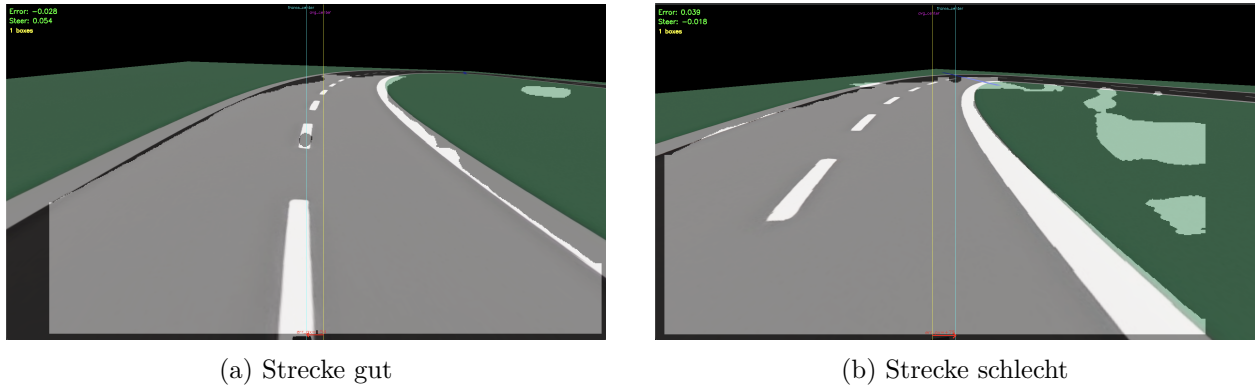


Figure 21: Strecke gut und schlecht nebeneinander.

Einsatz der ROI-Fallback-Erkennung In dieser Versuchsfahrt wurde die ROI-Fallback-Erkennung eingesetzt, um die in Abschnitt 4.3.1 beschriebenen Schwächen der *YOLO*-Segmentierung zu umgehen. Die Pipeline nutzt klassische Bildverarbeitung innerhalb eines definierten Bildausschnitts (ROI): farbbasierte Segmentierung der Fahrbahn und Kantenfilter werden kombiniert, die Spurmittellinie wird aus der ROI-Maske abgeleitet und zeitlich geglättet. Sämtliche Reglerparameter (*PID*) sowie die übrigen Versuchsbedingungen entsprechen exakt denen der *YOLO*-Konfiguration; die beobachteten Unterschiede sind daher allein auf die Wahrnehmung (Erkennung) zurückzuführen.

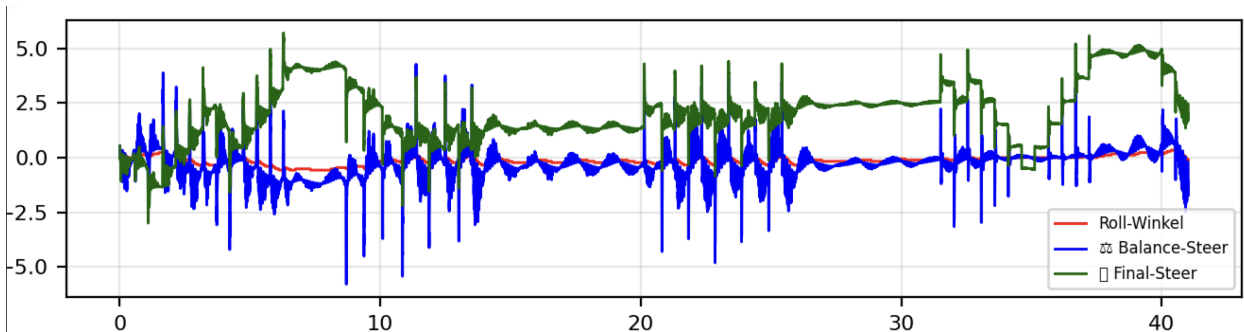


Figure 22: Kurvenfahrt mit PID-Regler und ROI-Fallback-Erkennung

Die Ergebnisse in Figure 22 zeigen einen deutlichen Qualitätsgewinn gegenüber der *YOLO*-Erkennung: sprunghafte Querfehler treten praktisch nicht mehr auf, und das Fahrrad fährt über weite Strecken stabil. Kurzzeitige Anregungen (z. B. zwischen Sekunde 20 und 25) führen zwar zu einer temporären Erhöhung der Stellgrößen, werden jedoch vom PID-Regler sauber kompensiert. Der anfängliche Anstieg des *Final-Steer* ist auf das Einfädeln in die Spur zurückzuführen; nach kurzer Transiente regelt sich das System auf einen kleinen stationären Fehler ein.

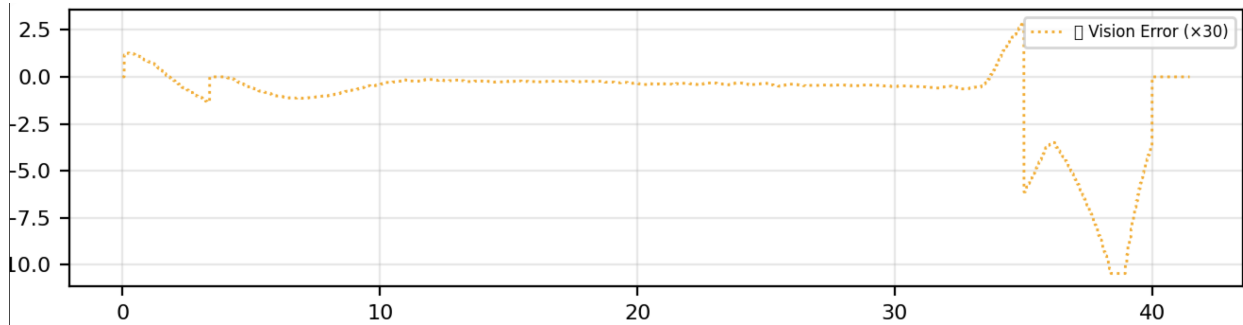


Figure 23: ROI-Querfehler während der PID-Regler-Fahrt (ROI)

Im Fehlerverlauf (Figure 23) ist außerdem ersichtlich, dass ab etwa Sekunde 35 die (rote) Schwelle zur scharfen Kurve überschritten wird und der Querfehler ansteigt. Dieses Verhalten ist konsistent mit den in der Regelkette gesetzten Lenkwinkel- und Ratenbegrenzungen (`max_steer_cmd`, `max_delta`): Bei hoher Streckenkrümmung gerät der Regler kurzzeitig in Sättigung, bevor der Fehler wieder abgebaut wird. Die zugehörige Trajektorie (Figure 24) belegt dennoch eine insgesamt mittige und ruhige Spurführung; Abweichungen konzentrieren sich auf den Scheitel der Kurve und bleiben begrenzt.

Fazit. Die ROI-Fallback-Erkennung erhöht die Robustheit der Spurführung, indem sie Ausreißer in der visuellen Messung reduziert und so die Kopplung zum Balance-Regelkreis entlastet. Ihre Wirksamkeit ist allerdings an Szenen mit gut trennbaren Fahrbahn-/Nebenflächen gebunden; für variable Beleuchtung, Texturen oder komplexe Hintergründe ist eine verbesserte (oder zusätzlich trainierte) lernbasierte Segmentierung weiterhin wünschenswert.

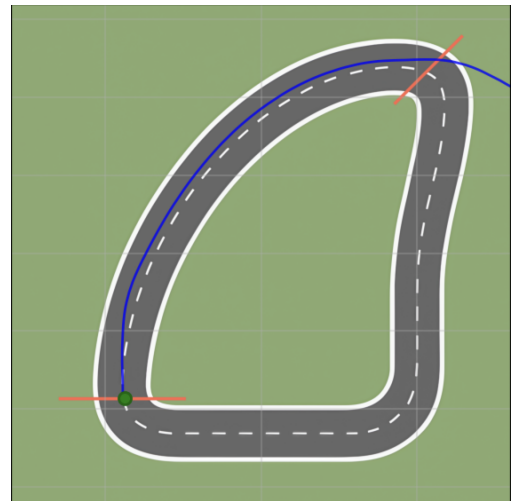


Figure 24: Gefahrene Trajektorie (PID-Regler, ROI)

6.2.2 S-Kurve

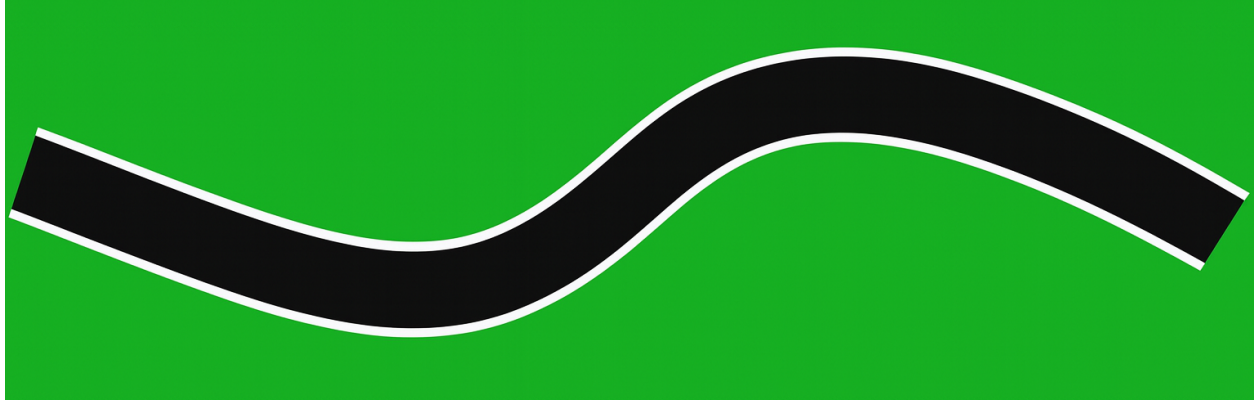


Figure 25: S-Kurve

Die Versuchsstrecke besteht aus einer etwa 100 m langen S-Kurve mit einer Fahrbahnbreite von rund 5,5 m. Das Szenario dient der Bewertung der Kopplung aus innerer Balance-Regelung (PID) und äußerer Spurführungsregelung (visionbasierter MPC) unter schnellem Krümmungswechsel. Für den Balance-PID wurden in diesem Versuch $K_p = 2.0$, $K_i = 0.05$ und $K_d = 0.2$ verwendet; im Vergleich zu anderen Strecken erwies sich hier eine reduzierte Gewichtung von Integral- und Derivatanteil als ausreichend, da die Balance an den Kurveneingängen sonst zu stark angeregt wird. Die innere Schleife arbeitet mit 2 ms Abtastzeit, die Vision-/MPC-Schleife mit 50 ms (20 Hz).

Einsatz des erweiterten Balance-PID mit YOLO-basierter Spurführung Wie bereits in den vorhergehenden Abschnitten gezeigt, ist die YOLOv8-Segmentierung die zentrale Wahrnehmungskomponente der äußeren Schleife. In der S-Kurve zeigt sich jedoch, dass die Segmentierung beim ersten starken Krümmungswechsel zeitweise unzuverlässig ist. Figure 26 illustriert, dass das System die Fahrbahn bereits nach der ersten Kurve partiell verliert; der MPC erhält damit nur intermittierend valide Referenzen. Formal lässt sich die Lenkvorgabe als

$$u_{\text{steer}}(t) = u_{\text{bal}}(t) + g_{\text{vis}}(t) u_{\text{mpc}}(t)$$

schreiben, wobei u_{bal} der schnelle Balance-Anteil, u_{mpc} die Vision-/MPC-Vorgabe und $g_{\text{vis}} \in [0, 1]$ ein Qualitätsgewicht ist. Fällt die Segmentierung aus, wird effektiv $g_{\text{vis}} \approx 0$ und die Fahrtrichtung wird nur noch durch den Balance-Regler gehalten.

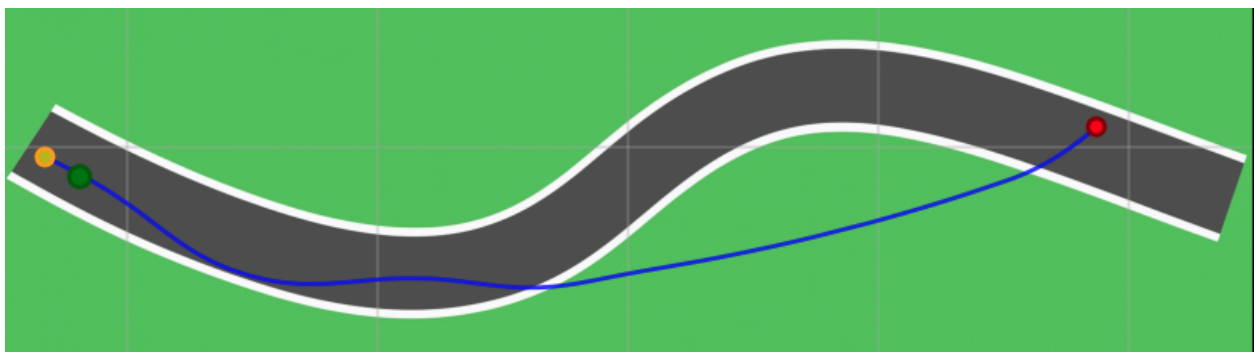


Figure 26: S-Kurve mit YOLO-Regler

Die Zeitverläufe in Figure 27 zeigen zu Versuchsbeginn eine unnötig starke Rechtsabbiegung mit anschließender Gegenlenkung nach links (um ca. 15 s). Diese Sequenz ist kein Folgeeffekt der Balance, sondern ein Artefakt der unstetigen Spurreferenz: das Segmentierungsmodell liefert kurzzeitig inkonsistente Mittellinien, die der MPC korrektiv verfolgt. Zwischen etwa 25 s und 40 s fällt die Spurdetektion stellenweise aus; ab ~ 30 s ist u_{mpc} mehrfach Null, sodass im **final_steer** praktisch nur u_{bal} wirksam bleibt. Erst ab ca. 45 s wird die Spur wieder sicher akquiriert und der MPC fordert erneut ein Rechtsabbiegen an.

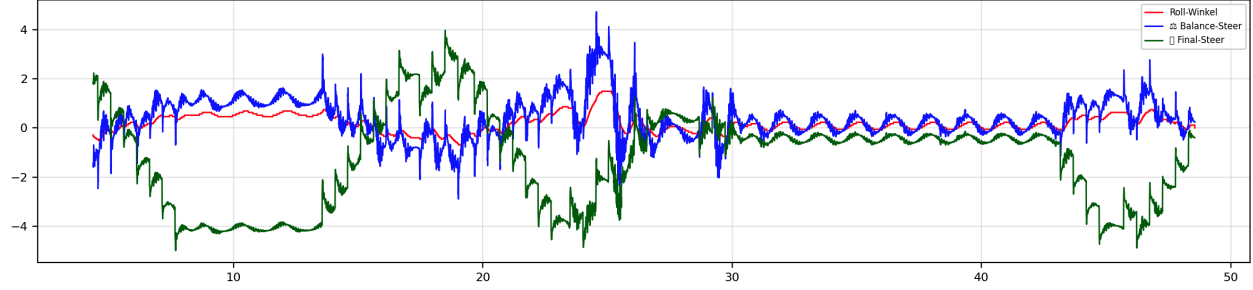


Figure 27: S-Kurve: ausgewählte Zeitreihen (Rollwinkel, Balance-Soll, **final_steer**)

Der Fehlerplot in Figure 28 bestätigt diese Diagnose: Der visionbasierte Querfehler $\tilde{e}_y$ ist im Intervall 25 s–40 s „nahe Null“ *nicht* aufgrund guter Spurhaltung, sondern weil kein verwertbares Signal vorliegt (Maskenfragmentierung bzw. Konfidenz < Schwellwert). Außerhalb der Aussetzer ist die Spurhaltung erkennbar hochfrequent und weist große Spitzen auf. Diese schnellen Lenkänderungen regen den Balance-Regler an (Kopplung zwischen Lenkdynamik und Rollwinkel), was die Rolldämpfung belastet und die Reserven gegenüber Störungen reduziert.

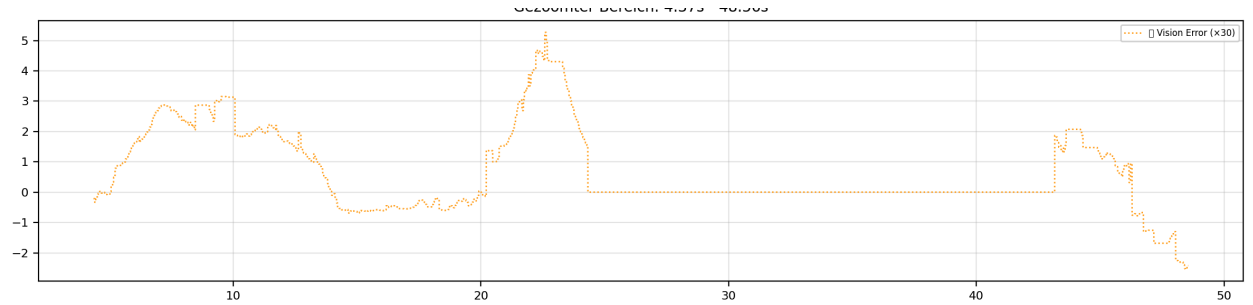


Figure 28: S-Kurve: visionbasierter Querfehler $\tilde{e}_y$

Fazit: Über beide Strecken hinweg (S-Kurve, Kurvenfahrt) zeigt die YOLO-basierte Spurführung die gleiche Schwäche: In stark gekrümmten Abschnitten liefert die Segmentierung intermittierend oder verspätet valide Spurmessungen. In der S-Kurve äußert sich dies als unnötige Anfangskorrektur (Rechts→Links, ~ 15 s) sowie als scheinbar „guter“ Querfehler nahe Null zwischen 25–40 s, der tatsächlich auf Aussetzer der Spurerkennung zurückgeht (vgl. Figure 26, Figure 27, Figure 28). In der Kurvenfahrt tritt das gleiche Muster ausgeprägter auf: sprunghafter Querfehleranstieg ab ~ 31 s und nachfolgendes Kippen trotz PID-Dämpfung, ausgelöst durch inkonsistente bzw. fehlerhafte Trajektorienvorgaben (Figure 18, Figure 19, Figure 20).

Regelungstechnisch führt die unstete Spurreferenz zu hochfrequenten Lenksignalen, die den Balancekreis anregen und die Rolldämpfung reduzieren. Demgegenüber zeigt der ROI-Fallback in

beiden Szenarien eine deutlich ruhigere Dynamik: Die Trajektorien bleiben mittig, Aussetzer entfallen, und der Querfehler weist die erwartete S-Kurven-Signatur ohne impulsartige Ausreißer auf (Figure 29, Figure 30, Figure 31).

Einsatz des erweiterten Balance-PID-Reglers mit ROI-Fallback-Erkennung

Figure 29 zeigt gegenüber der YOLO-Variante eine deutlich stabilere Spurhaltung über die gesamte S-Kurve. Die Trajektorie bleibt näher an der Spurmitte, unnötige großamplitudige Korrekturen entfallen weitgehend. Lediglich zu Beginn ist – analog zur YOLO-Fahrt – ein kurzer Rechts → Links-Korrekturzug zu erkennen, der als Einschwingvorgang der äußeren Regelung zu werten ist. Danach verläuft die Fahrt konsistent und ohne Abreißen der Spurinformation.

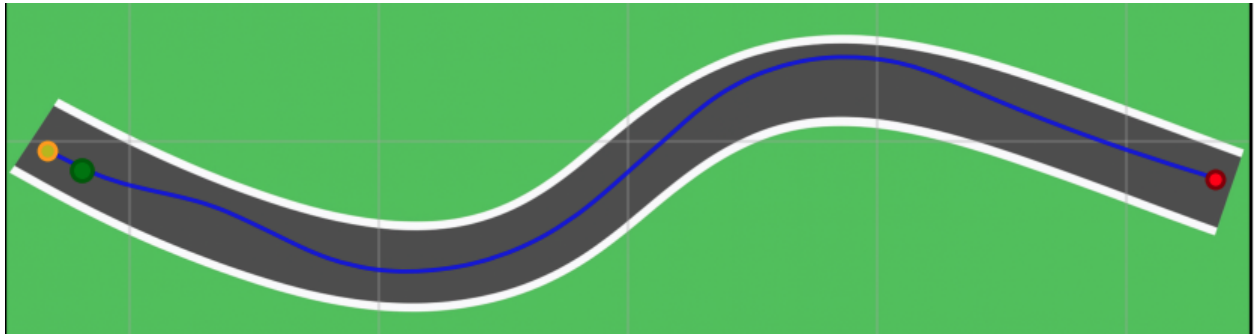


Figure 29: S-Kurve mit ROI-Regler

Die Zeitreihen in Figure 30 belegen dies während des Richtungswechsels zwischen ca. 25–35 s: Der Lenkbedarf steigt erwartungsgemäß am Krümmungsmaximum an, zugleich bleibt der `final_steer` frei von hochfrequenten Spitzen. Um etwa 35 s ist ein temporär größeres Pendeln des Rollwinkels sichtbar, das jedoch begrenzt bleibt und nach Passieren des zweiten Kurvenbogens rasch abklingt (wiederhergestellte Dämpfung, keine Aufschaukelung). Insgesamt resultiert eine klarere Kopplung zwischen Lenkbefehl und Fahrzeuglage ohne die bei YOLO beobachteten Artefakte.

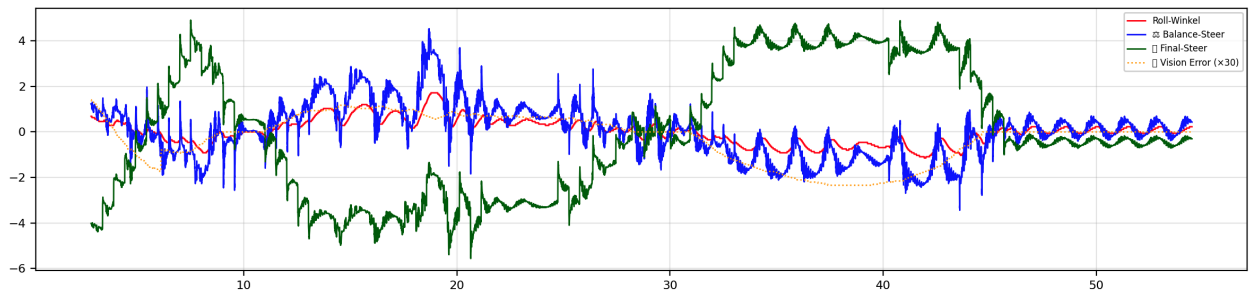


Figure 30: S-Kurve mit ROI-Regler Daten

Der Querfehlerverlauf in Figure 31 ist deutlich glatter und zeigt die erwartete S-Kurven-Signatur mit wohldefiniertem Vorzeichenwechsel in den beiden Bögen. Im Gegensatz zur YOLO-Fahrt treten keine Null-Plateaus durch Signalaussetzer und kaum impulsartige Ausreißer auf; die Varianz des Fehlers ist reduziert, und das Anfangsbias entspricht der kurzfristigen Suche nach der Spurmitte. Das ruhigere Fehlersignal führt zu geringerer Anregung des Balancekreises und damit zu einer robusteren Gesamtfahrt.

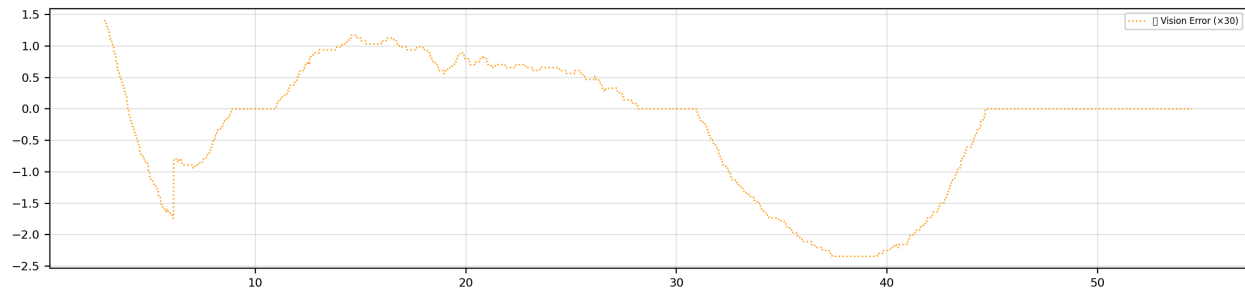


Figure 31: S-Kurve mit ROI-Regler Error

Fazit: Mit ROI-Fallback verbessert sich die Spurtreue über die gesamte Strecke, die Zahl und Amplitude unnötiger Korrekturen sinkt, Aussetzer der Spurinformation werden vermieden und die Balance bleibt trotz Krümmungswechseln gut beherrschbar. Die Resultate deuten auf eine höhere Robustheit der Regelkette in kurvigen Abschnitten hin.

6.2.3 Gerade Strecke mit Seitenwind

Die Versuchsumgebung ist eine $64\text{ m} \times 64\text{ m}$ große Karte mit einer $9,3\text{ m}$ breiten, geraden Fahrbahn. Entlang der Geraden sind drei etwa 20 m lange Segmente definiert (vgl. Figure 32), die als Windkanäle mit seitlichem Anströmfeld modelliert sind. Die Windrichtung wechselt zwischen den Segmenten (rechts $\rightarrow$ links $\rightarrow$ rechts). Die Fahrgeschwindigkeit wird konstant auf 7 km/h geregelt, die Reifenbreite beträgt 2 cm (im Gegensatz zu 6 cm bei der kombinierten Regelung). Die Kopplung zum Strömungsfeld erfolgt über die in *Webots* aktivierten aerodynamischen Drag-/Lift-Modelle; die Seitenwinde werden durch drei **Fluid-Volumina** mit Luftparametern $\rho = 1,225\text{ kg m}^{-3}$ und $\mu = 1,81 \times 10^{-5}\text{ Pas}$ realisiert, jeweils mit einer achsenparallelen **boundingObject**-Box von $64 \times 21 \times 20\text{ m}$ und den Translationen $(0, 21, 0)$, $(0, 1,86, 0)$ und $(0, -19,33, 0)$. Das Anströmfeld ist stationär und homogen; pro Segment wird **streamVelocity** als $(-v_w, 0, 0)$, $(+v_w, 0, 0)$ bzw. $(-v_w, 0, 0)$ gesetzt, wobei in den Auswertungen $v_w \in \{0,5, 1,0\}\text{ m/s}$ verwendet wird (Stresstests bis 2 m/s sind möglich). Geregelt wird in der inneren Schleife die Fahrzeugbalance (PID, Abtastzeit $\approx 2\text{ ms}$); die äußere Spurführung (YOLO/ROI, $\approx 50\text{ ms}$) hält die Mittellinie und wirkt hier primär als langsame Vorsteuerung. Sofern nicht anders angegeben, verwenden wir die Balance-PID-Parameter $K_p = 2,0$, $K_i = 0,05$ und $K_d = 0,2$ (Baseline).

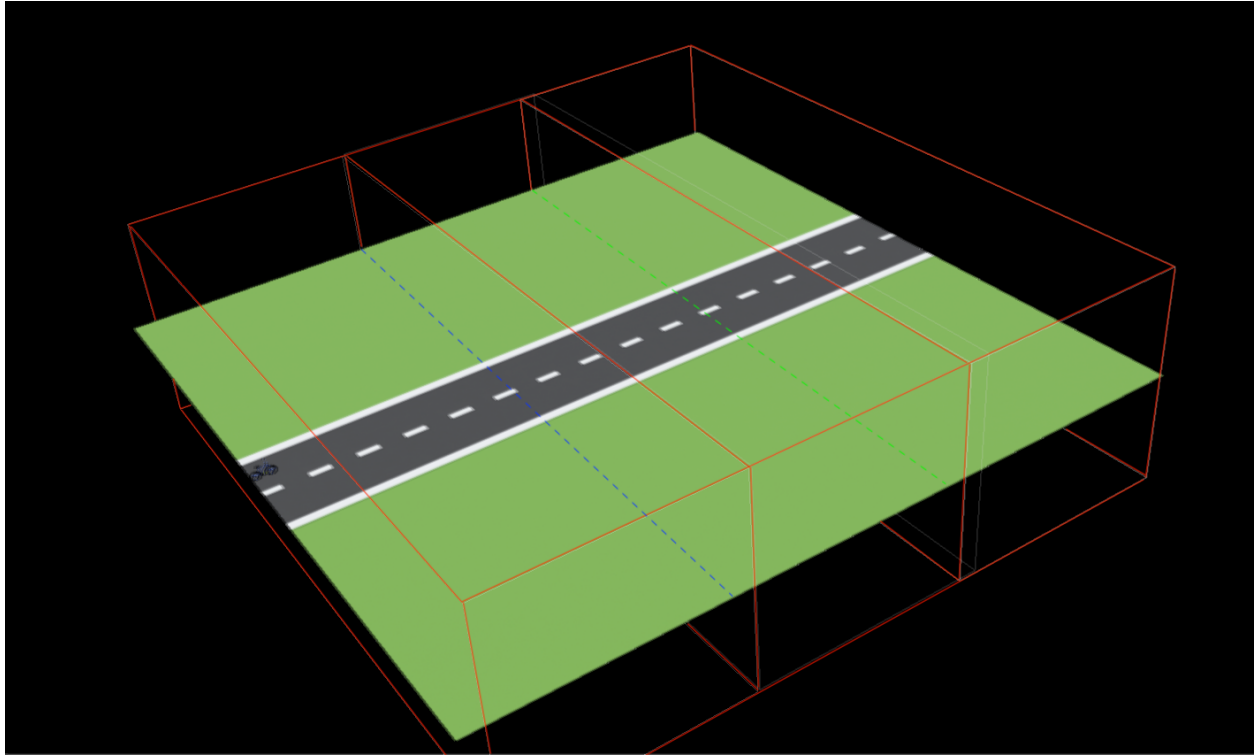


Figure 32: Versuchsaufbau: Gerade Strecke mit drei seitlichen Windkanälen (rote Drahtgitterboxen). Die Fahrbahnmitte dient als Referenz für die Spurführung.

Fall A: Seitenwind $0,5\text{ m/s}$ (Baseline-Regler) Bei moderatem Seitenwind ($0,5\text{ m/s}$) zeigt die Trajektorie in Figure 33 eine seitliche Auslenkung in Windrichtung und anschließend eine Rückkehr zur Ausgangsspur, nachdem das Fahrzeug den dritten Windkanal passiert hat. Die maximale laterale Abweichung beträgt dabei bis zu etwa 3 m , bleibt jedoch ohne Instabilität.



Figure 33: Trajektorie bei 0,5 m/s Seitenwind: Blaue Linie = Fahrzeugspur; Start (grün) und Endpunkt (rot). Die Auslenkung in den ersten beiden Windsegmenten wird nach dem dritten Segment wieder abgebaut.

Die Zeitreihen in Figure 34 verdeutlichen den Zusammenhang zwischen Rollwinkel und Lenkkommandos: Zu Beginn ist eine deutliche Rechtsneigung (positiver Rollwinkel) erkennbar, die der Regler mit gegenläufigen Balance- und Lenkimpulsen kompensiert. Die Signale bleiben beschränkt, der stationäre Versatz wird durch den Integralanteil weitgehend eliminiert.

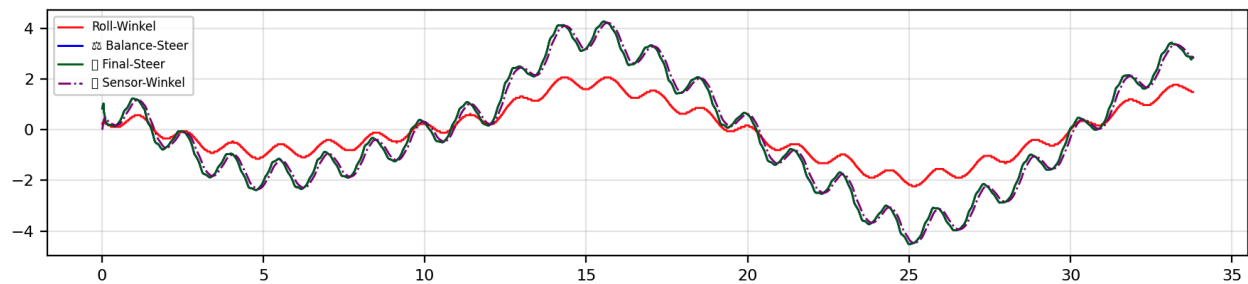


Figure 34: Zeitreihen bei 0,5 m/s Seitenwind: Rollwinkel, Balance-Steer und finales Lenkkommando. Die Richtung der Seitenböe spiegelt sich in der Vorzeichenfolge des Rollwinkels wider; der D-Anteil dämpft die schnelle Nick-/Roll-Dynamik.

Fall B: Seitenwind 1,0 m/s (Baseline-Regler) Erhöhen wir die Windstärke auf 1,0 m/s, vergrößert sich die laterale Abweichung deutlich (Figure 35). Die Zeitreihen in Figure 36 zeigen, dass der Rollwinkel nach etwa 7 s divergiert; das Fahrzeug kippt um. Interpretation: Die Störmomente aus Seitenwind übersteigen mit den Baseline-Gains die stellbare Gegenwirkung, der Regler läuft in Sättigungen (Lenkraten- und Amplitudenlimit), während der I-Anteil die anhaltende Störung nicht schnell genug kompensiert.

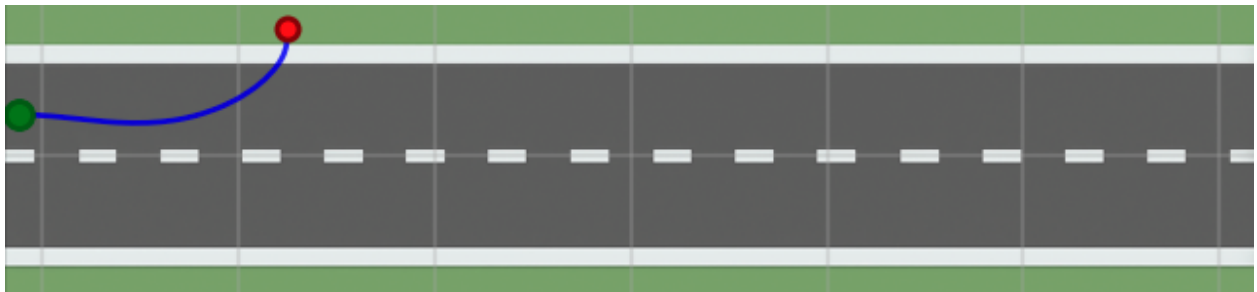


Figure 35: Trajektorie bei 1,0 m/s Seitenwind (Baseline-Gains): starke Abdrift nach rechts mit vorzeitigem Abbruch infolge Umkippens.

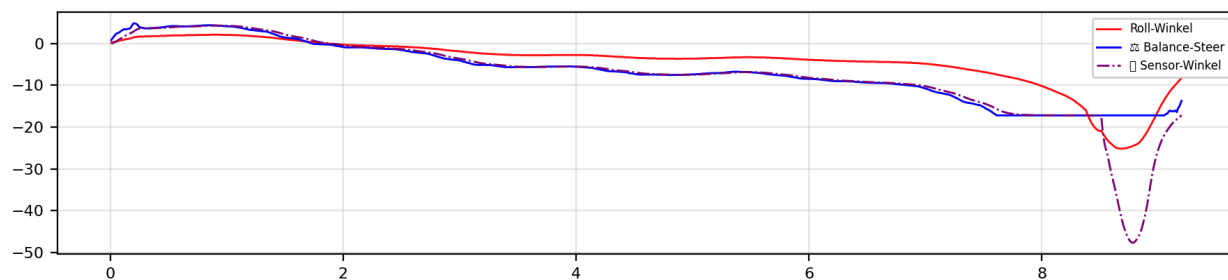


Figure 36: Zeitverlauf bei 1,0 m/s (Baseline-Gains): ansteigende Rollauslenkung und instabiles Verhalten nach ≈ 7 s.

Fall C: Gain-Erhöhung für die Balance-Regelung Zur Verbesserung der Robustheit gegenüber konstanten Seitenkräften erhöhen wir die Balance-Gains auf $K_p = 5,0$, $K_i = 0,2$ und $K_d = 0,4$. Der größere P-Anteil erhöht die Querstabilität (“Steifigkeit” gegenüber Winkelfehlern), der höhere D-Anteil liefert zusätzliche Dämpfung und verhindert Phasennachlauf, und der verstärkte I-Anteil reduziert den stationären Versatz durch die windinduzierte Momentenbilanz.

Die Trajektorie in Figure 37 bleibt nun über die gesamte Strecke beherrscht; die laterale Abweichung wächst nicht mehr unbeschränkt an. Gleichzeitig zeigen die Zeitreihen in Figure 38 ein stärkeres, aber kontrolliertes Schwingen des finalen Lenkkommandos: Die erhöhte Verstärkung erkaufte Stabilität durch höhere Stellaktivität, bleibt aber innerhalb der Aktuatorgrenzen.

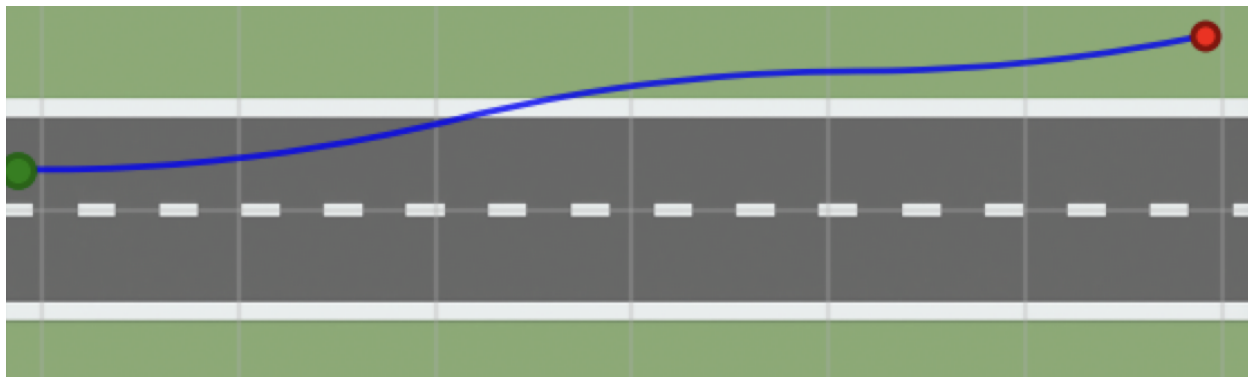


Figure 37: Trajektorie bei 1,0 m/s Seitenwind mit erhöhten Balance-Gains: stabilisierte Fahrt ohne Umkippen; die Abdrift wird begrenzt und anschließend abgebaut.

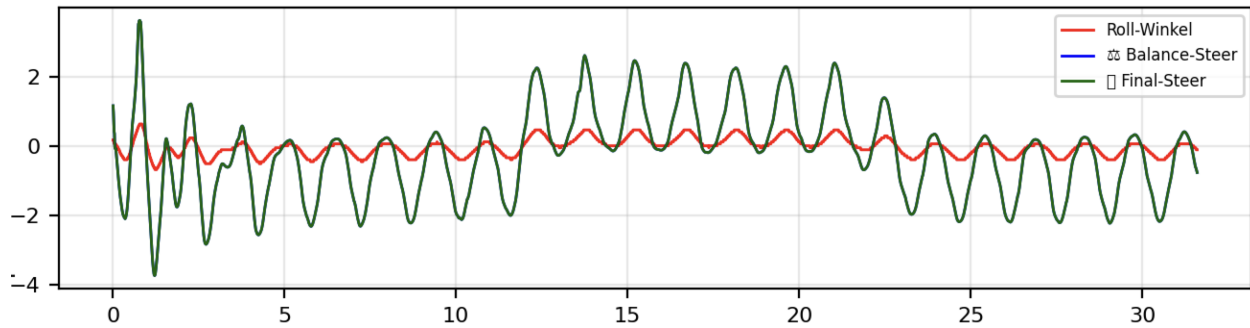


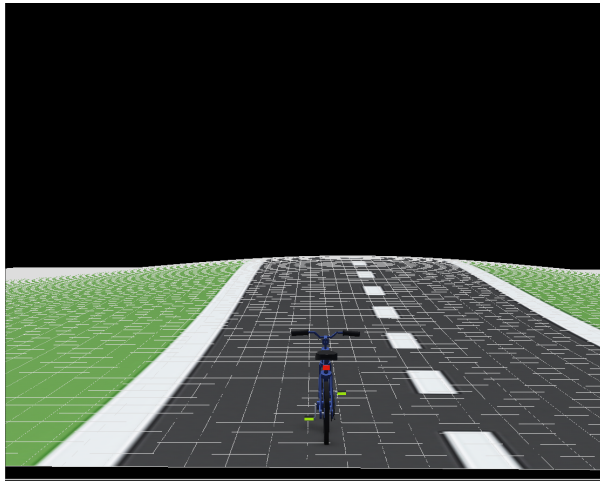
Figure 38: Zeitreihen bei 1,0 m/s mit erhöhten Gains: gedämpfter Rollwinkelverlauf; höhere, aber tolerierbare Lenkaktivität (keine Sättigung, keine Divergenz).

Einordnung und Grenzen Die Ergebnisse bestätigen die in dieser Arbeit gewählte Zwei-Ebenen-Architektur: Die schnelle Balance-Regelung muss Seitenwindstörungen primär aufnehmen; die langsamere Spurführung korrigiert den verbleibenden Versatz. Für moderate Böen ($\approx 0,5 \text{ m/s}$) genügt die Baseline-Parametrierung; bei stärkeren Böen ($\approx 1,0 \text{ m/s}$) ist eine Erhöhung von K_p , K_i und K_d erforderlich, um Umkippen zu vermeiden. Allerdings steigen dabei Stellaufwand und Oszillationstendenzen.

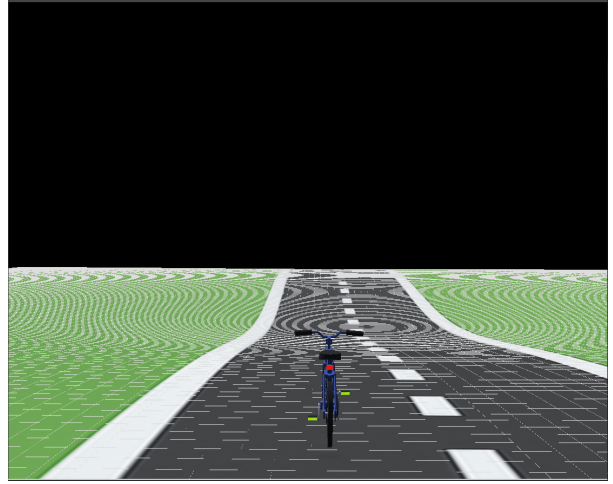
Ab einer gewissen Windstärke liefern reine PID-Anpassungen abnehmenden Zusatznutzen. Sinnvolle Erweiterungen sind (i) Anti-Windup und Lenkratenbegrenzung mit prädiktiver Phasenreserve, (ii) eine Störgrößen-Vorsteuerung (z. B. auf Basis eines Windschätzers oder der Querbeschleunigung), (iii) Gain-Scheduling über Geschwindigkeit und geschätzte Windlast sowie (iv) physikalische Anpassungen (größere Reifenbreite, tieferer Schwerpunkt, höhere Rotationsenergie der Räder). Diese Maßnahmen adressieren die beobachtete Kopplung von Roll- und Querfehler und erhöhen die Robustheit über das reine Tuning hinaus.

6.2.4 Gerade Strecke mit Höhenunterschied

In diesem Abschnitt bewerten wir die Stabilität der Balanceregung auf einer geraden Teststrecke mit definiertem Höhenprofil. Untersucht werden zwei archetypische Szenarien: das Überfahren eines Hügels sowie das Durchfahren einer Senke („Tunnel“), vgl. Figure 39. Soweit nicht anders angegeben, verwenden wir die Baseline-Parameter der Balance-PID-Regelung aus Kapitel 5.2.3. Die Höhendifferenz M wird schrittweise variiert (0,5 m, 1 m, 2 m), um den Einfluss auf Roll- und Spurverhalten systematisch zu untersuchen.



(a) Hügel



(b) Tunnel

Figure 39: Höhenprofile der Teststrecke: Hügel (a) und Senke/Tunnel (b).

Fall A: Senke, $M = 1,0\text{ m}$ Die Trajektorie in Figure 40 zeigt über weite Teile eine nahezu ideale Spurhaltung; geringe laterale Abweichungen treten erst beim Ausfahren aus der Senke auf. Die Zeitreihen in Figure 41 bestätigen dieses Bild: Der Rollwinkel bleibt klein und wird vom Balance-Regler zuverlässig kompensiert. Bei $t \approx 2,5\text{ s}$ ist eine kurzzeitige Amplitudenerhöhung sichtbar, die rasch ausklingt. Da die tiefste Stelle des Profils mittig liegt und die Ränder auf der Referenzhöhe (Nullhöhe) verlaufen, wirken die seitlichen Anteile der Hangabtriebskraft symmetrisch, was die beobachtete Stabilität der Talfahrt erklärt.



Figure 40: Trajektorie durch die Senke (Fall A, $M = 1,0\text{ m}$).

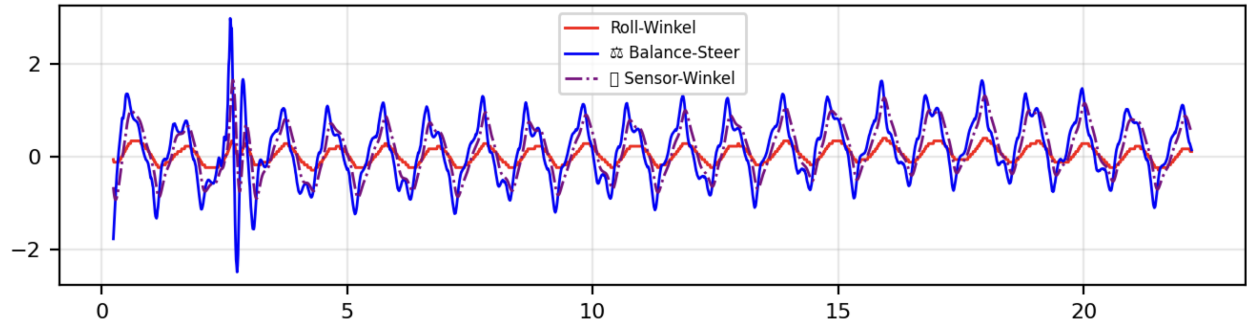


Figure 41: Zeitverläufe Rollwinkel/Balancelenkung (Fall A, $M = 1,0$ m).

Fall B: Senke, $M = 2,0$ m (Baseline-Gewichte) Wird die Höhendifferenz auf $M = 2,0$ m erhöht, verschlechtert sich das Verhalten mit den Baseline-Gewichten deutlich. Figure 42 dokumentiert eine wachsende Querabweichung im letzten Drittel der Strecke. In den Zeitreihen (Figure 43) steigt die Schwingungsamplitude des Rollwinkels progressiv an—ein Muster, das auf Unterdämpfung bei stärkerer Störmomentenerregung durch Längsneigung und Krümmungswechsel hinweist.

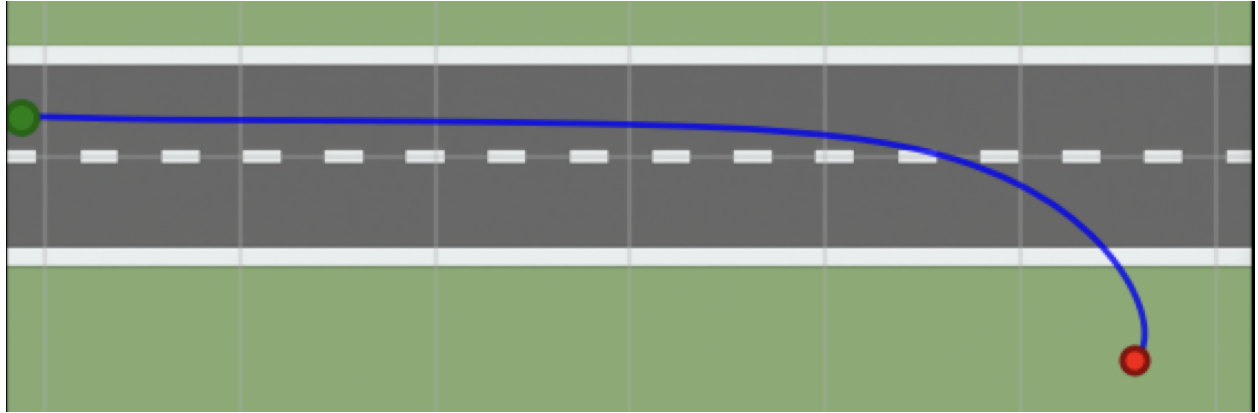


Figure 42: Trajektorie durch die Senke (Fall B, $M = 2,0$ m, Baseline-Gewichte).

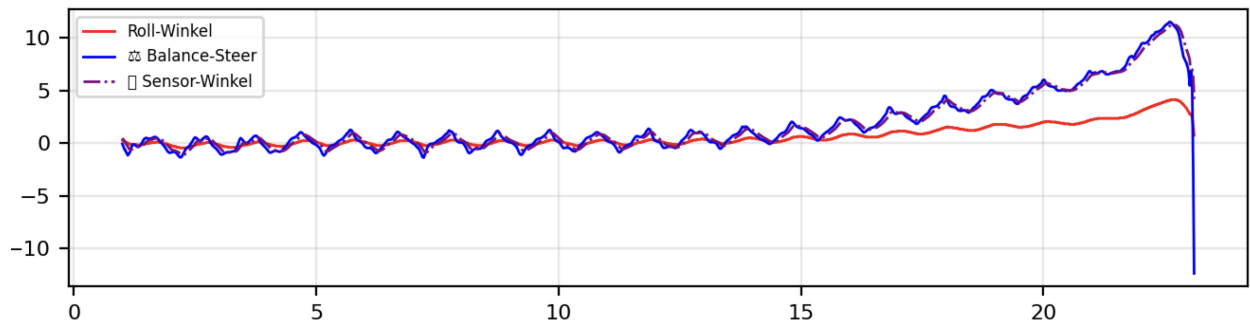


Figure 43: Zeitverläufe Rollwinkel/Balancelenkung (Fall B, $M = 2,0$ m, Baseline-Gewichte).

Fall C: Senke, $M = 2,0$ m mit angepassten Balance-PID-Gewichten Zur Wiederherstellung der Querstabilität wurden die Balance-Gains gezielt erhöht: $K_p = 3,5$, $K_i = 0,2$, $K_d = 0,4$. Die

resultierende Trajektorie (Figure 44) bleibt spurtreu; die Zeitreihen in Figure 45 zeigen gedämpfte Rollschwingungen mit rascher Rückkehr in den stationären Bereich. Die Anpassung adressiert die dominanten Effekte bei großem M : (i) stärkere proportionale Rückführung (K_p) zur Reduktion des Querfehlers und (ii) erhöhte Dämpfung (K_d) zur Phasenführung gegen die durch den Krümmungswechsel angeregten Eigenschwingungen. Der Integrationsanteil bleibt moderat und dient primär der Eliminierung kleiner stationärer Abweichungen (Anti-Windup aktiv).

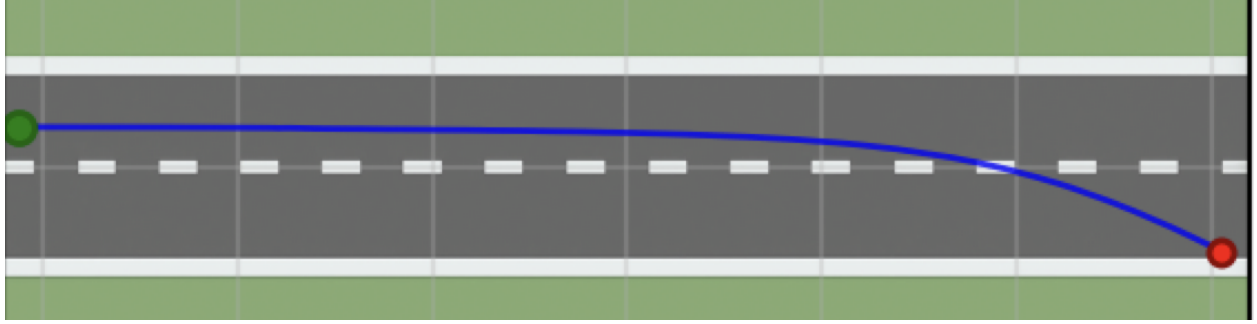


Figure 44: Trajektorie mit retuntem Balance-PID (Fall B, $M = 2,0$ m).

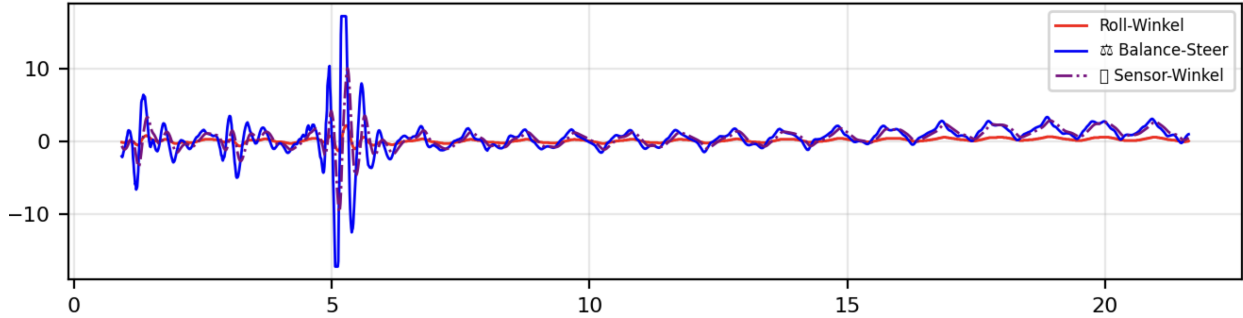


Figure 45: Zeitverläufe mit retuntem Balance-PID (Fall B, $M = 2,0$ m).

Zusammenfassung und Ableitungen Die Ergebnisse zeigen, dass mit zunehmender Höhendifferenz M die erforderlichen Balance-Gains systematisch ansteigen, um Querstabilität und Dämpfung sicherzustellen. In der hier verwendeten Konfiguration (Reifenbreite 2 cm, Streckenlänge 64 m mit Tiefpunkt bei $s = 32$ m) liegt die praktikable Grenze im Fall B bei $M \approx 2$ m; ohne Retuning treten darüber Instabilität bzw. Umkippen auf. Perspektivisch bietet sich eine steigungsadaptive Gain-Schedulierung entlang des Höhenprofils an, um manuelles Retuning zu vermeiden und die Robustheit gegenüber starken Längsneigungen weiter zu erhöhen.

Fall D: Hügel, $M = 2,0\text{ m}$ mit angepassten Balance PID Gewichten

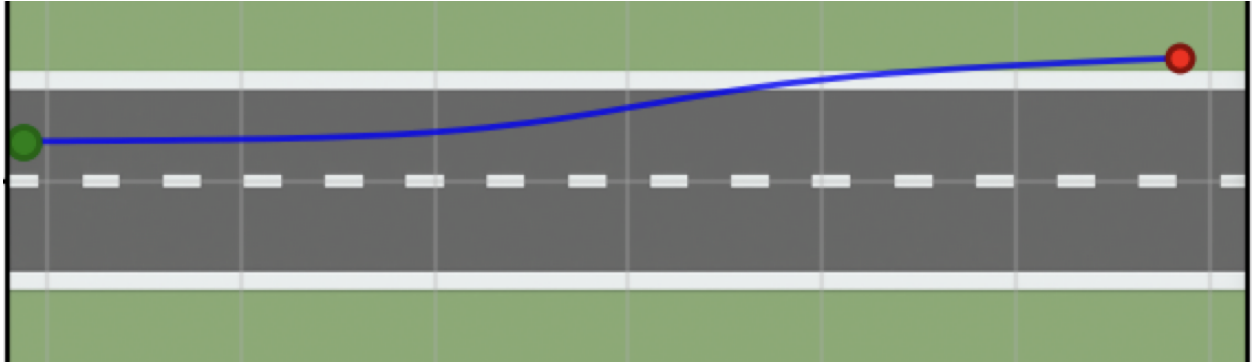


Figure 46: Trajektorie durch den Hügel (Fall D, $M = 2,0$ m).

Wir betrachten nun die Auffahrt auf einen Hügel mit einer maximalen Höhe von $+2$ m. Der Zwischenfall mit $+1$ m zeigt qualitativ das gleiche Verhalten und wird daher ausgelassen. Ziel ist die Beurteilung der Stabilität bei umgekehrtem Höhenprofil im Vergleich zur Senke.

Die Trajektorie in Figure 46 bleibt über weite Strecken spurtreu. Geringe laterale Abweichungen treten vor allem im Bereich des Scheitelpunkts auf. Dort ändert sich die Steigung am stärksten und die Balance Regelung muss zusätzliche Störmomente kompensieren. Die Zeitreihen in Figure 47 stützen diese Beobachtung. Rollwinkel und Balancelenkswinkel überlagern sich weitgehend und zeigen kleine gedämpfte Schwingungen. Kurz vor dem Scheitelpunkt ist eine moderate Amplitudenzunahme sichtbar. Nach der Überfahrt klingen die Schwingungen rasch ab und der Verlauf nähert sich wieder dem stationären Bereich.

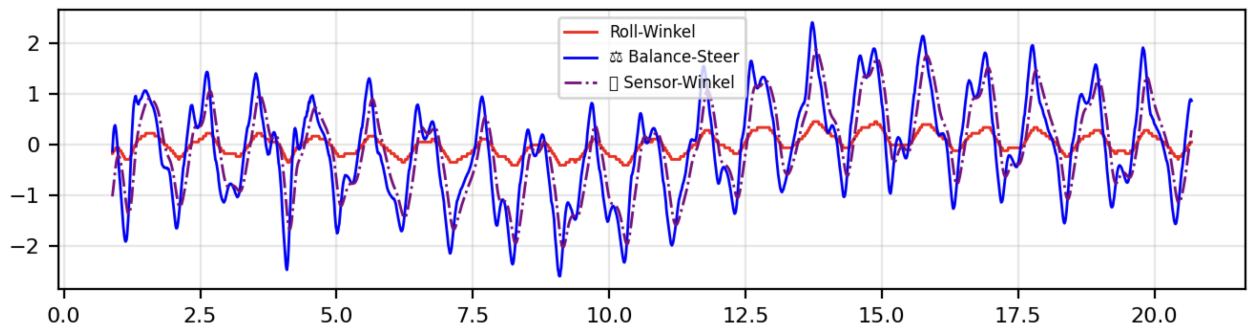


Figure 47: Zeitverläufe mit angepasstem Balance PID (Fall D, $M = 2,0$ m).

Fall E: Hügel, $M = 4,0$ m mit angepassten Balance PID Gewichten Mit den Parametern $K_p = 4,5$, $K_i = 0,2$ und $K_d = 0,4$ lässt sich eine Hügelhöhe von etwa 4 m beherrschen. Als Gütekriterium diente das sichere Vermeiden eines Umkippens. Spurtreue wurde nicht erzwungen.

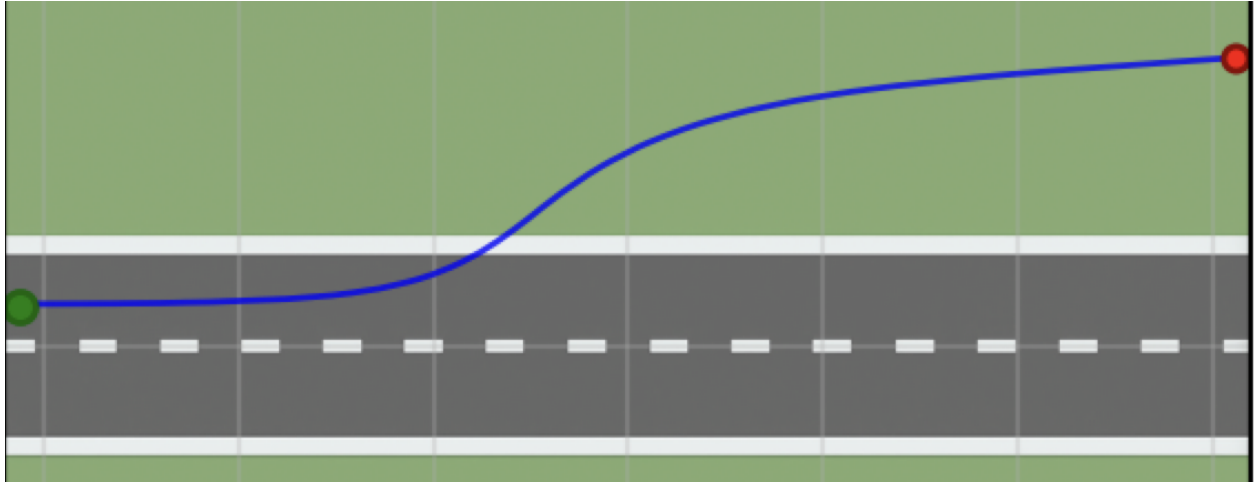


Figure 48: Trajektorie durch den Hügel (Fall E, $M = 4,0$ m).

Die Trajektorie in Figure 48 zeigt eine zentrale Annäherung an den Hügel. Nach rund 20 m setzt eine deutliche Ausweichbewegung ein. Bis zu diesem Punkt wurde bereits ein Höhengewinn von knapp 2 m erreicht. Die Regelung priorisiert in dieser Phase die Aufrechterhaltung der Balance gegenüber der Spurhaltung. Dadurch kann eine laterale Abdrift entstehen und der Scheitel wird mit seitlichem Versatz umfahren. Nach dem Scheitel stabilisiert sich die Fahrt in der anschließenden Abfahrt wieder und der Verlauf nähert sich einer geraden Spur.

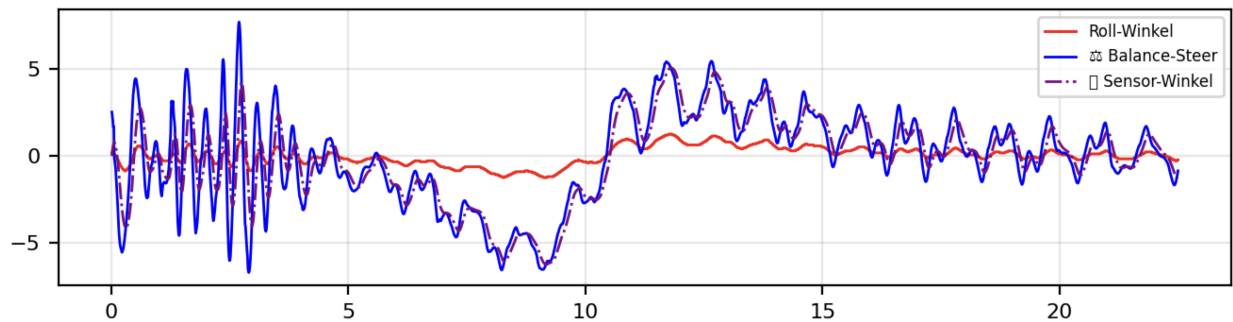


Figure 49: Zeitverläufe mit angepasstem Balance PID (Fall E, $M = 4,0$ m).

Die Zeitreihen in Figure 49 verdeutlichen die Dynamik. Zu Beginn treten stärkere Schwingungen auf, die im Bereich des steilen Anstiegs zunehmen. Kurz nach dem Scheitel gehen die Amplituden spürbar zurück und der Verlauf wird ruhiger. Dieses Muster ist konsistent mit den zusätzlichen Störmomenten durch die größere Längsneigung und den ausgeprägten Krümmungswechsel. Die gewählten Verstärkungen erhöhen die proportionale Rückführung und die Dämpfung. Der Integrationsanteil bleibt moderat und dient vor allem der Beseitigung kleiner stationärer Abweichungen. In Summe wird Balance erreicht, während die Spurhaltung bei $M = 4$ m nur eingeschränkt gewährleistet ist.

Zusammenfassung und Ableitungen Die Experimente mit Senken und Hügeln zeigen übereinstimmend, dass mit wachsender Höhendifferenz M die erforderlichen Balance Gains ansteigen,

damit Querstabilität und Dämpfung erhalten bleiben. Die Teststrecke besitzt eine Länge von 64 m und ein zentrales Extremum bei $s = 32$ m. In dieser Konfiguration beträgt die Reifenbreite 2 cm.

Senken. Bei $M = 1,0$ m bleibt das System mit Baseline Parametern stabil und spurtreu. Beim Übergang aus der Senke treten nur kleine transiente Abweichungen auf, die rasch abklingen. Bei $M = 2,0$ m führt das Baseline Tuning zu wachsender Rollschwingung und lateraler Drift. Ein Retuning mit $K_p = 3,5$, $K_i = 0,2$ und $K_d = 0,4$ stellt die Querstabilität wieder her. Die Talfahrt erweist sich als robust, weil die tiefste Stelle mittig liegt und die seitlichen Anteile der Hangabtriebskraft symmetrisch wirken.

Hügel. Für $M = 2,0$ m zeigt die Auffahrt mit angepassten Gewichten ein stabiles Verhalten. Bei $M = 4,0$ m bleibt das Fahrrad mit $K_p = 4,5$, $K_i = 0,2$ und $K_d = 0,4$ aufrecht, weicht jedoch seitlich aus und umgeht den Scheitelpunkt. Die stärkste Beanspruchung entsteht im Anstieg nahe dem Scheitel, wo sich Steigung und Krümmungswechsel überlagern. Nach dem Scheitel beruhigt sich das System, und die Abfahrt verläuft deutlich ruhiger als der Anstieg.

Gemeinsame Schlussfolgerungen.

- Die erforderlichen Gains wachsen mit M . Der proportionale Anteil reduziert den Querfehler. Der derivative Anteil erhöht die Dämpfung und verbessert die Phasenlage. Der integrale Anteil bleibt moderat und beseitigt kleine stationäre Abweichungen unter aktivem Anti Windup.
- Die kritischen Bereiche liegen an den Extrema des Höhenprofils. Dort ändern sich Neigung und Krümmung am stärksten, was transiente Rollmomente verstärkt und Spurabweichungen auslösen kann.
- In der vorliegenden Konfiguration liegt für Senken die praktikable Grenze ohne Retuning bei etwa $M = 2$ m. Größere Werte erfordern angepasste Gains. Hügel mit $M = 4$ m sind mit erhöhten Gewichten beherrschbar, jedoch mit Einbußen in der Spurhaltung.

Implikationen für die Regelung. Eine steigungsadaptive Gain Scheduling entlang der Strecke ist sinnvoll. Dabei können Vorsteueranteile aus der geschätzten Neigung und ihrer Änderung genutzt werden. Empfehlenswert sind zudem Begrenzungen der Lenkraten und Amplituden sowie eine konsequente Anti Windup Behandlung. Damit lässt sich das Retuning reduzieren, und die Robustheit gegenüber starken Längsneigungen steigt. 

6.3 Rechenzeit & Echtzeitfähigkeit in Webots (Vergleich zum Realfahrrad)

Motivation Dieses Teilkapitel ordnet die in den vorangehenden Abschnitten dargestellten Fahrversuche (Kurvenfahrt, S-Kurve, Senken-/Hügelfahrten) hinsichtlich Rechenzeit und Echtzeitfähigkeit ein. Im Fokus steht die Frage, welche zeitlichen Anforderungen die Simulation in Webots an den Code stellt und wie sich diese Anforderungen vom realen Prototyp unterscheiden.

Rechenzeit in Webots: virtuelle Zeit statt harter Echtzeit Webots trennt streng zwischen *virtueller Zeit* (Simulationszeit) und *Realzeit*. Die Schrittweite der Physik wird durch den `WorldInfo.basicTimeStep` vorgegeben (hier: $T_s = 2$ ms, entsprechend $f_s \approx 500$ Hz). Jeder Aufruf von `wb_robot_step(duration)` lässt genau die angegebene *virtuelle Zeit* vergehen; die tatsächlich benötigte *Rechenzeit* kann jedoch größer oder kleiner als `duration` sein.¹⁰ Damit gilt:

¹⁰Vgl. Webots Reference Manual: *Robot* – Synchronisation und `wb_robot_step`.

- Ist die Berechnung schneller als echtzeitfähig, läuft die Simulation schneller als Realzeit.
- Ist die Berechnung langsamer, *verlangsamt* Webots die Ausführung (Speedometer < 1); die Physikschritte bleiben konsistent, die Simulationsuhr schreitet lediglich langsamer fort.¹¹

Für unsere Konfiguration bedeutet das: Der schnelle Balance-Controller (C, 500 Hz) und die langsamere Visionschleife (Python, ≈ 20 Hz) müssen in Webots *nicht* harte Deadlines erfüllen, solange die Synchronisation aktiv ist (`synchronization = TRUE`). Die Simulationsdynamik bleibt deterministisch; nur die Ausführungsdauer in Realzeit steigt bei hoher Rechenlast.

Implikationen für die hier durchgeführten Versuche Die in Kap. 5 dargestellten Szenarien (u. a. Kurve/S-Kurve mit YOLO sowie ROI-Fallback, Senken/Hügel mit ansteigenden Balance-Gains) profitieren von dieser Entkopplung:

1. **Parametervariationen ohne Echtzeitdruck:** Gain-Sweeps, Hysterese-Tests und Filterstudien (EMA/Spline-Glättung) können mit identischer virtueller Zeitskala reproduziert werden, auch wenn einzelne Schritte in Realzeit länger dauern.
2. **Stabilitätsaussagen bleiben gültig:** Das Kippverhalten, Querfehler-Verlauf und die Dämpfungseigenschaften ergeben sich aus der Physik im festen T_s ; eine temporär langsamere Ausführung verzerrt diese Aussagen nicht.
3. **Architekturpriorität:** Für die Simulation ist eine *klare, modulare Code-Architektur* (Trennung Balance vs. Vision, wohldefinierte Schnittstellen, Logging) wichtiger als die Optimierung auf harte Echtzeitgrenzen.

Warum harte Echtzeit erst auf dem Realfahrrad kritisch wird Die früheren Arbeiten am realen Fahrrad (u. a. Ranz 2021, Zander 2023, Yasin 2024) beschreiben eine kaskadierte PID-Architektur mit typischen Zykluszeiten von 5 ms (Balance), 1 ms (Lenkpositionsregelung) und 150 ms (Geschwindigkeit). Die Implementierung nutzt auf dem BeagleBone Black u. a. PRUs für deterministische PWM/GPIO-Signale und einen Watchdog für Deadline-Überwachung. Hier gilt:

- **Harte Deadlines:** Wird die 5 ms-Balanceperiode verfehlt, akkumuliert sich der Rollfehler; das System kann innerhalb weniger hundert Millisekunden instabil werden.
- **E/A-Latenzen sind real:** IMU, Encoder und Motoransteuerung verursachen nicht verschiebbare Verzögerungen und Jitter, die in der Regelung kompensiert werden müssen (Anti-Windup, Derivat-Anteil, Begrenzungen).
- **Kein „Zeitdilatations“-Sicherheitsnetz:** Im Gegensatz zu Webots lässt sich die Realzeit nicht „verlangsamen“; das Betriebssystem und die Elektronik laufen unverändert weiter.

Folglich ist auf dem realen System eine *echtzeitfähige* Umsetzung (deterministische Scheduling-Strategie, Interrupt-nahe E/A, Worst-Case-Betrachtungen) zwingend; in der Simulation dagegen nicht.

¹¹Vgl. Webots User Guide: *The User Interface* – „Speedometer and Virtual Time“.

Konsequenzen für diese Arbeit und Folgeprojekte Aus den obigen Punkten leiten sich zwei zentrale Empfehlungen ab:

1. **Für Webots-Studien (diese Arbeit):** Beibehalten eines kleinen $T_s = 2\text{ ms}$ für die Physik, saubere Synchronisation (`wb_robot_step`), konsistente Logging-Zeitskalen. Rechenzeitoptimierung ist optional und zielt v. a. auf Durchsatz (mehr Szenarien/Parameter pro Zeiteinheit).
2. **Für den Transfer aufs reale Fahrrad:** Frühzeitige Vorbereitung einer echtzeitfähigen Architektur (PRU/ISR-basierte Ansteuerung, Pufferung von Sensordaten, feste Task-Zyklen, Watchdog, Timing-Telemetry). Die in Webots gefundenen Parameter/Strukturen bleiben nutzbar, müssen jedoch unter harten Zeitrestriktionen validiert werden.

Kurzfazit In Webots bestimmt die virtuelle Zeit (T_s) die physikalische Auflösung; Überschreitungen der Rechenzeit führen lediglich zu einer langsameren Ausführung in Realzeit, nicht zu dynamischen Artefakten. Daher ist für die Simulation keine strikte Echtzeitfähigkeit des Codes nötig. Auf dem realen Fahrrad hingegen sind harte Deadlines konstitutiv für die Stabilität; hier entscheidet Echtzeitfähigkeit unmittelbar über das Regelungsvermögen und die Sicherheit des Systems.

7 Grenzen der Arbeit

8 Zukünftige Arbeiten (Sensor-Fusion, Reinforcement Learning, Hardware-in-the-Loop)

9 Fazit

10 Literaturverzeichnis

References

- [1] D. Andreo, V. Cerone, D. Dzung, and D. Regruto. Experimental results on l_{pv} stabilization of a riderless bicycle. In *Proc. American Control Conference (ACC)*, pages 3124–3129, 2009.
- [2] Bosch Sensortec. Bno055 intelligent 9-axis absolute orientation sensor. <https://www.bosch-sensortec.com>, 2020.
- [3] Concurrent Real-Time. Inverted pendulum, 2024.
- [4] Cyberbotics. Webots – open source robot simulator. <https://cyberbotics.com>, 2025.
- [5] Cyberbotics Ltd. Controller programming. <https://cyberbotics.com/doc/guide/controller-programming>, 2025. Zugriff am 27.08.2025.
- [6] Cyberbotics Ltd. Emitter. <https://cyberbotics.com/doc/reference/emitter>, 2025. Zugriff am 27.08.2025.
- [7] Cyberbotics Ltd. Receiver. <https://cyberbotics.com/doc/reference/receiver>, 2025. Zugriff am 27.08.2025.
- [8] Devantech Ltd. Md49 dual motor driver datasheet. <https://www.robot-electronics.co.uk>, 2019.
- [9] do-mpc development team. Basics of model predictive control, 2024. Accessed 23 August 2025.
- [10] Yassin Ghidaei. Video- und sensorgestütztes autonomes fahren für ein selbstbalancierendes fahrrad. Master’s thesis, Goethe-Universität Frankfurt am Main, 2024.
- [11] Yassin Ghidaei and Amine Karabila. Yolo-based vision control for self-balancing bicycle. Internal project documentation, 2024. Vision controller implementation using YOLO for path detection and MPC for trajectory planning.
- [12] Yasin Giray, Amine Karabila, Jonah Zander, et al. Vision controller readme for self-balancing bicycle. https://github.com/arabila/selfbalancingbicycle/tree/master/Regelsteuerung/controllers/vision_control_py/README.md, 2023. Describes the MPC-based vision controller and its integration with the balance controller.
- [13] GitHub contributors. Webots simulation framework. <https://github.com/cyberbotics/webots>, 2025. Zugriff am 24.08.2025.
- [14] John Hedengren et al. Proportional integral derivative (pid). <https://apmonitor.com/pdc/index.php/Main/ProportionalIntegralDerivative>, 2024.
- [15] Alex Hunziker. Autonomous steering of an electric bicycle based on sensor fusion using model predictive control. Master’s thesis, Goethe-Universität Frankfurt am Main, 2019.
- [16] Infineon Technologies. Bts7960 high-current half-bridge driver. <https://www.infineon.com>, 2018.
- [17] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Yolo by ultralytics. <https://github.com/ultralytics/ultralytics>, 2023.

- [18] K. Kanjanawanishkul. Lqr and mpc controller design and comparison for a stationary self-balancing bicycle robot with a reaction wheel. *Kybernetika*, 54(1):173–191, 2015.
- [19] Amine Karabila. Auf dem weg zum selbstfahrenden fahrrad: Analyse und implementierung einer regelung der balance. Master’s thesis, Goethe-Universität Frankfurt, 2022.
- [20] J. D. G. Kooijman, J. P. Meijaard, Jim M. Papadopoulos, Andy Ruina, and A. L. Schwab. A bicycle can be self-stable without gyroscopic or caster effects. *Science*, 332(6027):339–342, 2011.
- [21] Cyberbotics Ltd. Inertialunit. <https://cyberbotics.com/doc/reference/inertialunit>, 2025. Geräte-Referenz, Zugriff: 24.08.2025.
- [22] J. P. Meijaard, Jim M. Papadopoulos, Andy Ruina, and A. L. Schwab. Linearized dynamics equations for the balance and steer of a bicycle: a benchmark and review. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 463(2084):1955–1982, 2007.
- [23] Olivier Michel. Cyberbotics Ltd. Webots: Professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):39–42, 2004.
- [24] OpenMMLab and Ultralytics. Dive into yolov8: How does this state-of-the-art model work?, 2023. Accessed 23 August 2025.
- [25] Sang-Hyun Park and Seung-Yun Yi. Active Balancing Control for Unmanned Bicycle Using Scissored-pair Control Moment Gyroscope. *International Journal of Control, Automation and Systems*, 18(1):217–224, 2020.
- [26] Jing Pei, Lei Deng, Sen Song, Mingguo Zhao, Shuang Zhang, et al. Towards artificial general intelligence with hybrid tianjic chip architecture. *Nature*, 572(7767):106–111, 2019.
- [27] Niklas Persson, Martin C. Ekström, Mikael Ekström, and Alessandro V. Papadopoulos. Trajectory tracking and stabilisation of a riderless bicycle. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 1859–1866, Indianapolis, IN, USA, 2021. IEEE.
- [28] Anna Ranz. Entwicklung eines selbstbalancierenden fahrradprototyps. Master’s thesis, HWR Berlin, 2021.
- [29] Wikipedia contributors. Image segmentation, 2025. Accessed 23 August 2025.
- [30] Wikipedia contributors. You only look once, 2025. Accessed 23 August 2025.
- [31] Jonah Zander. Entwurf einer Steuerung für ein selbstbalancierendes Fahrrad. Bachelorarbeit, Goethe Universität Frankfurt, 2023.
- [32] Xu Zhao, Wenchao Ding, Yongqi An, Yinglong Du, Tao Yu, Min Li, Ming Tang, and Jinqiao Wang. Fast segment anything. arXiv:2306.12156 [cs.CV], 2023.